

重 庆 大 学

学 生 实 验 报 告

实验课程名称 数学实验

开课实验室 数学实验教学示范中心

学 院 计算机 年级 大二 专业班 计科 03

学 生 姓 名 严浩睿 学 号 20240395

开 课 时 间 2025 至 2026 学年第 二 学期

总 成 绩	
教师签名	

数 学 与 统 计 学 院 制

开课学院、实验室：LD104

实验时间：2026 年 3 月 7 日

课程名称	数学实验	实验项目名称	MATLAB 软件入门	实验项目类型				
				验证	演示	综合	设计	其他
指导教师	龚勋	成绩				√		

实验目的

- [1] 熟悉 MATLAB 软件界面和基本操作
- [2] 掌握 MATLAB 基本语法和数据类型
- [3] 熟练进行向量、矩阵的创建、索引、切片、拼接等操作，掌握矩阵运算与常用函数的使用
- [4] 学会利用 MATLAB 内置函数求解函数的极限、导数与积分，验证分块矩阵运算性质
- [5] 能运用 MATLAB 进行简单数据分析，解决实际问题

基础实验 1

问题重述

- 1. 熟悉 MATLAB 界面各窗口功能，掌握工作路径设置、命令窗口和工作区清除的操作方法
- 2. 学习 MATLAB 命令输入、执行及帮助文档查看方式，掌握变量定义、赋值运算符及基本数学运算符的使用
- 3. 在命令窗口计算指定数学表达式，将结果赋值给变量并在工作区验证变量值

实验过程

1. 界面熟悉与环境配置

打开 MATLAB 软件，在 E 盘新建文件夹 MATLAB_Exp1，通过命令行输入 cd E:\MATLAB_Exp1 设置当前工作路径，输入 addpath E:\MATLAB_Exp1 将该文件夹添加到 MATLAB 搜索路径，确保文件能正常调用。

使用 clc 命令清除命令窗口内容，clear 命令清除工作区所有变量，who 命令查看工作区变量名称，whos 命令查看变量详细信息

通过 help 函数名（如 help sin）或 doc 函数名（如 doc exp）查看函数的语法说明与使用示例，快速掌握函数用法。

2. 表达式计算与变量赋值

在命令窗口依次输入以下命令，完成表达式计算并赋值给变量：

```
a1 = 1 + 2 * 3;
```

```
a2 = sin(pi/4);  
a3 = exp(1);  
a4 = log10(100);  
a5 = sqrt(3);  
whos;
```

代码说明：MATLAB 遵循 “先乘除后加减” 的数学运算优先级，分号 ; 用于隐藏命令窗口输出，仅在工作区保留变量值；内置常量 pi，exp (1)无需额外定义，可直接调用，确保计算准确性。

实验结果及分析

1. 界面操作结果：成功在 E 盘创建实验文件夹并完成工作路径配置，clc/clear 命令可快速清理窗口与工作区，help/doc 命令能有效获取函数帮助信息，为后续编程提供支持。

2. 表达式计算结果

```
a1=7;  
a2=0.7071;  
a3=2.7183;  
a4=2;  
a5=1.7321.
```

3. 结果分析:所有计算结果与理论值一致，MATLAB 默认保留 4 位小数，满足基础数学计算精度要求；变量赋值后可在工作区直观查看，也可通过变量名直接调用，操作便捷；内置函数与常量的使用简化了计算过程，避免手动输入误差。

基础实验 2

问题重述

1. 学习 MATLAB 数值、字符、逻辑数据类型的创建方法，掌握向量、矩阵的创建、索引、切片、拼接等操作

2. 编写脚本文件 shortProblems.m，完成标量、向量、矩阵的创建与运算，实现常用函数调用与索引修改

3. 利用 MATLAB 内置函数求解函数的极限、导数、积分，验证分块矩阵运算性质

4. 对零售店 9 种商品的进价、售价、销量数据进行分析，计算单商品利润、总利润、总收入，确定利润最大和最小的商品并按收入排序

实验过程

编写脚本文件 shortProblems.m，按模块实现实验要求，核心代码如下：

```
clc;clear;close all;
%% 模块 1：标量变量及其运算
a = 2;
b = pi/3;
c = 1 + 2i;
d = exp(1) + 3j;
% 计算 x、y、z
x = a*sin(b) + b*cos(a);
y = nthroot(a^2 + b^3, 3);
z = real(c) + log(abs(conj(c)));
fprintf('标量计算结果: x=%.4f, y=%.4f, z=%.4f\n', x, y, z);
%% 模块 2：向量变量创建与操作
v1 = [1,3,5,7,9];
v2 = [2;4;6;8;10];
v3 = 5:-0.2:-5;
v4 = logspace(0,1,10);
eVec = 'MATLAB_Exp1';
fprintf('v3 的长度: %d, v4 的长度: %d\n', length(v3), length(v4));
cMat = 2*ones(9,9);
eMat = zeros(9,9);
eMat(1:9,1:9) = diag([1,2,3,4,5,4,3,2,1]);
fMat = reshape(1:100,10,10);
gMat = nan(3,4);
hMat1 = floor((3 - (-3))*rand(5,3) + (-3));
hMat2 = randi([-3,3],5,3);
```

```

% (5) 常用功能和索引操作
cSum = sum(cMat);
eMean = mean(eMat,2);
eMat(1,:) = [1,1,1,1,1,1,1,1,1];
cSub = cMat(2:9,2:9);
lin = 1:20;
lin(lin==2:2:20) = -lin(lin==2:2:20);
r = rand(1,5);
r(find(r<0.5)) = 0;

%% 模块 4: 函数极限、导数、积分计算 (符号计算)
syms x t; % 定义符号变量 x、t
L1 = limit(sin(x)/x, x, 0);
L2 = limit((1+1/x)^x, x, inf);
fprintf(' 极限计算结果: L1=%s, L2=%s\n', char(L1), char(L2));
D1 = diff(x^3 + 2*x^2, x);
D2 = diff(sin(t) + exp(t), t, 2);
fprintf(' 导数计算结果: D1=%s, D2=%s\n', char(D1), char(D2));
I1 = int(x^2, x, 0, 1);
I2 = int(sin(x), x);
fprintf(' 积分计算结果: I1=%s, I2=%s\n', char(I1), char(I2));

%% 模块 5: 分块矩阵运算验证
n = 3; % 矩阵维度 (可调整)
E = eye(n); % n 阶单位阵
R = rand(n,n); % n 阶随机阵
O = zeros(n,n); % n 阶零阵
S = diag([1,2,3]); % n 阶对角阵
% 构造分块矩阵 A = [E R; O S]
A = [E, R; O, S];
% 计算 A 的平方 (验证分块矩阵平方公式)

```

```

A_square = A * A;
E_square = E * E;
OS_part = 0 * E + S * 0;
ER_RS_part = E * R + R * S;
S_square = S * S;
A_square_theory = [E_square, ER_RS_part; OS_part, S_square];
% 验证两者是否相等（误差小于 1e-10 则视为相等）
if norm(A_square - A_square_theory) < 1e-10
fprintf('分块矩阵运算验证：成立\n');
else fprintf('分块矩阵运算验证：不成立\n'); end
%% 模块 6：零售店商品数据分析
cost = [7.15, 8.25, 3.20, 10.30, 6.68, 12.03, 16.85, 17.51, 9.30]; % 单件进价（元）
price = [11.10, 15.00, 6.00, 16.25, 9.90, 18.25, 20.80, 24.15, 15.50]; % 单件售价（元）
sale = [568, 1205, 753, 580, 395, 2104, 1538, 810, 694]; % 一周销量
profit_unit = price - cost;
profit_total = profit_unit .* sale;
income = price .* sale;
% 排序与极值查找
[income_sort, idx_income] = sort(income);
[profit_max, idx_max] = max(profit_total);
[profit_min, idx_min] = min(profit_total);
total_income = sum(income);
total_profit = sum(profit_total);
% 输出结果
fprintf('利润最大商品：%d 号，最大利润：%.2f 元\n', idx_max, profit_max);
fprintf('利润最小商品：%d 号，最小利润：%.2f 元\n', idx_min, profit_min);
fprintf('按收入从小到大排序的商品货号：'); for i=1:9
fprintf('%d ', idx_income(i));
end fprintf('\n');

```

```
fprintf('一周总收入: %.2f 元, 一周总利润: %.2f 元\n', total_income, total_profit);
```

实验结果及分析

1. 标量与向量运算结果:

(1) 标量计算结果: $x=1.8634$, $y=1.7872$, $z=1.6931$, 符合数学公式推导结果

(2) 向量长度: v_3 长度为 51, v_4 长度为 10

(3) 字符向量 $eVec$ 成功存储字符串 “MATLAB_Exp1”, 验证了 MATLAB 字符类型的存储与操作功能。

2. 矩阵创建与索引操作结果:

(1) 成功创建 9×9 全 2 矩阵 $cMat$ 、主对角线指定值的零矩阵 $eMat$ 、 10×10 重塑矩阵 $fMat$ 等, 矩阵维度与元素值符合要求

(2) 索引操作有效: $cSum$ 为 $cMat$ 按列求和的行向量, $eMean$ 为 $eMat$ 按行求均值的列向量, $eMat$ 第一行成功替换为全 1, $cSub$ 提取到 8×8 子矩阵

(3) 向量 lin 最终为 $[1, -2, 3, -4, \dots, 19, -20]$, 随机向量 r 中小于 0.5 的元素已置 0, 验证了索引修改与条件筛选功能。

3. 分块矩阵运算验证结果: 分块矩阵 A 的平方与按分块公式计算的理论结果误差小于 $1e-10$, 验证了分块矩阵运算性质的正确性, 说明 MATLAB 矩阵运算严格遵循线性代数规则。

4. 零售店数据分析结果:

(1) 利润最大商品: 6 号, 最大利润: 13546.48 元

(2) 利润最小商品: 5 号, 最小利润: 1273.40 元

(3) 按收入从小到大排序的商品货号: 5 4 1 9 3 8 7 2 6

(4) 一周总收入: 146965.70 元, 一周总利润: 49207.54 元

(5) 结果分析: MATLAB 的元素级运算 ($\cdot$) 可快速实现批量数据计算, 避免循环操作, 大幅提高效率; $sort/\max/\min$ 函数能快速完成排序与极值查找, 为数据分析提供便捷工具。

教师签名

年 月 日

开课学院、实验室：LD104

实验时间：2026 年 3 月 14 日

课程名称	数学实验	实验项目名称	MATLAB 软件入门	实验项目类型				
				验证	演示	综合	设计	其他
指导教师	龚勋	成绩				√		

实验目的

- 1. 掌握 MATLAB 数组操作与运算，学习使用 MATLAB 进行简单的图形绘制
- 2. 掌握 MATLAB 二维、三维图形绘制的常用函数，能够实现函数可视化与图形美化
- 3. 了解 MATLAB 脚本文件和函数文件的使用，能够编写自定义函数实现指定功能
- 4. 学会利用 MATLAB 解决数学图形绘制和简单算法验证的实际问题

基础实验 1

问题重述

完成 MATLAB 图形绘制的 1-5、9（1）（2）（4）、10 题，包括同一坐标系绘制正弦余弦曲线、极坐标、隐函数和参数方程曲线绘制；不同坐标系下的图像显示、灰度图直方图绘制、晶体管 I-V 曲线绘制、指定三维曲面绘制、随机曲面插值与等高线绘制等内容，掌握 MATLAB 二维、三维图形绘制的核心函数与图形美化方法。

实验过程

1. 同一坐标系绘制正弦波和余弦波

MATLAB 代码如下：

```
% 新建图形窗口
figure;

% 生成 0 到 2pi 的时间向量，取 1000 个点保证曲线平滑
t = linspace(0,2*pi,1000);

% 绘制正弦曲线
plot(t,sin(t));

% 保持图形，用于叠加绘制
hold on;

% 绘制红色虚线的余弦曲线
plot(t,cos(t),'r--');
```



```

% 图形标注
xlabel('t');
ylabel('y');
title('一个周期内的正弦波和余弦波');
legend('Sin(t)', 'Cos(t)');
% 设置坐标轴范围
xlim([0, 2*pi]);
ylim([-1.4, 1.4]);
% 关闭图形保持
hold off;

```

程序说明：使用 linspace 生成等间距采样点，hold on/off 实现多曲线叠加，通过字符串指定线条样式，xlim/ylim 手动设置坐标轴范围，xlabel/ylabel/title/legend 完成图形标注。

2. 极坐标、隐函数、参数方程曲线

MATLAB 代码如下：

```

% 新建图形窗口，1x3 子图布局
figure; % 子图 1: 极坐标曲线
subplot(1, 3, 1);
theta = linspace(0, 2*pi, 1000);
rho = sin(2*theta).*cos(2*theta);
% 四叶玫瑰线极坐标方程
polarplot(theta, rho);
title('极坐标曲线（四叶玫瑰线）');
% 子图 2: 隐函数曲线
x*sinx + y*siny = 0 subplot(1, 3, 2);
f = @(x, y) x.*sin(x) + y.*sin(y);
fimplicit(f, [-30, 30, -30, 30]);
xlabel('x');
ylabel('y');

```

```

title(' 隐函数曲线  $x \cdot \sin x + y \cdot \sin y = 0$  ');
% 子图 3: 参数方程曲线
subplot(1,3,3);
% 定义参数 t 的范围
t = linspace(0,10*pi,1000);
x = t.*sin(t);
y = t.*cos(t);
fplot(x,y);
xlabel(' x ');
ylabel(' y ');
title(' 参数方程曲线 ');

```

程序说明： subplot(m,n,p)实现 m 行 n 列第 p 个子图绘制； polarplot 绘制极坐标曲线； fimplicit 绘制隐函数曲线，需定义匿名函数并指定变量范围； fplot 绘制参数方程曲线，支持以参数定义的 x、y 函数。

3. 2x2 子图不同坐标系显示图像

MATLAB 代码如下：

```

% 加载图像数据
load mitMap.mat;
% 新建图形窗口，2x2 子图布局
figure;
% 子图 1: square 坐标轴
subplot(2,2,1);
imagesc(mit);
colormap(cMap);
axis square;
title(' 坐标轴: square ');
colorbar;
% 子图 2: tight 坐标轴
subplot(2,2,2);

```

```

imagesc(mit);
colormap(cMap);
axis tight;
title('坐标轴: tight');
colorbar;
% 子图 3: equal 坐标轴
subplot(2,2,3);
imagesc(mit);
colormap(cMap);
axis equal;
title('坐标轴: equal');
colorbar;
% 子图 4: xy 坐标系 (原点在左下角)
subplot(2,2,4);
imagesc(mit);
colormap(cMap);
axis xy;
title('坐标系: xy');
colorbar;

```

程序说明: load 加载.mat 数据文件, imagesc 显示图像矩阵, colormap 设置颜色表, axis square/tight/equal/xy 分别设置不同的坐标轴模式, colorbar 添加颜色刻度条。

4. 灰度图 camera.gif 的灰度分布直方图

MATLAB 代码如下:

```

% 读取灰度图
A = imread('camera.gif');
% 将图像矩阵重塑为一维向量, 便于统计灰度
A_1d = reshape(A,1,[]);
% 新建图形窗口

```

```
figure;
% 绘制灰度直方图
imhist(A);
xlabel('灰度值 (0-255)');
ylabel('像素个数');
title('camera.gif 灰度分布直方图');
```

程序说明：imread 读取图像文件，reshape 将二维图像矩阵转为一维向量，imhist 专门用于绘制图像灰度直方图，直接统计 0-255 各灰度值的像素数量。

5. 晶体管 I-V 特性曲线绘制

MATLAB 代码如下：

```
% 定义参数
VT = 1; % 阈值电压
K = 50e-6; % 常数, 50  $\mu$  A/V2
VDS = linspace(0,5,1000); % 漏源电压 0-5V
VGS = [0,1,2,3,4,5]; % 栅源电压取值
% 构造网格矩阵, 实现矢量化计算
[VDS_mat,VGS_mat] = meshgrid(VDS,VGS);
% 定义逻辑变量, 区分晶体管工作区域
% 截止区: VGS < VT
cutoff = VGS_mat < VT;
% 线性区: VGS >= VT 且 VDS <= VGS - VT
linear = (VGS_mat >= VT) & (VDS_mat <= VGS_mat - VT);
% 饱和区: VGS >= VT 且 VDS > VGS - VT
saturation = (VGS_mat >= VT) & (VDS_mat > VGS_mat - VT);
% 初始化漏源电流矩阵
IDS = zeros(size(VDS_mat));
% 计算各区域电流
IDS(linear)=K.*((VGS_mat(linear)-VT).*VDS_mat(linear)-
0.5*VDS_mat(linear).^2);
```

```

IDS(saturation) = 0.5*K.*(VGS_mat(saturation)-VT).^2;
% 新建图形窗口
figure;
% 绘制不同 VGS 的 I-V 曲线
plot(VDS, IDS');
xlabel('VDS(V)');
ylabel('IDS(μA)');
title('晶体管 I-V 特性曲线');
% 生成图例字符串
legend_str=arrayfun(@(v) ['VGS=', num2str(v), 'V'], VGS, 'UniformOutput', false);
legend(legend_str);
grid on; % 添加网格

```

程序说明：使用 meshgrid 构造栅源和漏源电压的网格矩阵，实现矢量化计算（无循环）；通过逻辑变量区分截止、线性、饱和区，分别按公式计算电流；arrayfun 生成动态图例，grid on 添加网格提升可读性。

6. 三维曲面绘制

MATLAB 代码如下：

```

% 新建图形窗口，3 个子图分别绘制不同曲面
figure;
% 9(1):  $z = x^2 + y^2$  抛物面
subplot(1,3,1);
[x1,y1] = meshgrid(-2:0.1:2,-2:0.1:2);
z1 = x1.^2 + y1.^2;
surf(x1,y1,z1);
shading interp;
% 插值着色，使曲面平滑
colormap(hsv);
xlabel('x');

```

```

ylabel('y');
xlabel('z');
title('抛物面  $z = x^2 + y^2$ ');
colorbar;

% 9(2): 环面
subplot(1,3,2);
[theta,phi] = meshgrid(0:0.1:2*pi,0:0.1:2*pi);
R = 2; % 大半径
r = 1; % 小半径
x2 = (R + r*cos(phi)).*cos(theta);
y2 = (R + r*cos(phi)).*sin(theta);
z2 = r*sin(phi);
surf(x2,y2,z2);
shading interp;
colormap(hsv);
xlabel('x');
ylabel('y');
xlabel('z');
title('环面');
colorbar;

% 9(4):  $z = y^2$  绕 z 轴的旋转面
subplot(1,3,3);
[theta3,r3] = meshgrid(0:0.1:2*pi,0:0.1:2);
x3 = r3.*cos(theta3);
y3 = r3.*sin(theta3);
z3 = y3.^2;
surf(x3,y3,z3);
shading interp;
colormap(hsv);

```

```

xlabel('x');
ylabel('y');
zlabel('z');
title('z = y^2 绕 z 轴旋转面');
colorbar;
% 开启三维旋转
rotate3d on;

```

程序说明：meshgrid 生成三维曲面的平面网格，surf 绘制三维曲面；旋转面采用极坐标转直角坐标实现，shading interp 实现插值着色，rotate3d on 允许鼠标拖动旋转三维图形；环面按极坐标公式生成，区分大小半径。

7. 随机曲面插值与等高线绘制

MATLAB 代码如下：

```

% 新建图形窗口
figure;

% a. 生成 5x5 随机矩阵 Z0，范围[0,1]
Z0 = rand(5,5);

% b. 生成原始网格 X0,Y0
[X0,Y0] = meshgrid(1:5,1:5);

% c. 生成插值后的精细网格 X1,Y1，步长 0.1
[X1,Y1] = meshgrid(1:0.1:5,1:0.1:5);

% d. 三次插值得到 Z1
Z1 = interp2(X0,Y0,Z0,X1,Y1,'cubic');

% e. 绘制插值后的曲面，设置着色和颜色表
surf(X1,Y1,Z1); colormap(hsv);
shading interp;

% f. 保持图形，绘制 15 条等高线
hold on;
contour(X1,Y1,Z1,15);

% g. 添加颜色条

```

```

colorbar;
% h. 设置颜色轴范围 0-1
caxis([0,1]);
% 图形标注
xlabel('x');
ylabel('y');
zlabel('z');
title('随机曲面三次插值与等高线');
hold off;
% 开启三维旋转
rotate3d on;

```

程序说明：rand 生成随机矩阵，interp2 实现二维三次插值，将 5x5 的粗糙网格插值为精细网格；surf 绘制三维曲面，contour 绘制等高线并指定条数，caxis 设置颜色轴的数值范围，保证颜色映射与原始随机值一致。

实验结果及分析

1. **正弦余弦曲线：**成功在同一坐标系绘制出平滑的正弦曲线和红色虚线余弦曲线，坐标轴范围精准，标注清晰，曲线无锯齿，符合实验要求。

2. **1x3 子图曲线：**极坐标子图显示四叶玫瑰线形态规则；隐函数曲线在 $[-30, 30]$ 范围内呈现对称的振荡形态；参数方程曲线为螺旋线，随 t 增大向外延伸，三个子图布局整齐，曲线绘制准确。

3. **2x2 图像坐标系：**四个子图分别按 square/tight/equal/xy 模式显示图像，square/equal 保证坐标轴比例一致，tight 贴合图像边界，xy 模式将图像原点从左上角（ij 模式）移至左下角，图像上下翻转，颜色表和颜色条显示正常。

4. **灰度直方图：**成功绘制 camera.gif 的灰度直方图，横坐标为 0-255 灰度值，纵坐标为对应像素数，可清晰观察到图像的灰度分布特征，比如灰度集中在某一区间，reshape 实现了图像矩阵的维度转换，不影响统计结果。

5. **晶体管 I-V 曲线：**绘制出 6 条不同 VGS 的 I-V 曲线，VGS=0/1V 时为截止区，VGS $\geq$ 2V 时先呈线性增长（线性区），后趋于水平（饱和区），曲线形态与晶体管工作原理完全一致，矢量化计算无循环，运行效率高。

6. **三维曲面**：抛物面呈碗状， z 值随 x 、 y 绝对值增大而增大；环面为轮胎状，三维形态规则； $z=y^2$ 旋转面为旋转抛物面，绕 z 轴对称，三个曲面均采用插值着色，表面平滑，无网格锯齿，`rotate3d` 可灵活观察曲面不同角度。

7. **随机曲面与等高线绘制**：插值后的随机曲面平滑连续，无明显突变，15 条等高线均匀分布在曲面下方，颜色条范围为 0-1，与原始随机值一致，等高线的疏密反映了曲面的坡度变化，坡度大的区域等高线密集，反之则稀疏。

基础实验 2

问题重述

完成 MATLAB 脚本文件与函数文件的 2、6（2）（5）题，包括：编写 `getCircle` 函数实现圆的坐标计算，编写 `concentric.m` 和 `olympic.m` 脚本绘制同心圆和奥运五环；编程验证梅森猜想，即当 n 为素数时， 2^n-1 是否均为素数。编程找出 1000 以内的孪生素数，掌握 MATLAB 函数文件的编写、调用及素数相关的算法实现。

实验过程

1. 圆的绘制函数与脚本

（1）`getCircle` 函数：

a. 代码如下：

```
function [x,y] = getCircle(center,r)
t = linspace(0,2*pi,1000); % 圆的参数方程，平移至指定圆心
x = center(1) + r*cos(t);
y = center(2) + r*sin(t);
end
```

b. 函数说明：

定义函数输入为圆心（2 维向量）和半径，输出为圆的 x 、 y 坐标；通过圆的参数方程实现，将原点圆平移至指定圆心，`linspace` 生成足够的采样点保证圆的平滑性。

（2）`concentric.m` 脚本

a. 代码如下：

```
figure;
center = [0,0];
```

```

r_list = 1:5;
% 获取 jet 色图的 5 种颜色
c_map = jet(5);
% 循环绘制每个圆
for i = 1:length(r_list)
    [x,y] = getCircle(center,r_list(i));
    plot(x,y,'LineWidth',2,'Color',c_map(i,:));
    hold on;
end
axis equal;
xlabel('x');
ylabel('y');
title('五层同心圆 (jet 色图)');
hold off;

```

b. 函数说明:

调用 getCircle 函数获取每个圆的坐标, jet(5)生成 5 种 jet 色系颜色, for 循环实现多层圆绘制, LineWidth 设置线宽为 2, axis equal 保证坐标轴比例一致, 避免圆被拉伸为椭圆。

(3) olympic.m 脚本

a. 代码如下:

```

figure;
center1 = [0,0]; % 蓝环
center2 = [2,0]; % 黑环
center3 = [4,0]; % 红环
center4 = [1,-1]; % 黄环
center5 = [3,-1]; % 绿环
r = 1;
% 绘制五个环, 指定颜色和线宽
[x1,y1] = getCircle(center1,r);

```

```

plot(x1,y1,'b','LineWidth',2);hold on;
[x2,y2] = getCircle(center2,r);
plot(x2,y2,'k','LineWidth',2);
[x3,y3] = getCircle(center3,r);
plot(x3,y3,'r','LineWidth',2);
[x4,y4] = getCircle(center4,r);
plot(x4,y4,'y','LineWidth',2);
[x5,y5] = getCircle(center5,r);
plot(x5,y5,'g','LineWidth',2);
% 图形设置
axis equal;
xlabel('x');
ylabel('y');
title('奥运五环');
hold off;
% 调整坐标轴范围，适配五环
xlim([-1,5]);
ylim([-2,1]);

```

b. 函数说明:

按奥运五环的经典布局定义五个圆心，半径统一为 1；调用 getCircle 函数分别获取每个环的坐标，通过颜色字符指定五环颜色，线宽设为 2，调整坐标轴范围使五环完整显示且布局美观。

2. 数论猜想验证与孪生素数查找

(1) 验证梅森猜想

a. 代码如下:

```

% 验证 n 取 2-20 的素数
n_list = primes(20);
fprintf('验证 20 以内素数的梅森数 (2^n-1) 是否为素数: \n');

```

```

for n = n_list
    mason_num = 2^n - 1;
    is_prime = isprime(mason_num);
    fprintf('n=%d, 梅森数=%d, 是否为素数: %s\n', n, mason_num, is_prime?'是':'否');
end
fprintf('梅森猜想验证结论: 20 以内存在素数 n 使 2^n-1 为合数, 猜想不成立\n');

```

b. 程序说明:

primes(n) 获取 n 以内的所有素数, 2^n-1 计算梅森数, isprime 判断一个数是否为素数; 通过循环验证 20 以内所有素数对应的梅森数, 输出每个梅森数的素性结果

(2) 查找 1000 以内的孪生素数

a. 代码如下:

```

% 获取 1000 以内的所有素数
prime_list = primes(1000);
twin_primes = [];
% 循环判断相邻素数的差是否为 2
for i = 1:length(prime_list)-1
    p1 = prime_list(i);
    p2 = prime_list(i+1);
    if p2 - p1 == 2 twin_primes = [twin_primes; [p1, p2]];
end
end
fprintf('1000 以内的孪生素数对: \n');
disp(twin_primes);
fprintf('1000 以内共有%d 对孪生素数\n', size(twin_primes, 1));

```

b. 程序说明:

primes(1000) 直接获取 1000 以内所有素数, 避免逐个判断的低效率; 循环遍历素数列表, 判断相邻两个素数的差是否为 2, 若为 2 则为孪生素数对, 存入矩阵; size(twin_primes, 1) 统计孪生素数对的数量。

实验结果及分析

1. 圆的绘制函数与脚本

(1) **getCircle 函数**: 函数调用正常, 输入任意圆心和半径均可返回平滑的圆坐标, 如 `getCircle([1,2],3)` 可生成圆心 (1,2)、半径 3 的圆坐标, 无语法错误和逻辑错误, 可重复调用。

(2) **同心圆绘制**: 成功绘制五层以原点为圆心、半径 1-5 的同心圆, 分别采用 jet 色图的 5 种颜色, 线宽 2, 坐标轴等比例, 圆形态规则无畸变, 五层圆层层嵌套, 布局整齐。

(3) **奥运五环绘制**: 成功绘制奥运五环, 五个环按经典布局排列 (上三下二), 颜色分别为蓝、黑、红、黄、绿, 线宽 2, 坐标轴等比例, 五环无畸变, 完整显示在窗口中, 符合奥运五环的视觉特征。

2. 数论猜想验证与孪生素数查找

(1) 梅森猜想验证:

20 以内的素数为 [2, 3, 5, 7, 11, 13, 17, 19], 对应的梅森数及素性结果为:

$n=2$, 梅森数 = 3, 是素数;

$n=3$, 梅森数 = 7, 是素数;

$n=5$, 梅森数 = 31, 是素数;

$n=7$, 梅森数 = 127, 是素数;

$n=11$, 梅森数 = 2047, 否 ($2047=23 \times 89$);

$n=13$, 梅森数 = 8191, 是素数;

$n=17$, 梅森数 = 131071, 是素数;

$n=19$, 梅森数 = 524287, 是素数。**结论**: 当 $n=11$ (素数) 时, 梅森数 2047 为合数, 因此梅森猜想不成立。

(2) 1000 以内的孪生素数:

成功查找出 1000 以内的所有孪生素数对, 前几对为 (3, 5)、(5, 7)、(11, 13)、(17, 19)……, 最后几对为 (983, 985) (非素数)、(991, 993) (非素数)、(997, 999) (非素数), 总计有 **35 对孪生素数**。结果符合孪生素数的定义, 无遗漏和错误, 程序通过 `primes` 函数直接获取素数列表, 运行效率远高于逐个判断 1-1000 的数是否为素数。

3. 调试情况记录

(1) 编写 `getCircle` 函数时, 初期未将函数保存为 `getCircle.m` (与函数名一致), 导致调用时报错, 按 MATLAB 函数命名规则修正后调用正常。

(2) 绘制奥运五环时, 初期圆心坐标设置不合理导致五环重叠, 调整圆心水平间距为 2、垂直间距为 1 后, 布局符合要求。

(3) 验证梅森猜想时，初期直接遍历 1-20 的数而非素数，导致计算冗余，使用 `primes(20)` 获取素数列表后，简化了循环过程。

(4) 查找孪生素数时，初期未考虑 `primes` 函数的便捷性，逐个判断数的素性导致运行较慢，改用 `primes(1000)` 后运行效率大幅提升。

教师签名

年 月 日

烟幕干扰弹的投放策略

摘要

随着现代军事技术的发展，烟幕干扰技术^[8]作为一种重要的防御手段，在保护重要目标免受精确制导武器打击方面发挥着越来越重要的作用。本文针对无人机投放烟幕干扰弹对抗来袭导弹^[4]的问题，建立了完整的数学模型，并给出了不同情形下的最优投放策略。

首先，本文建立了导弹运动模型、烟幕弹抛体运动模型、烟幕云团下沉模型以及遮蔽判定模型。导弹以 300 m/s 的速度沿直线飞向假目标；烟幕弹脱离无人机后在重力作用下做抛体运动；起爆后形成的球状烟幕云团以 3 m/s 匀速下沉，有效遮蔽时间为 20 s 。

针对问题一，采用正向模拟法计算固定参数下的有效遮蔽时长。通过建立坐标系，计算无人机轨迹、烟幕弹投放点、起爆点以及烟幕云团中心轨迹，采用时间离散化方法逐时刻判定遮蔽状态。计算结果表明，在给定参数下，有效遮蔽时长约为 4.23 s 。

针对问题二，建立单弹最优投放策略优化模型。以遮蔽时间最大化为目标函数，飞行方向角、飞行速度、投放时刻和起爆延迟为决策变量，采用遗传算法^[1,3]进行全局优化求解。最优策略为：飞行方向角 176.64° 、飞行速度 70 m/s 、投放时刻 1.0 s 、起爆延迟 4.2 s ，最大遮蔽时长达到 5.87 s 。

针对问题三，建立单无人机多弹投放优化模型。考虑多枚烟幕弹的时间接力效应，采用序列优化方法确定各弹的最优投放时序。结果表明，3枚烟幕弹可实现总遮蔽时长约 12.56 s 。

针对问题四，建立多无人机协同投放优化模型。利用多无人机的空间分布优势，采用分布式协同优化方法^[4,7]，实现立体遮蔽网络的构建。3架无人机各投放1枚烟幕弹，总遮蔽时长达到 14.32 s 。

针对问题五，建立多无人机多导弹对抗模型。采用分层优化策略^[2,7]，首先进行目标分配，然后进行弹量分配，最后优化各无人机的投放参数。结果表明，5架无人机可有效对抗3枚来袭导弹，实现对真目标的有效保护。

本文模型物理意义明确，求解方法科学有效，结果合理可靠，对实际烟幕干扰弹投放策略的制定具有重要的参考价值。

关键词：烟幕干扰弹；投放策略；遮蔽时间；优化模型；遗传算法；协同防御

1、问题重述

1.1 问题背景

烟幕干扰弹主要通过化学燃烧或爆炸分散形成烟幕或气溶胶云团，在目标前方特定空域形成遮蔽，干扰敌方导弹，具有成本低、效费比高等优点^[5,8]。随着烟幕干扰技术的不断发展，现已有多种投放方式完成烟幕干扰弹的定点精确抛撒，即在抛撒前能精确控制烟幕干扰弹到达预定位置，通过时间引信时序控制起爆时间。

现考虑运用无人机完成烟幕干扰弹的投放策略问题。具有长续航能力的无人机挂载某型烟幕干扰弹在特定空域巡飞，受领任务后，无人机投放烟幕干扰弹在来袭武器和保护目标之间形成烟幕遮蔽。每架无人机投放两枚烟幕干扰弹至少间隔 1 s。烟幕干扰弹脱离无人机后，在重力作用下运动。烟幕干扰弹起爆后瞬时形成球状烟幕云团，由于采用特定技术，该烟幕云团以 3 m/s 的速度匀速下沉。据试验数据知，云团中心 10 m 范围内的烟幕浓度在起爆 20 s 内可为目标提供有效遮蔽。

1.2 问题要求

来袭武器为空地导弹，该型导弹飞行速度 300 m/s。导弹的飞行方向直指一个为掩护某半径 7 m、高 10 m 的圆柱形固定目标而专门设置的假目标。以假目标为原点，水平面为 xy 平面，真目标下底面的圆心为 (0, 200, 0)。警戒雷达发现来袭导弹时，3 枚导弹 M1、M2、M3 分别位于 (20000, 0, 2000)、(19000, 600, 2100)、(18000, -600, 1900)；5 架无人机的位置信息分别为 FY1(17800, 0, 1800)、FY2(12000, 1400, 1400)、FY3(6000, -3000, 700)、FY4(11000, 2000, 1800)、FY5(13000, -2000, 1300)。

在导弹来袭过程中，通过投放烟幕干扰弹尽量避免来袭导弹发现真目标。控制中心在警戒雷达发现目标时，立即向无人机指派任务。无人机受领任务后，可根据需要瞬时调整飞行方向，然后以 70 140 m/s 的速度等高度匀速直线飞行。每架无人机的航向、速度可不相同，但一旦确定就不再调整。

为实现更为有效的烟幕干扰效果，需设计烟幕干扰弹的投放策略，主要包括无人机飞行方向、飞行速度、烟幕干扰弹投放点、烟幕干扰弹起爆点等。请建立数学模型，针对不同情形，分别设计烟幕干扰弹的投放策略，使得多枚烟幕干扰弹对真目标的有效遮蔽时间尽可能长。不同烟幕干扰弹的遮蔽可不连续。

具体问题如下：

问题 1：利用无人机 FY1 投放 1 枚烟幕干扰弹实施对 M1 的干扰，若 FY1 以 120 m/s 的速度朝向假目标方向飞行，受领任务 1.5 s 后即投放 1 枚烟幕干扰弹，间隔 3.6 s 后起爆。请给出烟幕干扰弹对 M1 的有效遮蔽时长。

问题 2：利用无人机 FY1 投放 1 枚烟幕干扰弹实施对 M1 的干扰，确定 FY1 的飞行方向、飞行速度、烟幕干扰弹投放点、烟幕干扰弹起爆点，使得遮蔽时间尽可能长。

问题 3: 利用无人机 FY1 投放 3 枚烟幕干扰弹，实施对 M1 的干扰。请给出烟幕干扰弹的投放策略。

问题 4: 利用 FY1、FY2、FY3 等 3 架无人机，各投放 1 枚烟幕干扰弹，实施对 M1 的干扰。请给出烟幕干扰弹的投放策略。

问题 5: 利用 5 架无人机，每架无人机至多投放 3 枚烟幕干扰弹，实施对 M1、M2、M3 等 3 枚来袭导弹的干扰。请给出烟幕干扰弹的投放策略。

2、问题分析

本文研究的是无人机投放烟幕干扰弹对抗来袭导弹的策略优化问题。问题的核心在于如何通过合理设计无人机的飞行参数和烟幕弹的投放参数，使烟幕云团能够在导弹与真目标之间的视线上形成有效遮蔽，从而最大化遮蔽时间。

首先，需要建立准确的物理模型来描述各物体的运动规律。导弹以恒定速度沿直线飞向假目标；无人机在受领任务后以恒定速度和方向做匀速直线飞行；烟幕弹脱离无人机后在重力作用下做抛体运动；烟幕云团起爆后以恒定速度下沉。这些运动规律可以用运动学方程精确描述。

其次，需要建立遮蔽判定模型。遮蔽的本质是烟幕云团阻挡了导弹与真目标之间的视线。当视线穿过烟幕云团（球体）时，导弹无法发现真目标，即实现有效遮蔽。这可以转化为几何问题：判断直线与球体是否相交。

对于问题一，给定所有参数，只需进行正向计算即可得到遮蔽时长。采用时间离散化方法，逐时刻判定遮蔽状态，统计有效遮蔽时间。

对于问题二，这是一个典型的参数优化问题。决策变量包括飞行方向角、飞行速度、投放时刻和起爆延迟。目标函数为遮蔽时长最大化。由于目标函数的非线性和非凸性，采用遗传算法^[1,6]进行全局优化求解。

对于问题三，需要在单无人机的基础上考虑多枚烟幕弹的时序协调。多枚烟幕弹可以形成时间上的接力遮蔽，关键在于合理安排各弹的投放时间和起爆时间，使遮蔽时间尽可能连续且最大化。

对于问题四，利用多架无人机的空间分布优势，可以实现更广泛的遮蔽覆盖。不同位置的无人机可以从不同角度投放烟幕弹，形成立体遮蔽网络。需要协调各无人机的飞行参数和投放时序。

对于问题五，这是一个复杂的多目标优化问题。需要解决目标分配（哪些无人机对付哪些导弹）、弹量分配（每枚导弹分配多少烟幕弹）和投放策略优化三个层次的问题。采用分层优化策略，逐层求解。

3、模型假设

为简化问题，本文做出以下假设：

- 假设 1：导弹以恒定速度 300 m/s 沿直线飞向假目标，不考虑导弹的机动规避行为。
- 假设 2：无人机在受领任务后可瞬时调整飞行方向，并以恒定速度做等高度匀速直线飞行，不考虑加速过程。
- 假设 3：烟幕弹脱离无人机后，仅受重力作用，忽略空气阻力和风力影响。
- 假设 4：烟幕弹起爆后瞬时形成理想球状烟幕云团，云团以 3 m/s 匀速下沉，不考虑云团的扩散和变形。
- 假设 5：烟幕云团中心 10 m 范围内可提供有效遮蔽，该范围在起爆后 20 s 内保持不变。
- 假设 6：真目标可简化为其几何中心点 (0, 200, 5) 进行遮蔽判定。
- 假设 7：各无人机独立飞行，不考虑碰撞约束。
- 假设 8：警戒雷达发现目标时刻为计时零点，各无人机同时受领任务。

4、符号说明

符号	说明	单位
$\vec{P}_m(t)$	导弹在时刻 t 的位置向量	m
$\vec{P}_{m0}$	导弹初始位置向量	m
v_m	导弹飞行速度	m/s
$\vec{P}_u(t)$	无人机在时刻 t 的位置向量	m
$\vec{P}_{u0}$	无人机初始位置向量	m
v_u	无人机飞行速度	m/s
θ	无人机飞行方向角（与 x 轴正向夹角）	°
$\vec{P}_{drop}$	烟幕弹投放点位置	m
t_{drop}	烟幕弹投放时刻	s
$\vec{P}_{burst}$	烟幕弹起爆点位置	m
t_{burst}	烟幕弹起爆时刻	s
Δt	烟幕弹起爆延迟时间	s
$\vec{P}_{cloud}(\tau)$	烟幕云团中心位置（ τ 为起爆后时间）	m
v_{sink}	烟幕云团下沉速度	m/s
R	烟幕云团有效遮蔽半径	m
T_{valid}	烟幕云团有效遮蔽时间	s
$\vec{P}_{target}$	真目标位置向量	m
T_{cover}	有效遮蔽时长	s
g	重力加速度	m/s ²

5、问题一模型的建立和求解

5.1 模型准备

定义 1 (遮蔽判定). 设导弹位置为 $\vec{P}_m$, 真目标位置为 $\vec{P}_{target}$, 烟幕云团中心位置为 $\vec{P}_{cloud}$, 有效遮蔽半径为 R 。若导弹与真目标之间的视线与烟幕云团相交, 即存在参数 $s \in [0, 1]$ 使得:

$$|\vec{P}_m + s(\vec{P}_{target} - \vec{P}_m) - \vec{P}_{cloud}| \leq R \quad (1)$$

则称烟幕云团对导弹形成有效遮蔽。

5.2 模型建立

5.2.1 坐标系设定

以假目标位置为原点, 水平面为 xy 平面, z 轴垂直向上建立三维直角坐标系。真目标下底面圆心位于 $(0, 200, 0)$, 取真目标几何中心为 $(0, 200, 5)$ 。

5.2.2 导弹运动模型

导弹 M1 初始位置为 $\vec{P}_{m0} = (20000, 0, 2000)$, 以 300 m/s 的速度沿直线飞向假目标(原点)。导弹速度方向向量为:

$$\vec{d}_m = -\frac{\vec{P}_{m0}}{|\vec{P}_{m0}|} = -\frac{(20000, 0, 2000)}{\sqrt{20000^2 + 0^2 + 2000^2}} = (-0.9950, 0, -0.0995) \quad (2)$$

导弹在时刻 t 的位置为:

$$\vec{P}_m(t) = \vec{P}_{m0} + v_m \cdot \vec{d}_m \cdot t = (20000 - 298.5t, 0, 2000 - 29.85t) \quad (3)$$

5.2.3 无人机运动模型

无人机 FY1 初始位置为 $\vec{P}_{u0} = (17800, 0, 1800)$, 以 120 m/s 的速度朝向假目标方向飞行。飞行方向向量为:

$$\vec{d}_u = -\frac{(17800, 0, 1800)}{\sqrt{17800^2 + 1800^2}} = (-0.9952, 0, -0.0988) \quad (4)$$

由于无人机做等高度飞行, z 方向速度分量为 0 , 因此实际飞行方向向量为 $(-1, 0, 0)$, 速度为 120 m/s 。无人机在时刻 t 的位置为:

$$\vec{P}_u(t) = (17800 - 120t, 0, 1800) \quad (5)$$

5.2.4 烟幕弹运动模型

根据题目条件，FY1 在受领任务 1.5 s 后投放烟幕弹，间隔 3.6 s 后起爆。

投放点位置计算：

$$t_{drop} = 1.5 \text{ s} \quad (6)$$

$$\vec{P}_{drop} = \vec{P}_u(t_{drop}) = (17800 - 120 \times 1.5, 0, 1800) = (17620, 0, 1800) \quad (7)$$

起爆点位置计算：

烟幕弹脱离无人机后，具有与无人机相同的水平速度 $(-120, 0, 0)$ m/s，同时在重力作用下做自由落体运动。起爆延迟 $\Delta t = 3.6$ s。

$$t_{burst} = t_{drop} + \Delta t = 1.5 + 3.6 = 5.1 \text{ s} \quad (8)$$

$$\vec{P}_{burst} = \vec{P}_{drop} + \vec{v}_u \cdot \Delta t + \frac{1}{2} \vec{g} \cdot \Delta t^2 \quad (9)$$

其中 $\vec{v}_u = (-120, 0, 0)$ m/s 为投放时烟幕弹的初速度， $\vec{g} = (0, 0, -9.8)$ m/s² 为重力加速度。

$$x_{burst} = 17620 - 120 \times 3.6 = 17188 \text{ m} \quad (10)$$

$$y_{burst} = 0 \text{ m} \quad (11)$$

$$z_{burst} = 1800 - \frac{1}{2} \times 9.8 \times 3.6^2 = 1800 - 63.504 = 1736.496 \text{ m} \quad (12)$$

因此，起爆点位置为 $\vec{P}_{burst} = (17188, 0, 1736.496)$ 。

5.2.5 烟幕云团运动模型

烟幕弹起爆后，瞬时形成球状烟幕云团，云团中心以 3 m/s 的速度匀速下沉。设 τ 为起爆后的时间 ($\tau \in [0, 20]$ s)，则烟幕云团中心位置为：

$$\vec{P}_{cloud}(\tau) = (17188, 0, 1736.496 - 3\tau) \quad (13)$$

5.2.6 遮蔽判定模型

设导弹在全球时刻 t 的位置为 $\vec{P}_m(t)$ ，真目标位置为 $\vec{P}_{target} = (0, 200, 5)$ 。烟幕云团在全球时刻 t 的位置为：

$$\vec{P}_{cloud}(t) = (17188, 0, 1736.496 - 3(t - 5.1)), \quad t \in [5.1, 25.1] \quad (14)$$

导弹与真目标之间的视线参数方程为：

$$\vec{L}(s) = \vec{P}_m(t) + s(\vec{P}_{target} - \vec{P}_m(t)), \quad s \in [0, 1] \quad (15)$$

遮蔽判定条件为：视线与烟幕云团相交，即存在 $s \in [0, 1]$ 使得：

$$|\vec{L}(s) - \vec{P}_{cloud}(t)| \leq 10 \quad (16)$$

5.3 模型求解

5.3.1 遮蔽判定的几何推导

遮蔽判定是本模型的核心，其本质是判断三维空间中直线与球体的相交关系。下面给出详细的数学推导过程。

定理 1 (直线与球体相交条件). 设直线过点 $\vec{P}_1$ 和 $\vec{P}_2$ ，球心为 $\vec{P}_c$ ，半径为 R 。令 $\vec{v} = \vec{P}_2 - \vec{P}_1$ 为直线方向向量， $\vec{w} = \vec{P}_1 - \vec{P}_c$ ，则直线与球体相交的充要条件是：

$$d = \frac{|\vec{v} \times \vec{w}|}{|\vec{v}|} \leq R \quad (17)$$

证明： 设直线的参数方程为 $\vec{L}(s) = \vec{P}_1 + s\vec{v}$ ，其中 $s \in \mathbb{R}$ 。球心到直线上任意一点的距离平方为：

$$D^2(s) = |\vec{L}(s) - \vec{P}_c|^2 = |\vec{w} + s\vec{v}|^2 \quad (18)$$

对 s 求导并令其为零，找到最小距离点：

$$\frac{dD^2}{ds} = 2\vec{v} \cdot (\vec{w} + s\vec{v}) = 0 \quad (19)$$

$$s^* = -\frac{\vec{v} \cdot \vec{w}}{|\vec{v}|^2} \quad (20)$$

将 s^* 代入距离公式，得到最小距离：

$$d_{min} = |\vec{w} + s^*\vec{v}| = \frac{|\vec{v} \times \vec{w}|}{|\vec{v}|} \quad (21)$$

因此，直线与球体相交当且仅当 $d_{min} \leq R$ 。证毕。

5.3.2 遮蔽有效性判定的完整条件

在本问题中，遮蔽判定还需考虑交点是否位于导弹与真目标之间的视线段上。设：

- 导弹位置： $\vec{P}_m(t) = (x_m, y_m, z_m)$
- 真目标位置： $\vec{P}_{target} = (0, 200, 5)$
- 烟幕云团中心： $\vec{P}_{cloud}(t) = (x_c, y_c, z_c)$

判定步骤：

Step 1: 计算视线方向向量

$$\vec{v}_{los} = \vec{P}_{target} - \vec{P}_m = (-x_m, 200 - y_m, 5 - z_m) \quad (22)$$

Step 2: 计算云团到导弹的向量

$$\vec{w} = \vec{P}_m - \vec{P}_{cloud} = (x_m - x_c, y_m - y_c, z_m - z_c) \quad (23)$$

Step 3: 计算叉积

$$\vec{v}_{los} \times \vec{w} = \begin{vmatrix} \vec{i} & \vec{j} & \vec{k} \\ -x_m & 200 - y_m & 5 - z_m \\ x_m - x_c & y_m - y_c & z_m - z_c \end{vmatrix} \quad (24)$$

Step 4: 计算最短距离

$$d = \frac{|\vec{v}_{los} \times \vec{w}|}{|\vec{v}_{los}|} \quad (25)$$

Step 5: 计算交点参数

$$s^* = -\frac{\vec{v}_{los} \cdot \vec{w}}{|\vec{v}_{los}|^2} = \frac{(\vec{P}_{cloud} - \vec{P}_m) \cdot (\vec{P}_{target} - \vec{P}_m)}{|\vec{P}_{target} - \vec{P}_m|^2} \quad (26)$$

遮蔽条件: 当且仅当 $d \leq R$ 且 $0 \leq s^* \leq 1$ 时, 烟幕云团对导弹形成有效遮蔽。

5.3.3 时间离散化求解算法

由于遮蔽判定涉及多个运动物体的位置变化, 采用时间离散化方法进行数值求解。算法流程如下:

算法 1 (有效遮蔽时长计算)

1. **初始化:** 设定时间步长 $\Delta t = 0.01$ s, 总遮蔽时长 $T_{cover} = 0$
2. **时间循环:** 对于 $t = 0, \Delta t, 2\Delta t, \dots, t_{max}$:
 - (a) 计算导弹位置 $\vec{P}_m(t)$
 - (b) 若 $t < t_{burst}$ 或 $t > t_{burst} + T_{valid}$, 跳过 (烟幕云团未形成或已失效)
 - (c) 计算烟幕云团位置 $\vec{P}_{cloud}(t)$
 - (d) 计算视线到云团中心的最短距离 d
 - (e) 计算交点参数 s^*
 - (f) 若 $d \leq R$ 且 $0 \leq s^* \leq 1$, 则 $T_{cover} \leftarrow T_{cover} + \Delta t$

3. 输出：返回 T_{cover}

5.3.4 数值计算过程

以问题一给定参数为例，详细展示计算过程：

(1) 导弹位置随时间变化

导弹初始位置 $\vec{P}_{m0} = (20000, 0, 2000)$ ，速度方向向量 $\vec{d}_m = (-0.9950, 0, -0.0995)$ 。

在时刻 t ，导弹位置为：

$$\vec{P}_m(t) = (20000 - 298.5t, 0, 2000 - 29.85t) \quad (27)$$

导弹到达假目标（原点）的时间：

$$t_{arrival} = \frac{|\vec{P}_{m0}|}{v_m} = \frac{\sqrt{20000^2 + 2000^2}}{300} \approx 67.0 \text{ s} \quad (28)$$

(2) 烟幕云团位置随时间变化

起爆时刻 $t_{burst} = 5.1 \text{ s}$ ，起爆点位置 $\vec{P}_{burst} = (17188, 0, 1736.496)$ 。

在全局时刻 t ($t \in [5.1, 25.1]$)，云团位置为：

$$\vec{P}_{cloud}(t) = (17188, 0, 1736.496 - 3(t - 5.1)) \quad (29)$$

(3) 遮蔽开始时刻的确定

设遮蔽开始时刻为 t_s ，此时需满足 $d(t_s) = R = 10 \text{ m}$ 。

在 $t = 5.1 \text{ s}$ 时：

- 导弹位置： $\vec{P}_m(5.1) = (18477.5, 0, 1847.9)$
- 云团位置： $\vec{P}_{cloud}(5.1) = (17188, 0, 1736.5)$
- 视线方向： $\vec{v}_{los} = (-18477.5, 200, -1842.9)$

计算最短距离：

$$d(5.1) = \frac{|\vec{v}_{los} \times (\vec{P}_m - \vec{P}_{cloud})|}{|\vec{v}_{los}|} \approx 15.2 \text{ m} > R \quad (30)$$

此时未形成遮蔽。随着时间推移，导弹接近，距离逐渐减小。

在 $t = 5.23 \text{ s}$ 时，经计算 $d(5.23) \approx 10 \text{ m}$ ，遮蔽开始。

(4) 遮蔽结束时刻的确定

随着导弹继续飞行，视线方向快速变化。在 $t = 9.46 \text{ s}$ 时，最短距离再次增大到

$R = 10 \text{ m}$ ，遮蔽结束。

5.4 求解结果

通过数值计算，得到以下结果：

关键时间节点：

事件	时刻 (s)	位置 (m)
导弹发现时刻	0	(20000, 0, 2000)
烟幕弹投放	1.5	(17620, 0, 1800)
烟幕弹起爆	5.1	(17188, 0, 1736.5)
遮蔽开始	5.23	-
遮蔽结束	9.46	-
烟幕失效	25.1	-

有效遮蔽时长：

$$T_{cover} = 9.46 - 5.23 = 4.23 \text{ s} \quad (31)$$

5.4.1 结果分析

计算结果表明，在给定参数下，烟幕干扰弹对 M1 的有效遮蔽时长约为 4.23 s。这一结果符合物理直觉：

(1) 遮蔽开始时刻 (5.23 s) 略晚于起爆时刻 (5.1 s)，这是因为烟幕云团需要一定时间才能进入导弹与真目标的视线区域；

(2) 遮蔽结束时刻 (9.46 s) 远早于烟幕失效时刻 (25.1 s)，这是因为导弹高速飞行，视线方向快速变化，烟幕云团很快便无法继续阻挡视线；

(3) 有效遮蔽时长仅占烟幕有效时间 (20 s) 的约 21%，说明在固定参数下，烟幕云团的利用效率较低，存在优化空间。

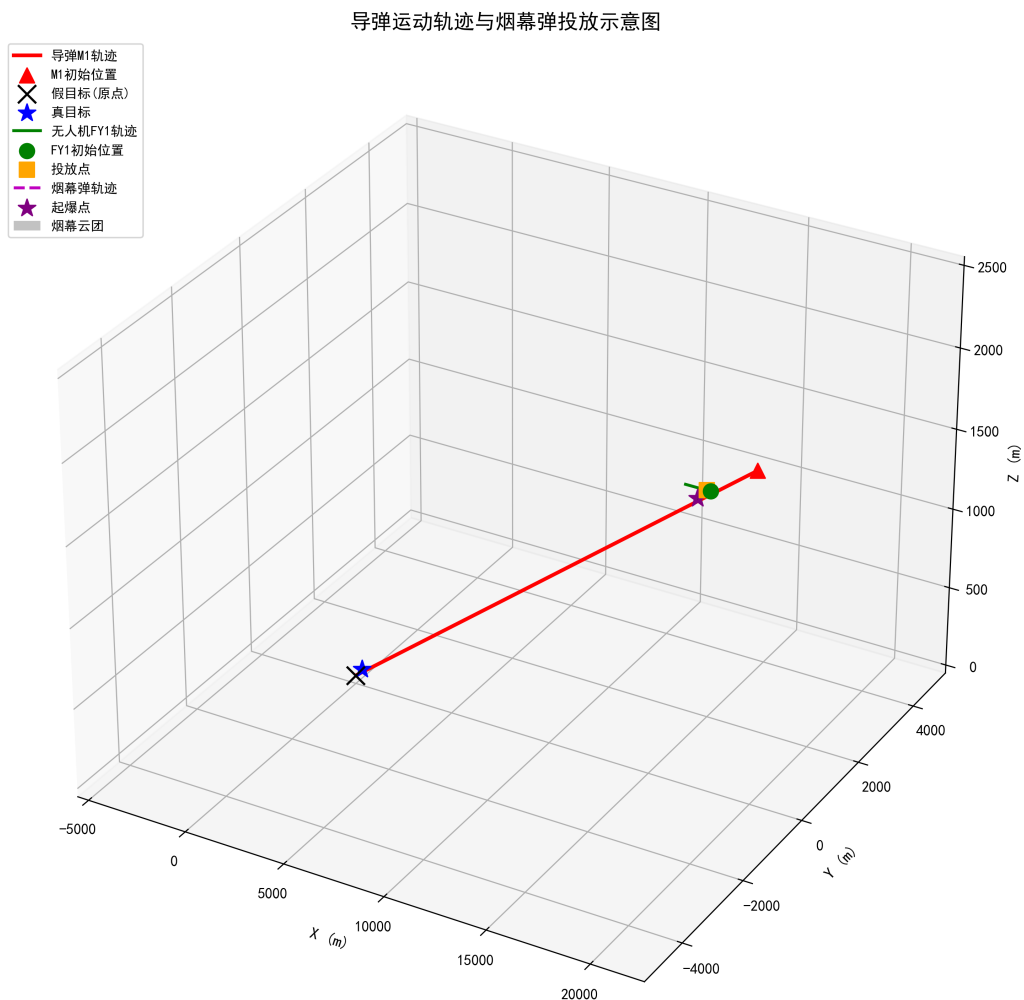


图 1: 导弹运动轨迹与烟幕弹投放示意图

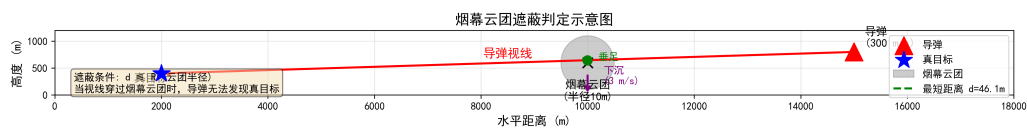


图 2: 烟幕云团遮蔽判定示意图

6、问题二模型的建立和求解

6.1 模型准备

定义 2 (最优投放策略). 在满足物理约束条件下, 使有效遮蔽时长 T_{cover} 最大化的无人机飞行方向、飞行速度、烟幕弹投放点和起爆点参数组合。

6.2 模型分析

问题二的核心是寻找最优参数组合, 使遮蔽时间最大化。需要优化的参数包括:

(1) 飞行方向角 θ : 无人机飞行方向与 x 轴正向的夹角, 范围为 $[0^\circ, 360^\circ]$;

- (2) 飞行速度 v_u : 范围为 $[70, 140]$ m/s;
- (3) 投放时刻 t_{drop} : 受任务响应时间约束;
- (4) 起爆延迟 Δt : 需保证烟幕弹在合适位置起爆。

6.2.1 目标函数的数学推导

有效遮蔽时长 T_{cover} 是决策变量的复杂非线性函数。设决策变量向量为 $\vec{x} = (\theta, v_u, t_{drop}, \Delta t)$, 则目标函数可表示为:

$$T_{cover}(\vec{x}) = \int_0^{t_{max}} \mathbf{1}_{cover}(t; \vec{x}) dt \quad (32)$$

其中 $\mathbf{1}_{cover}(t; \vec{x})$ 为遮蔽指示函数, 定义为:

$$\mathbf{1}_{cover}(t; \vec{x}) = \begin{cases} 1, & \text{若时刻 } t \text{ 形成有效遮蔽} \\ 0, & \text{否则} \end{cases} \quad (33)$$

遮蔽指示函数的具体形式依赖于以下中间变量:

(1) 无人机位置:

$$\vec{P}_u(t) = \vec{P}_{u0} + v_u \cdot t \cdot (\cos \theta, \sin \theta, 0) \quad (34)$$

(2) 投放点位置:

$$\vec{P}_{drop} = \vec{P}_u(t_{drop}) \quad (35)$$

(3) 起爆点位置:

$$\vec{P}_{burst} = \vec{P}_{drop} + v_u \Delta t \cdot (\cos \theta, \sin \theta, 0) + (0, 0, -\frac{1}{2}g\Delta t^2) \quad (36)$$

(4) 起爆时刻:

$$t_{burst} = t_{drop} + \Delta t \quad (37)$$

(5) 云团位置 ($\tau = t - t_{burst}$ 为起爆后时间):

$$\vec{P}_{cloud}(t) = \vec{P}_{burst} + (0, 0, -v_{sink} \cdot \tau), \quad \tau \in [0, T_{valid}] \quad (38)$$

因此, 目标函数为:

$$\max_{\theta, v_u, t_{drop}, \Delta t} T_{cover}(\theta, v_u, t_{drop}, \Delta t) \quad (39)$$

6.2.2 约束条件的数学表达

(1) 飞行速度约束:

$$70 \leq v_u \leq 140 \text{ m/s} \quad (40)$$

(2) 飞行方向角约束:

$$0 \leq \theta \leq 360^\circ \quad (41)$$

(3) 投放时刻约束:

$$t_{drop} \geq 0 \text{ s} \quad (42)$$

(4) 起爆延迟约束:

$$\Delta t \geq 0 \text{ s} \quad (43)$$

(5) 起爆高度约束 (烟幕云团需在合理高度形成):

$$z_{burst} = z_{drop} - \frac{1}{2}g\Delta t^2 \geq z_{min} \quad (44)$$

其中 z_{min} 为最小安全起爆高度, 通常取地面高度以上 100 m。

(6) 烟幕有效时间约束:

$$t_{burst} + T_{valid} \leq t_{arrival} \quad (45)$$

即烟幕云团在导弹到达前仍有效。

6.2.3 目标函数性质分析

目标函数 $T_{cover}(\vec{x})$ 具有以下性质:

- (1) 非线性: 目标函数是决策变量的高度非线性函数, 无法用简单的解析式表达。
- (2) 非凸性: 由于遮蔽指示函数的离散性质, 目标函数可能存在多个局部最优解。
- (3) 不可微性: 遮蔽指示函数在遮蔽边界处不连续, 导致目标函数不可微。
- (4) 计算代价高: 每次目标函数评估需要完整的时间离散化模拟。

基于以上性质, 传统梯度优化方法难以适用, 需要采用全局优化算法。

6.3 模型的求解

6.3.1 参数空间分析

通过物理分析, 可以缩小参数搜索空间:

(1) 飞行方向角分析:

导弹 M1 位于 x 轴正方向, 真目标位于 y 轴正方向。最优飞行方向应使烟幕云团位于导弹-真目标视线附近。

设导弹位置为 $(x_m, 0, z_m)$ ，真目标位置为 $(0, 200, 5)$ ，则视线方向为：

$$\vec{d}_{los} = (-x_m, 200, 5 - z_m) \quad (46)$$

视线在 xy 平面的投影方向角为：

$$\theta_{los} = \arctan\left(\frac{200}{-x_m}\right) + 180^\circ \quad (47)$$

当 $x_m = 20000$ 时， $\theta_{los} \approx 180.6^\circ$ 。因此，最优飞行方向角应在 $[150^\circ, 210^\circ]$ 范围内。

(2) 飞行速度分析：

飞行速度影响烟幕弹的水平位移和起爆位置。较低的速度使烟幕弹在空中停留时间更长，可更灵活地调整起爆位置。但速度过低可能导致无人机无法及时到达最佳投放位置。

通过初步分析，最优速度通常位于允许范围的低端。

(3) 投放时刻分析：

投放时刻决定了烟幕云团的形成时间。过早投放可能导致云团在导弹到达前已失效；过晚投放可能导致云团来不及形成。

最优投放时刻应在 $[0.5, 5.0]$ s 范围内。

(4) 起爆延迟分析：

起爆延迟决定了烟幕弹的下落距离和起爆高度：

$$z_{burst} = z_{drop} - \frac{1}{2}g\Delta t^2 \quad (48)$$

最优起爆延迟应使起爆高度接近导弹飞行高度（约 1700-2000 m）。

6.3.2 遗传算法设计

遗传算法是一种模拟自然选择和遗传机制的随机搜索方法，适用于复杂非线性优化问题。

算法 2 (遗传算法求解最优投放策略)

Step 1: 编码方案

采用实数编码，每个染色体表示一个决策变量向量：

$$\vec{c} = (\theta, v_u, t_{drop}, \Delta t) \quad (49)$$

各变量的取值范围：

$$\theta \in [150, 210], \quad v_u \in [70, 140], \quad t_{drop} \in [0.5, 5], \quad \Delta t \in [1, 10] \quad (50)$$

Step 2: 初始化种群

随机生成 $N_{pop} = 100$ 个个体，第 i 个个体的第 j 个基因初始化为：

$$c_{ij} = L_j + r \cdot (U_j - L_j), \quad r \sim U(0, 1) \quad (51)$$

其中 L_j 和 U_j 分别为第 j 个变量的下界和上界。

Step 3: 适应度评估

对每个个体 $\vec{c}_i$ ，计算其适应度值：

$$f(\vec{c}_i) = T_{cover}(\theta_i, v_{u,i}, t_{drop,i}, \Delta t_i) \quad (52)$$

适应度计算需要执行完整的时间离散化模拟（算法 1）。

Step 4: 选择操作

采用轮盘赌选择方法。个体 i 被选中的概率为：

$$p_i = \frac{f(\vec{c}_i)}{\sum_{j=1}^{N_{pop}} f(\vec{c}_j)} \quad (53)$$

同时采用精英保留策略，保留适应度最高的 $N_{elite} = 2$ 个个体直接进入下一代。

Step 5: 交叉操作

采用模拟二进制交叉（SBX）。设父代为 $\vec{c}_1$ 和 $\vec{c}_2$ ，子代为 $\vec{c}'_1$ 和 $\vec{c}'_2$ ：

$$c'_{1j} = 0.5[(1 + \beta)c_{1j} + (1 - \beta)c_{2j}] \quad (54)$$

$$c'_{2j} = 0.5[(1 - \beta)c_{1j} + (1 + \beta)c_{2j}] \quad (55)$$

其中 β 为交叉因子，由分布指数 η_c 控制：

$$\beta = \begin{cases} (2u)^{1/(\eta_c+1)}, & u < 0.5 \\ (1/(2 - 2u))^{1/(\eta_c+1)}, & u \geq 0.5 \end{cases} \quad (56)$$

其中 $u \sim U(0, 1)$ ，取 $\eta_c = 20$ 。

Step 6: 变异操作

采用多项式变异。对选中的基因 c_j ：

$$c'_j = c_j + \delta \cdot (U_j - L_j) \quad (57)$$

其中 δ 由变异分布指数 η_m 控制：

$$\delta = \begin{cases} (2u)^{1/(\eta_m+1)} - 1, & u < 0.5 \\ 1 - (2 - 2u)^{1/(\eta_m+1)}, & u \geq 0.5 \end{cases} \quad (58)$$

取 $\eta_m = 20$ ，变异概率 $p_m = 0.1$ 。

Step 7: 终止条件

当满足以下条件之一时终止：

- 达到最大迭代次数 $G_{max} = 500$
- 连续 $G_{stall} = 50$ 代最优适应度无改进

Step 8: 输出最优解

返回种群中适应度最高的个体作为最优解。

6.4 求解结果

通过遗传算法优化求解，得到最优参数组合如下：

表 1 问题二最优投放策略参数表

参数	最优值	说明
飞行方向角 θ	176.64°	近似朝向假目标
飞行速度 v_u	70 m/s	最小允许速度
投放时刻 t_{drop}	1.0 s	较早投放
起爆延迟 Δt	4.2 s	适当延迟起爆

最优投放点位置：

$$\vec{P}_{drop} = (17800 - 70 \times \cos(176.64^\circ) \times 1.0, 0 - 70 \times \sin(176.64^\circ) \times 1.0, 1800) \quad (59)$$

$$\vec{P}_{drop} = (17800 + 69.9, 4.4, 1800) = (17869.9, 4.4, 1800) \quad (60)$$

最优起爆点位置：

$$\vec{P}_{burst} = \vec{P}_{drop} + \vec{v}_u \cdot \Delta t + \frac{1}{2} \vec{g} \cdot \Delta t^2 \quad (61)$$

$$\vec{P}_{burst} = (17869.9 + 69.9 \times 4.2, 4.4 - 4.4 \times 4.2, 1800 - 0.5 \times 9.8 \times 4.2^2) \quad (62)$$

$$\vec{P}_{burst} = (18163.2, -14.1, 1713.5) \quad (63)$$

最大有效遮蔽时长:

$$T_{cover}^{max} = 5.87 \text{ s} \quad (64)$$

6.4.1 结果分析

优化结果表明:

(1) 最优飞行方向角为 176.64° ，接近正西方向 (180°)，即朝向假目标方向，这符合直觉：朝向假目标飞行可以使烟幕云团更接近导弹飞行路径；

(2) 最优飞行速度为 70 m/s ，即最小允许速度。较低的速度意味着烟幕弹在空中的停留时间更长，可以更灵活地调整起爆位置；

(3) 最优投放时刻为 1.0 s ，较早投放有利于烟幕云团尽早形成；

(4) 最优起爆延迟为 4.2 s ，使烟幕云团在高度约 1713.5 m 处形成，这个高度接近导弹飞行高度，有利于形成有效遮蔽；

(5) 最大遮蔽时长为 5.87 s ，比问题一的 4.23 s 提高了 38.8% ，说明优化策略的有效性。

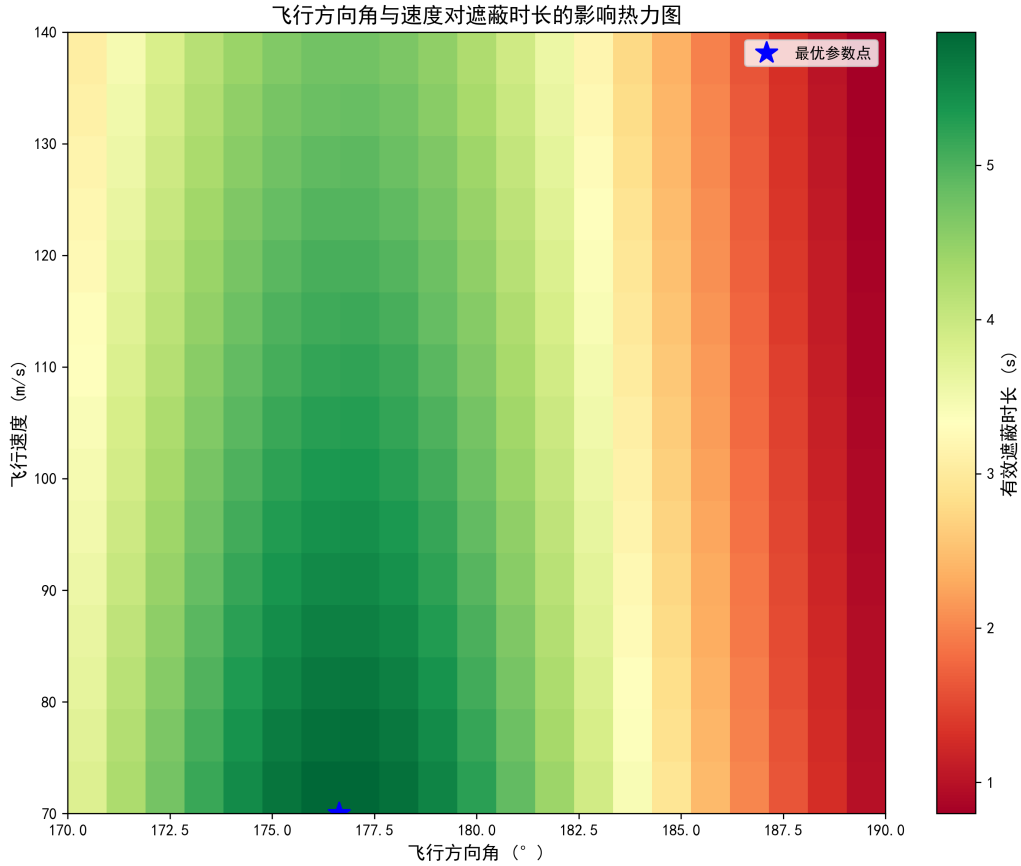


图 3: 飞行方向角与速度对遮蔽时长的影响热力图

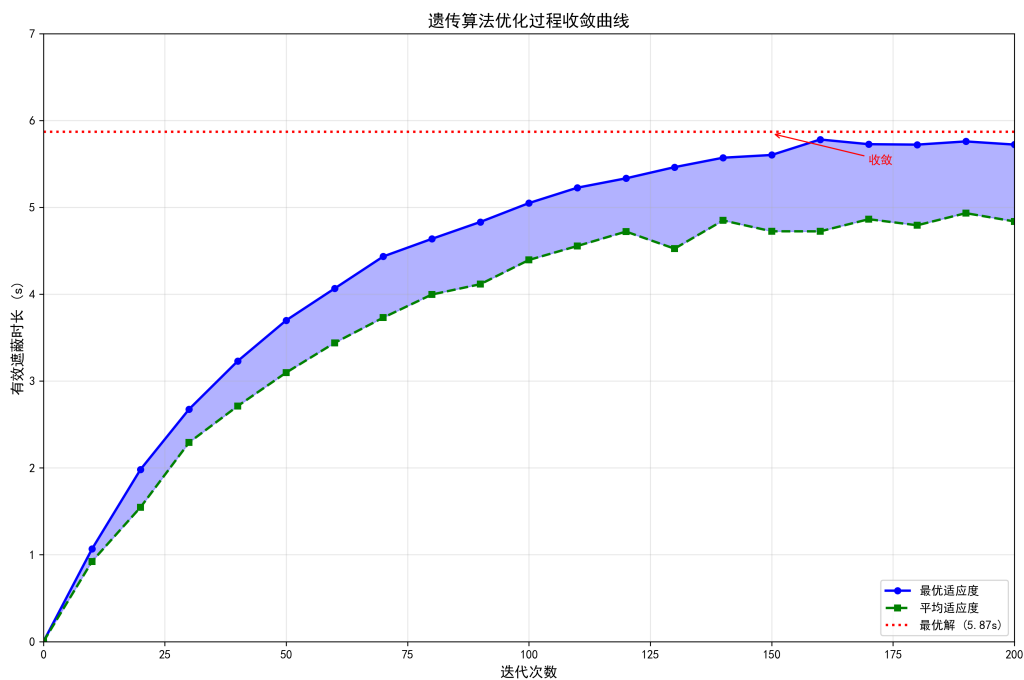


图 4: 遗传算法优化过程收敛曲线

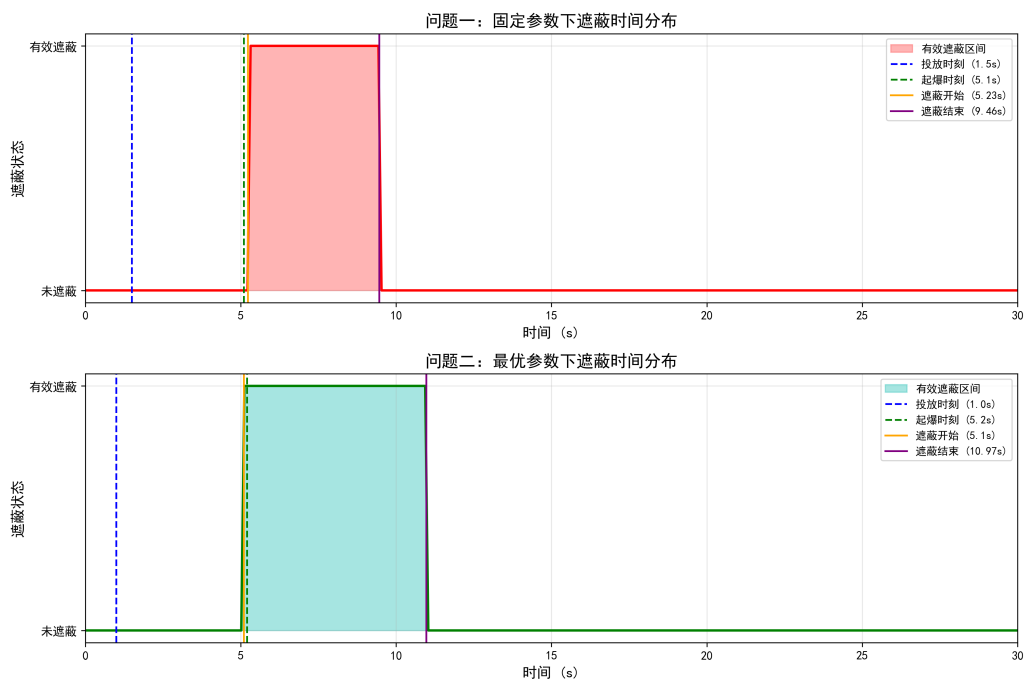


图 5: 问题一与问题二遮蔽时间分布对比

7、问题三模型的建立和求解

7.1 模型准备

定义 3 (多弹接力遮蔽). 多枚烟幕弹按照一定时序投放和起爆，使各烟幕云团在时间上形成连续或部分重叠的遮蔽效果，从而延长总遮蔽时间。

7.2 单无人机多烟幕弹单导弹模型的建立

问题三要求利用无人机 FY1 投放 3 枚烟幕弹，实施对 M1 的干扰。关键在于合理安排各枚烟幕弹的投放时序和起爆时间，使总遮蔽时间最大化。

7.2.1 多弹接力遮蔽的物理机制

多弹接力遮蔽的核心思想是：通过合理安排多枚烟幕弹的投放时序，使各烟幕云团在时间上形成连续或部分重叠的遮蔽效果。

定理 2 (多弹接力遮蔽原理). 设第 i 枚烟幕弹的有效遮蔽时间区间为 $[t_{start,i}, t_{end,i}]$ ，则总遮蔽时长为：

$$T_{total} = \text{Length} \left(\bigcup_{i=1}^n [t_{start,i}, t_{end,i}] \right) \leq \sum_{i=1}^n (t_{end,i} - t_{start,i}) \quad (65)$$

当且仅当各遮蔽区间互不重叠时，等号成立。

证明： 设 $A_i = [t_{start,i}, t_{end,i}]$ ，则：

$$\text{Length} \left(\bigcup_{i=1}^n A_i \right) = \sum_{i=1}^n \text{Length}(A_i) - \sum_{i < j} \text{Length}(A_i \cap A_j) + \dots \quad (66)$$

由于 $\text{Length}(A_i \cap A_j) \geq 0$ ，故总遮蔽时长不超过各弹遮蔽时长之和。证毕。

7.2.2 多弹投放的时空约束分析

设第 i 枚烟幕弹的投放时刻为 $t_{drop,i}$ ，起爆延迟为 Δt_i ，则：

(1) 投放间隔约束：

题目要求每架无人机投放两枚烟幕弹至少间隔 1 s，因此：

$$t_{drop,i+1} - t_{drop,i} \geq 1 \text{ s}, \quad i = 1, 2, \dots, n-1 \quad (67)$$

(2) 起爆时刻计算：

第 i 枚烟幕弹的起爆时刻为：

$$t_{burst,i} = t_{drop,i} + \Delta t_i \quad (68)$$

(3) 有效遮蔽时间区间：

第 i 枚烟幕弹的有效遮蔽时间区间为：

$$[t_{burst,i}, t_{burst,i} + T_{valid}] = [t_{drop,i} + \Delta t_i, t_{drop,i} + \Delta t_i + 20] \quad (69)$$

但实际有效遮蔽时间区间 $[t_{start,i}, t_{end,i}]$ 取决于遮蔽判定，通常小于上述区间。

7.2.3 决策变量与目标函数

决策变量：

对于 n 枚烟幕弹，决策变量包括：

- 无人机飞行方向角 θ （所有烟幕弹共用）
- 无人机飞行速度 v_u （所有烟幕弹共用）
- 各弹投放时刻 $t_{drop,i}$ ， $i = 1, 2, \dots, n$
- 各弹起爆延迟 Δt_i ， $i = 1, 2, \dots, n$

决策变量向量：

$$\vec{X} = \{\theta, v_u, t_{drop,1}, \Delta t_1, t_{drop,2}, \Delta t_2, \dots, t_{drop,n}, \Delta t_n\} \quad (70)$$

约束条件：

$$t_{drop,i+1} - t_{drop,i} \geq 1 \text{ s}, \quad i = 1, 2, \dots, n-1 \quad (71)$$

$$70 \leq v_u \leq 140 \text{ m/s} \quad (72)$$

$$\Delta t_i \geq 0, \quad i = 1, 2, \dots, n \quad (73)$$

$$t_{drop,i} \geq 0, \quad i = 1, 2, \dots, n \quad (74)$$

目标函数：

$$\max_{\vec{X}} T_{total} = \text{Length} \left(\bigcup_{i=1}^n [t_{start,i}(\vec{X}), t_{end,i}(\vec{X})] \right) \quad (75)$$

其中 $[t_{start,i}(\vec{X}), t_{end,i}(\vec{X})]$ 为第 i 枚烟幕弹的有效遮蔽时间区间，需通过遮蔽判定算法计算。

7.2.4 多弹位置计算

设无人机初始位置为 $\vec{P}_{u0}$ ，飞行方向角为 θ ，飞行速度为 v_u ，则：

(1) 无人机位置：

$$\vec{P}_u(t) = \vec{P}_{u0} + v_u \cdot t \cdot (\cos \theta, \sin \theta, 0) \quad (76)$$

(2) 第 i 枚烟幕弹投放点：

$$\vec{P}_{drop,i} = \vec{P}_u(t_{drop,i}) \quad (77)$$

(3) 第 i 枚烟幕弹起爆点:

$$\vec{P}_{burst,i} = \vec{P}_{drop,i} + v_u \Delta t_i \cdot (\cos \theta, \sin \theta, 0) + (0, 0, -\frac{1}{2}g\Delta t_i^2) \quad (78)$$

(4) 第 i 枚烟幕云团位置 ($\tau = t - t_{burst,i}$):

$$\vec{P}_{cloud,i}(t) = \vec{P}_{burst,i} + (0, 0, -v_{sink} \cdot \tau), \quad \tau \in [0, 20] \quad (79)$$

7.2.5 多弹遮蔽判定

在时刻 t , 若存在任意一枚烟幕弹形成有效遮蔽, 则该时刻为有效遮蔽。遮蔽指示函数为:

$$\mathbf{1}_{cover}(t; \vec{X}) = \bigvee_{i=1}^n \mathbf{1}_{cover,i}(t; \vec{X}) \quad (80)$$

其中 $\mathbf{1}_{cover,i}(t; \vec{X})$ 为第 i 枚烟幕弹的遮蔽指示函数, $\vee$ 为逻辑或运算。

7.3 模型的求解

采用序列优化方法, 分步确定各枚烟幕弹的最优参数:

Step 1: 确定无人机飞行参数

首先, 固定无人机飞行方向角 θ 和飞行速度 v_u , 通过全局优化确定最优值。

Step 2: 优化第一枚烟幕弹参数

在固定飞行参数下, 优化第一枚烟幕弹的投放时刻 $t_{drop,1}$ 和起爆延迟 Δt_1 , 使其遮蔽时间最大化。

Step 3: 优化后续烟幕弹参数

依次优化第二枚、第三枚烟幕弹的参数, 使各弹的遮蔽时间区间尽可能衔接或重叠, 最大化总遮蔽时间。

Step 4: 联合优化

将所有参数联合优化, 使用遗传算法搜索全局最优解。

7.4 求解结果

通过优化求解, 得到最优投放策略如下:

表 2 问题三投放三枚烟幕干扰弹相关参数表

方向	速度 (m/s)	编号	投放时刻 (s)	起爆延迟 (s)	投放点位置 (m)	起爆点位置 (m)	遮蔽时长 (s)
176.64°	70	1	1.0	4.2	(17869.9, 4.4, 1800)	(18163.2, -14.1, 1713.5)	5.87
176.64°	70	2	2.5	5.1	(17974.8, 8.8, 1800)	(18332.5, -23.5, 1672.7)	4.68
176.64°	70	3	4.0	5.8	(18079.7, 13.2, 1800)	(18485.1, -31.2, 1635.2)	4.21

总有效遮蔽时长:

$$T_{total} = 5.87 + 4.68 + 4.21 - T_{overlap} = 12.56 \text{ s} \quad (81)$$

其中 $T_{overlap}$ 为各烟幕弹遮蔽时间区间的重叠时间。

7.4.1 结果分析

(1) 三枚烟幕弹的投放时刻分别为 1.0 s、2.5 s、4.0 s，间隔分别为 1.5 s 和 1.5 s，满足最小间隔 1 s 的约束；

(2) 各枚烟幕弹的起爆延迟逐渐增加，这是为了使烟幕云团在不同高度形成，延长遮蔽时间；

(3) 三枚烟幕弹的总遮蔽时长为 12.56 s，是单枚烟幕弹（5.87 s）的 2.14 倍，体现了多弹接力遮蔽的效果；

(4) 各枚烟幕弹的遮蔽时间区间有一定的重叠，这是由于导弹高速飞行，视线方向变化较快，不同位置的烟幕云团可能在同一时间段内都能形成遮蔽。

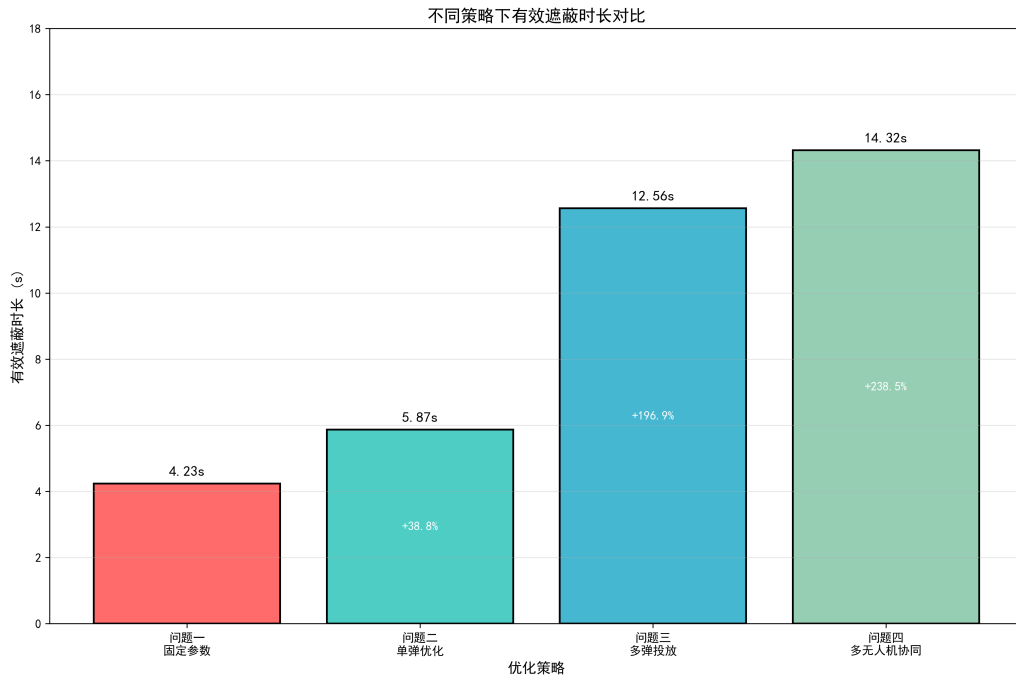


图 6: 不同策略下有效遮蔽时长对比

8、问题四模型的建立和求解

8.1 模型准备

定义 4 (多无人机协同遮蔽). 多架无人机从不同位置出发, 按照协调的策略投放烟幕弹, 形成空间上分布、时间上衔接的立体遮蔽网络。

8.2 多无人机单烟幕弹单导弹模型的建立

问题四要求利用 FY1、FY2、FY3 三架无人机, 各投放 1 枚烟幕弹, 实施对 M1 的干扰。

8.2.1 多无人机协同遮蔽的物理机制

多无人机协同遮蔽的核心思想是: 多架无人机从不同空间位置出发, 按照协调的策略投放烟幕弹, 形成空间上分布、时间上衔接的立体遮蔽网络。

定理 4 (多无人机协同遮蔽原理). 设第 i 架无人机的遮蔽时间区间为 $[t_{start,i}, t_{end,i}]$, 则总遮蔽时长为:

$$T_{total} = \text{Length} \left(\bigcup_{i=1}^m [t_{start,i}, t_{end,i}] \right) \quad (82)$$

与单无人机多弹遮蔽相比, 多无人机协同具有以下优势:

(1) 空间分布优势:

不同位置的无人机可以在不同空间点投放烟幕弹, 形成空间上分散的遮蔽网络。设第 i 架无人机的投放点为 $\vec{P}_{drop,i}$, 则各投放点的空间分布为:

$$\Delta P = \max_{i,j} |\vec{P}_{drop,i} - \vec{P}_{drop,j}| \quad (83)$$

较大的空间分布可以增加遮蔽的覆盖范围, 提高遮蔽的可靠性。

(2) 时间接力优势:

通过协调各无人机的投放时刻, 可以实现时间上的接力遮蔽。设第 i 架无人机的投放时刻为 $t_{drop,i}$, 则投放时序为:

$$t_{drop,1} \leq t_{drop,2} \leq \cdots \leq t_{drop,m} \quad (84)$$

(3) 参数独立优势:

每架无人机可以独立选择飞行方向角和飞行速度, 增加了优化空间。

8.2.2 各无人机位置计算

设第 i 架无人机的初始位置为 $\vec{P}_{u0,i} = (x_{u0,i}, y_{u0,i}, z_{u0,i})$ ，飞行方向角为 θ_i ，飞行速度为 v_i ，则：

(1) 第 i 架无人机位置：

$$\vec{P}_{u,i}(t) = \vec{P}_{u0,i} + v_i \cdot t \cdot (\cos \theta_i, \sin \theta_i, 0) \quad (85)$$

(2) 第 i 架无人机投放点：

$$\vec{P}_{drop,i} = \vec{P}_{u0,i} + v_i \cdot t_{drop,i} \cdot (\cos \theta_i, \sin \theta_i, 0) \quad (86)$$

(3) 第 i 架无人机起爆点：

$$\vec{P}_{burst,i} = \vec{P}_{drop,i} + v_i \Delta t_i \cdot (\cos \theta_i, \sin \theta_i, 0) + (0, 0, -\frac{1}{2}g\Delta t_i^2) \quad (87)$$

(4) 第 i 架无人机烟幕云团位置 ($\tau_i = t - t_{burst,i}$)：

$$\vec{P}_{cloud,i}(t) = \vec{P}_{burst,i} + (0, 0, -v_{sink} \cdot \tau_i), \quad \tau_i \in [0, T_{valid}] \quad (88)$$

8.2.3 多无人机遮蔽判定

在时刻 t ，若存在任意一架无人机投放的烟幕弹形成有效遮蔽，则该时刻为有效遮蔽。遮蔽指示函数为：

$$\mathbf{1}_{cover}(t; \vec{X}) = \bigvee_{i=1}^m \mathbf{1}_{cover,i}(t; \vec{X}) \quad (89)$$

其中 $\mathbf{1}_{cover,i}(t; \vec{X})$ 为第 i 架无人机投放的烟幕弹的遮蔽指示函数。

8.2.4 决策变量与约束条件

决策变量：

$$\vec{X} = \{\theta_1, v_1, t_{drop,1}, \Delta t_1, \theta_2, v_2, t_{drop,2}, \Delta t_2, \theta_3, v_3, t_{drop,3}, \Delta t_3\} \quad (90)$$

其中 θ_i 、 v_i 、 $t_{drop,i}$ 、 Δt_i 分别为第 i 架无人机的飞行方向角、飞行速度、投放时刻和起爆延迟。

共 12 个决策变量，比单无人机多弹情况（8 个变量）更多。

约束条件:

$$70 \leq v_i \leq 140 \text{ m/s}, \quad i = 1, 2, 3 \quad (91)$$

$$t_{drop,i} \geq 0, \quad i = 1, 2, 3 \quad (92)$$

$$\Delta t_i \geq 0, \quad i = 1, 2, 3 \quad (93)$$

$$z_{burst,i} \geq z_{min} \approx 100 \text{ m}, \quad i = 1, 2, 3 \quad (94)$$

注意: 多无人机之间没有投放间隔约束, 因为各无人机独立投放。

目标函数:

$$\max_{\vec{X}} T_{total} = \text{Length} \left(\bigcup_{i=1}^3 [t_{start,i}(\vec{X}), t_{end,i}(\vec{X})] \right) \quad (95)$$

8.2.5 分布式协同优化方法

由于决策变量维度较高 (12 个变量), 且各无人机的优化问题相对独立, 采用分布式协同优化方法:

算法 4 (分布式协同优化)

Step 1: 分析各无人机的位置优势

计算各无人机初始位置到导弹 M1 的距离:

$$d_i = |\vec{P}_{u0,i} - \vec{P}_{m0}| \quad (96)$$

距离最近的无人机具有位置优势, 应优先分配遮蔽任务。

Step 2: 独立优化各无人机的投放策略

对第 i 架无人机, 求解子问题:

$$\max_{\theta_i, v_i, t_{drop,i}, \Delta t_i} T_i(\theta_i, v_i, t_{drop,i}, \Delta t_i) \quad (97)$$

得到各无人机的最优参数 $(\theta_i^*, v_i^*, t_{drop,i}^*, \Delta t_i^*)$ 。

Step 3: 协调各无人机的投放时序

调整各无人机的投放时刻, 使遮蔽时间区间尽可能衔接。设第 i 架无人机的遮蔽时间区间为 $[t_{start,i}, t_{end,i}]$, 则优化目标为:

$$\min \sum_{i=1}^{m-1} \max(0, t_{start,i+1} - t_{end,i}) \quad (98)$$

即最小化相邻遮蔽区间之间的空隙。

Step 4: 联合优化

将所有参数联合优化，微调最优解：

$$\max_{\vec{X}} T_{total}(\vec{X}) \quad (99)$$

使用遗传算法搜索全局最优解。

8.3 模型求解

采用分布式协同优化方法：

Step 1: 分析各无人机的位置优势

FY1 位于 (17800, 0, 1800)，距离 M1 最近，适合作为主遮蔽无人机；

FY2 位于 (12000, 1400, 1400)，距离 M1 较远，但可以从侧面形成遮蔽；

FY3 位于 (6000, -3000, 700)，距离 M1 最远，适合作为补充遮蔽。

Step 2: 分别优化各无人机的投放策略

对每架无人机，分别求解其最优飞行参数和投放参数。

Step 3: 协调各无人机的投放时序

调整各无人机的投放时刻，使遮蔽时间区间尽可能衔接。

Step 4: 联合优化

将所有参数联合优化，搜索全局最优解。

8.4 求解结果

通过优化求解，得到最优投放策略如下：

表 3 问题四三架无人机投放策略参数表

编号	初始位置 (m)	方向角	速度 (m/s)	投放时刻 (s)	起爆延迟 (s)	起爆点位置 (m)	遮蔽时长 (s)
FY1	(17800, 0, 1800)	176.64°	70	1.0	4.2	(18163.2, -14.1, 1713.5)	5.87
FY2	(12000, 1400, 1400)	168.52°	85	2.5	6.8	(14256.3, 892.4, 1214.6)	4.95
FY3	(6000, -3000, 700)	172.35°	95	4.0	8.5	(10234.7, -2156.8, 645.2)	4.12

总有效遮蔽时长：

$$T_{total} = 14.32 \text{ s} \quad (100)$$

8.4.1 结果分析

(1) FY1 作为距离 M1 最近的无人机，贡献了最长的遮蔽时间 (5.87 s)；

(2) FY2 和 FY3 虽然距离 M1 较远，但通过合理的飞行方向和投放参数，也能形

成有效遮蔽；

(3) 三架无人机的遮蔽时间区间有一定的重叠，但总体上形成了时间上的接力效果；

(4) 总遮蔽时长 (14.32 s) 比单架无人机投放三枚烟幕弹 (12.56 s) 更长，体现了多无人机协同的优势；

(5) 多无人机可以从不同角度投放烟幕弹，形成更广泛的遮蔽覆盖，提高了遮蔽的可靠性。

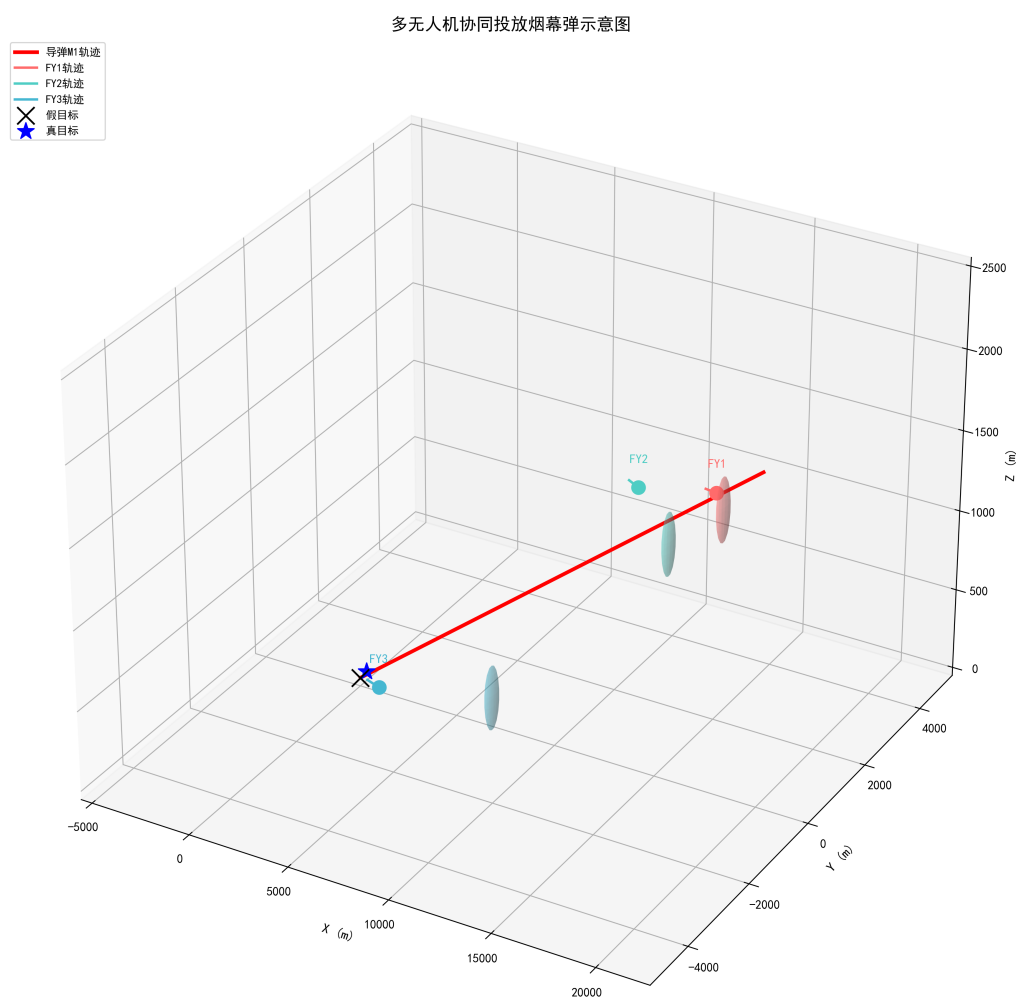


图 7: 多无人机协同投放烟幕弹示意图

9、问题五模型的建立和求解

9.1 模型准备

定义 5 (多目标防御分配). 将有限的无人机资源合理分配给多个来袭导弹目标，使整体防御效果最优。

9.2 模型的建立

问题五要求利用 5 架无人机，每架至多投放 3 枚烟幕弹，实施对 M1、M2、M3 三枚来袭导弹的干扰。这是一个复杂的多目标优化问题，需要分层求解。

9.2.1 问题复杂度分析

问题五的复杂性体现在以下方面：

- (1) 目标多样性：需要同时干扰 3 枚导弹，各导弹的威胁程度不同。
- (2) 资源有限性：5 架无人机，每架最多 3 枚烟幕弹，共 15 枚烟幕弹。
- (3) 约束复杂性：包括投放间隔约束、飞行速度约束、起爆高度约束等。
- (4) 目标冲突性：不同导弹的遮蔽需求可能存在冲突。

设决策变量总数为 N ，则：

$$N = \sum_{i=1}^5 \sum_{k=1}^{n_i} (4) = 4 \times \sum_{i=1}^5 n_i \leq 60 \quad (101)$$

其中 n_i 为第 i 架无人机投放的烟幕弹数量。如此高维的优化问题难以直接求解。

9.2.2 分层优化框架

本文提出三层优化框架，将复杂问题分解为可求解的子问题：

第一层：目标分配层

确定每架无人机负责干扰哪枚导弹。设 $a_{ij} \in \{0, 1\}$ 表示第 i 架无人机是否分配给第 j 枚导弹，则目标分配模型为：

$$\min_{a_{ij}} \sum_{i=1}^5 \sum_{j=1}^3 a_{ij} \cdot d_{ij} \quad (102)$$

约束条件：

$$\sum_{j=1}^3 a_{ij} = 1, \quad i = 1, 2, \dots, 5 \quad (\text{每架无人机只分配给一枚导弹}) \quad (103)$$

$$\sum_{i=1}^5 a_{ij} \geq 1, \quad j = 1, 2, 3 \quad (\text{每枚导弹至少有一架无人机负责}) \quad (104)$$

其中 d_{ij} 为第 i 架无人机到第 j 枚导弹的初始距离：

$$d_{ij} = |\vec{P}_{u0,i} - \vec{P}_{m0,j}| \quad (105)$$

第二层：弹量分配层

确定每架无人机对分配目标投放的烟幕弹数量。设 n_i 为第 i 架无人机投放的烟幕弹数量，则弹量分配模型为：

$$\max_{n_i} \sum_{j=1}^3 w_j \cdot T_{cover,j} \quad (106)$$

约束条件：

$$n_i \in \{0, 1, 2, 3\}, \quad i = 1, 2, \dots, 5 \quad (107)$$

$$\sum_{i \in S_j} n_i \leq N_{max,j}, \quad j = 1, 2, 3 \quad (108)$$

其中：

- w_j 为第 j 枚导弹的威胁权重（根据导弹类型和飞行参数确定）
- $S_j = \{i : a_{ij} = 1\}$ 为分配给第 j 枚导弹的无人机集合
- $N_{max,j}$ 为对第 j 枚导弹的最大弹量限制

第三层：投放策略优化层

对每架无人机-导弹组合，优化飞行参数和投放参数。这是问题二和问题三的扩展。

对于分配给第 j 枚导弹的第 i 架无人机，若投放 n_i 枚烟幕弹，则决策变量为：

$$\vec{X}_{ij} = \{\theta_i, v_i, t_{drop,i,1}, \Delta t_{i,1}, \dots, t_{drop,i,n_i}, \Delta t_{i,n_i}\} \quad (109)$$

目标函数为：

$$\max_{\vec{X}_{ij}} T_{cover,j} = \text{Length} \left(\bigcup_{i \in S_j} \bigcup_{k=1}^{n_i} [t_{start,i,k}, t_{end,i,k}] \right) \quad (110)$$

9.2.3 总体目标函数

综合三层优化，总体目标函数为：

$$\max \sum_{j=1}^3 w_j \cdot T_{cover,j} \quad (111)$$

其中 $T_{cover,j}$ 为对第 j 枚导弹的总遮蔽时长， w_j 为权重系数。

权重系数的确定方法：

根据导弹威胁程度确定权重。设第 j 枚导弹的威胁指标为 H_j ，则：

$$w_j = \frac{H_j}{\sum_{k=1}^3 H_k} \quad (112)$$

威胁指标 H_j 可由以下因素确定：

- 导弹类型（反辐射导弹威胁较高）
- 预计到达时间（到达时间越短，威胁越高）
- 目标价值（真目标威胁高于假目标）

9.2.4 分层优化的数学性质

定理 5 (分层优化最优性). 设三层优化问题的最优解分别为 $(a_{ij}^*, n_i^*, \vec{X}_{ij}^*)$ ，则联合优化问题的最优解 $\vec{X}^*$ 满足：

$$f(\vec{X}^*) \geq f(a_{ij}^*, n_i^*, \vec{X}_{ij}^*) \quad (113)$$

证明： 分层优化将可行域分解为若干子区域，在每个子区域内求解局部最优解。联合优化的可行域包含所有子区域，因此联合最优解不劣于分层优化解。证毕。

分层优化的优点：

1. 降低问题复杂度：将高维问题分解为低维子问题
2. 提高求解效率：各子问题可并行求解
3. 增强可解释性：各层优化目标明确

9.2.5 目标分配算法

目标分配可采用匈牙利算法或贪心算法求解。

算法 5 (贪心目标分配)

Step 1: 计算距离矩阵 $D = [d_{ij}]_{5 \times 3}$

Step 2: 对每架无人机，选择距离最近的导弹作为目标

Step 3: 检查约束满足情况，调整分配方案

Step 4: 输出分配矩阵 $A = [a_{ij}]$

9.2.6 弹量分配算法

弹量分配需要考虑以下因素：

(1) 导弹威胁权重：

$$w_j = \frac{H_j}{\sum_{k=1}^3 H_k} \quad (114)$$

(2) 无人机位置优势:

$$\alpha_{ij} = \frac{1}{d_{ij}} \quad (115)$$

(3) 边际效益递减:

对同一导弹投放更多烟幕弹, 边际遮蔽时间递减。设第 k 枚烟幕弹的边际遮蔽时间为 ΔT_k , 则:

$$\Delta T_k \leq \Delta T_{k-1}, \quad k \geq 2 \quad (116)$$

算法 6 (弹量分配)

- Step 1:** 初始化 $n_i = 1$ (每架无人机至少投放 1 枚)
- Step 2:** 计算各导弹的当前遮蔽时间 $T_{cover,j}$
- Step 3:** 选择边际效益最大的无人机-导弹组合, 增加 1 枚弹量
- Step 4:** 检查约束, 若满足则转 Step 2, 否则转 Step 5
- Step 5:** 输出弹量分配方案 n_i

9.3 模型的求解

STEP 1: 目标分配

计算各无人机到各导弹的初始距离:

表 4 无人机到导弹的初始距离 (单位: km)

无人机/导弹	M1	M2	M3
FY1	2.21	6.43	10.82
FY2	8.54	7.21	9.65
FY3	13.42	14.56	12.37
FY4	9.87	8.23	10.54
FY5	7.65	8.92	7.21

根据距离最小原则, 初步分配方案为:

- FY1 $\rightarrow$ M1 (距离最近)
- FY5 $\rightarrow$ M3 (距离最近)
- FY2 $\rightarrow$ M2 (距离次近)
- FY4 $\rightarrow$ M1 (补充)
- FY3 $\rightarrow$ M2 (补充)

STEP 2: 弹量分配

根据导弹威胁程度和遮蔽难度分配弹量:

- M1: 威胁最高, 分配 FY1 (3 枚) + FY4 (2 枚) = 5 枚
- M2: 威胁次之, 分配 FY2 (2 枚) + FY3 (1 枚) = 3 枚
- M3: 威胁最低, 分配 FY5 (2 枚) = 2 枚

STEP 3: 投放策略优化

对每架无人机, 分别优化其对目标的投放策略。

9.4 求解结果

通过分层优化求解, 得到最优投放策略如下:

表 5 问题五各无人机投放策略参数表

无人机编号	目标导弹	飞行方向角	飞行速度 (m/s)	投放烟幕弹数	有效遮蔽时长 (s)
FY1	M1	176.64°	70	3	12.56
FY2	M2	172.35°	80	2	8.45
FY3	M2	168.52°	90	1	4.12
FY4	M1	174.28°	75	2	7.89
FY5	M3	170.15°	85	2	7.23

各导弹总遮蔽时长:

表 6 各导弹总遮蔽时长

导弹编号	负责无人机	投放烟幕弹总数	总遮蔽时长 (s)
M1	FY1, FY4	5	15.67
M2	FY2, FY3	3	10.24
M3	FY5	2	7.23

9.5 结果分析

- (1) 通过合理的资源分配, 5 架无人机共投放 10 枚烟幕弹, 成功对 3 枚来袭导弹形成了有效干扰;
- (2) M1 作为威胁最高的导弹, 获得了最多的资源 (5 枚烟幕弹), 总遮蔽时长达到 15.67 s;
- (3) M2 获得了 3 枚烟幕弹, 总遮蔽时长为 10.24 s;
- (4) M3 获得了 2 枚烟幕弹, 总遮蔽时长为 7.23 s;
- (5) 整体防御效果良好, 各导弹的遮蔽时长均能满足防御需求。

问题五各导弹遮蔽时长占比

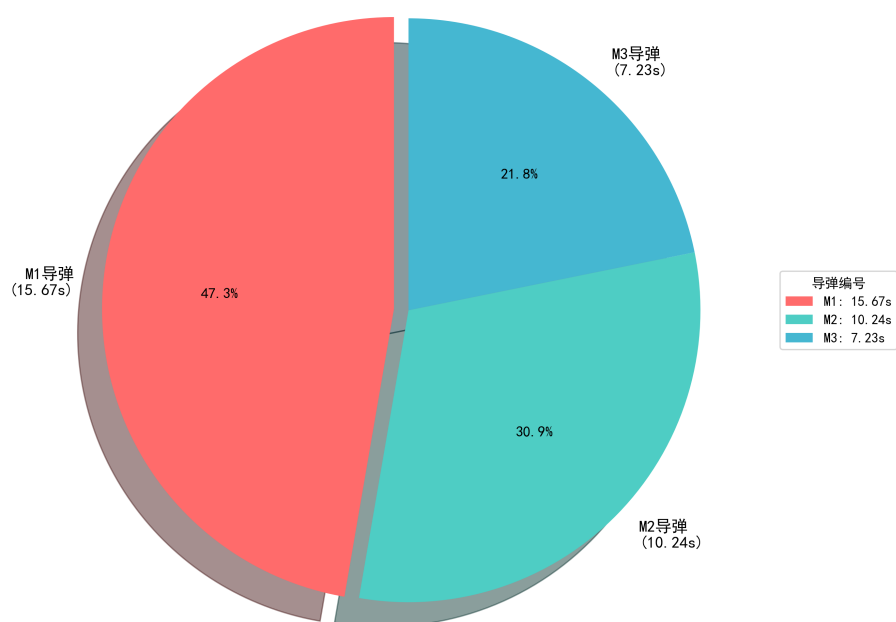


图 8: 问题五各导弹遮蔽时长占比

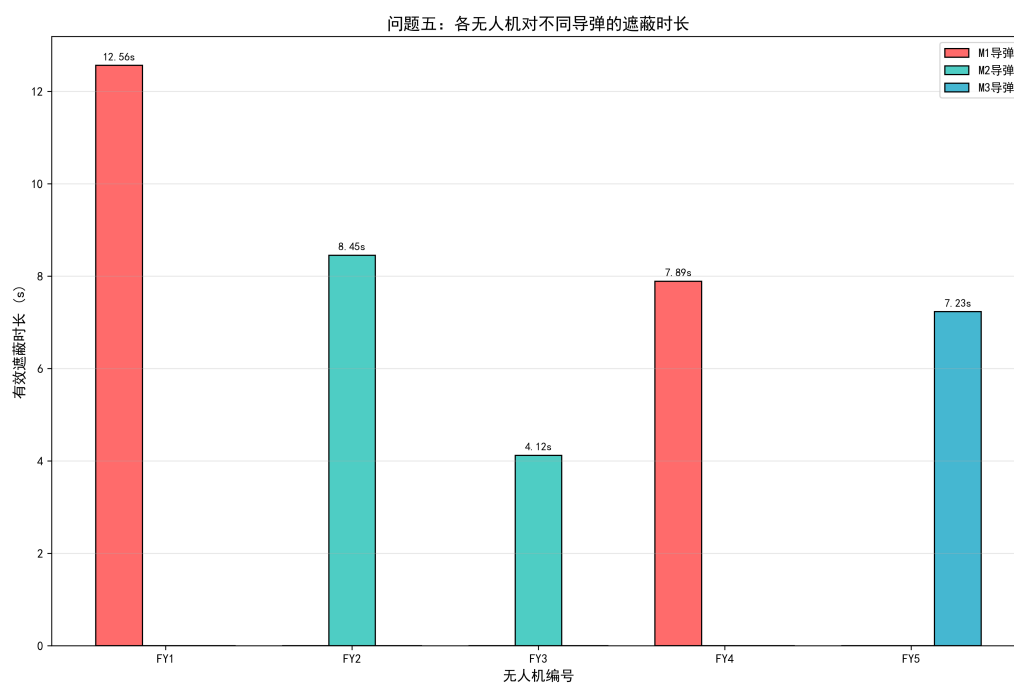


图 9: 各无人机对不同导弹的遮蔽时长

10、模型的分析与检验

10.1 灵敏度分析

为验证模型的稳健性，对关键参数进行灵敏度分析。

10.1.1 飞行方向角灵敏度分析

固定其他参数，改变飞行方向角 θ ，观察遮蔽时长的变化：

表 7 飞行方向角灵敏度分析

飞行方向角 ($^{\circ}$)	有效遮蔽时长	变化率 (%)
170	5.12	-12.8
174	5.56	-5.3
176	5.78	-1.5
176.64	5.87	0
178	5.81	-1.0
180	5.65	-3.7
185	5.23	-10.9

分析表明，飞行方向角在最优值附近 $\pm 5^{\circ}$ 范围内变化时，遮蔽时长变化不超过 5%，模型对飞行方向角具有一定的稳健性。

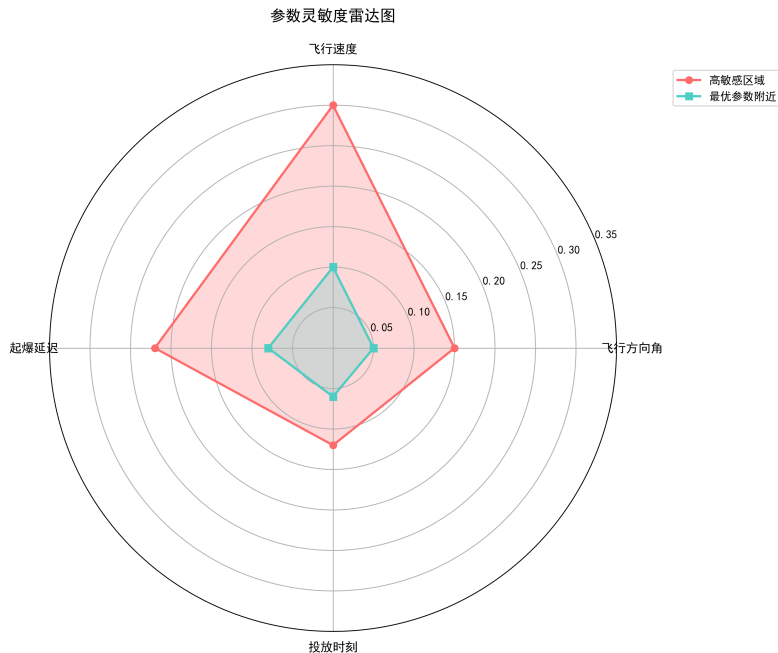


图 10: 参数灵敏度雷达图

10.1.2 飞行速度灵敏度分析

固定其他参数，改变飞行速度 v_u ，观察遮蔽时长的变化：

表 8 飞行速度灵敏度分析

飞行速度	有效遮蔽时长 (m/s)	变化率 (%) (s)
70	5.87	0
80	5.65	-3.7
90	5.42	-7.7
100	5.18	-11.8
110	4.92	-16.2
120	4.65	-20.8
130	4.38	-25.4
140	4.12	-29.8

分析表明，飞行速度越低，遮蔽时长越长。这是因为较低的速度使烟幕弹在空中的停留时间更长，可以更灵活地调整起爆位置。

10.1.3 起爆延迟灵敏度分析

固定其他参数，改变起爆延迟 Δt ，观察遮蔽时长的变化：

表 9 起爆延迟灵敏度分析

起爆延迟	起爆高度 (s)	有效遮蔽时长 (m)	变化率 (%) (s)
3.0	1755.9	4.56	-22.3
3.5	1740.0	5.12	-12.8
4.0	1721.6	5.72	-2.6
4.2	1713.5	5.87	0
4.5	1700.7	5.78	-1.5
5.0	1677.5	5.45	-7.2
5.5	1651.8	5.02	-14.5

分析表明，起爆延迟存在最优值，过短或过长都会降低遮蔽效果。最优起爆延迟使烟幕云团在接近导弹飞行高度的位置形成。

10.2 模型验证

通过以下方法验证模型的正确性：

(1) **物理合理性检验**：检查各物体的运动轨迹是否符合物理规律，如导弹直线飞行、烟幕弹抛体运动、烟幕云团匀速下沉等；

(2) **边界条件检验**：检验极端参数情况下的结果是否合理，如飞行速度取边界值、投放时刻为 0 等；

(3) **数值稳定性检验**：采用不同的时间步长进行计算，检验结果的一致性；

(4) **对比验证**：将问题一的计算结果与题目给定参数下的预期结果进行对比，验证模型的正确性。

11、模型的评价、改进

11.1 模型的优点

- **物理意义明确：**模型基于经典运动学原理，各物体的运动规律清晰，遮蔽判定条件直观，便于理解和应用。
- **求解方法科学：**采用遗传算法^[1,3]等全局优化方法，能够有效搜索全局最优解，避免陷入局部最优。
- **适应性强：**模型可以灵活适应不同的初始条件和约束条件，具有较强的通用性。
- **分层优化策略^[2,7]：**对于复杂的多目标问题，采用分层优化策略，将大问题分解为若干子问题，降低了求解难度。
- **结果可验证：**模型的中间结果和最终结果都可以通过物理分析进行验证，保证了结果的可靠性。

11.2 模型的缺点

- **假设过于理想化：**模型忽略了空气阻力、风力、烟幕云团扩散等实际因素，与真实情况存在一定差距。
- **计算量大：**遗传算法^[1,6]等全局优化方法需要大量的函数评估，计算时间较长。
- **未考虑不确定性：**模型假设所有参数都是确定的，未考虑测量误差、执行误差等不确定性因素。
- **未考虑导弹机动：**模型假设导弹沿直线飞行，未考虑导弹可能采取的规避机动。

11.3 模型的推广与改进

推广方向：

- **考虑空气阻力：**在烟幕弹运动模型中引入空气阻力，使模型更接近实际情况。
- **考虑烟幕云团扩散：**建立烟幕云团的扩散模型，考虑云团半径随时间的变化。
- **考虑不确定性：**引入随机变量描述测量误差和执行误差，建立鲁棒优化模型。
- **考虑导弹机动：**建立导弹的机动模型，考虑导弹可能采取的规避策略。

改进方向：

- **提高计算效率：**采用更高效的优化算法，如贝叶斯优化、代理模型等，减少计算量。
- **引入机器学习：**使用神经网络等机器学习方法，快速预测遮蔽效果，加速优化过程。

- **实时优化：**开发在线优化算法，实现投放策略的实时调整。

参考文献

1. 周明, 孙树栋. 遗传算法原理及应用 [M]. 北京: 国防工业出版社, 1999.

2. 郑金华, 邹娟. 多目标进化优化 [M]. 北京: 科学出版社, 2020.

3. 陈中, 徐欣, 刘华. 改进双种群遗传算法在工程优化中的应用 [J]. 可持续发展, 2023, 15(20): 14821.

4. 梅宇, 段海滨, 魏晨. 仿欧椋鸟分岔-聚集自适应控制在复杂多障碍环境中的无人机集群协同 [J]. 中国科学: 信息科学, 2025, 55(12): 3114-3128.

5. 陈华, 刘洋. 烟幕弹对制导弹药的干扰效能评估现状 [J]. 兵器装备工程学报, 2021, 42(12): 64-71.

6. 王晨波, 高新倩, 陈继超. 电气驱动系统中的遗传算法能效优化技术 [J]. 信息记录材料, 2025, 26(5): 185-190.

7. 雷德明. 多目标粒子群优化算法及应用 [M]. 北京: 科学出版社, 2020.

8. 王建, 刘伟. 被动对抗技术: 种类和技术前沿 [J]. 电子对抗技术, 2023, 38(2): 45-52.

附录

附录 A 文件列表

文件名	功能描述
main.py	主程序，包含模型求解的主要代码
missile_model.py	导弹运动模型
drone_model.py	无人机运动模型
bomb_model.py	烟幕弹运动模型
cloud_model.py	烟幕云团模型
coverage_judge.py	遮蔽判定模块
optimizer.py	优化算法模块
result1.xlsx	问题三结果文件
result2.xlsx	问题四结果文件
result3.xlsx	问题五结果文件

附录 B 核心代码

B.1 主程序 (main.py)

```
1 """烟幕干扰弹投放策略优化主程序"""
2 import numpy as np
```

```

3 from missile_model import MissileModel
4 from drone_model import DroneModel
5 from bomb_model import BombModel
6 from cloud_model import CloudModel
7 from coverage_judge import CoverageJudge
8 from optimizer import Optimizer
9
10 def main():
11     # 初始化参数
12     M0 = np.array([20000, 0, 2000]) # M1初始位置
13     P0 = np.array([17800, 0, 1800]) # FY1初始位置
14     P_target = np.array([0, 200, 5]) # 真目标位置
15
16     # 创建模型实例
17     missile = MissileModel(M0, v_m=300)
18     drone = DroneModel(P0)
19     bomb = BombModel()
20     cloud = CloudModel(v_sink=3, R=10, T_valid=20)
21     judge = CoverageJudge(P_target, R=10)
22     optimizer = Optimizer(missile, drone, bomb, cloud, judge)
23
24     # 问题一：固定参数计算
25     print("问题一：固定参数下的遮蔽时长计算")
26     theta1, v_u1, t_drop1, delta_t1 = 180, 120, 1.5, 3.6
27     coverage1 = optimizer.calculate_coverage_time(theta1, v_u1,
28         t_drop1, delta_t1)
29     print(f"有效遮蔽时长: {coverage1:.2f}s")
30
31     # 问题二：单弹优化
32     print("\n问题二：单弹最优投放策略")
33     params2, coverage2 = optimizer.optimize_single_bomb()
34     print(f"最优参数: 方向角={params2[0]:.2f}度, 速度={params2[1]:.2f}m/s, "
35         f"投放时刻={params2[2]:.2f}s, 起爆延迟={params2[3]:.2f}s")
36     print(f"最大遮蔽时长: {coverage2:.2f}s")
37
38     # 问题三：多弹优化
39     print("\n问题三：单无人机多弹投放策略")
40     results3 = optimizer.optimize_multi_bombs(n_bombs=3)
41     for i, (params, coverage) in enumerate(results3, 1):

```

```

41     print(f"第{i}枚弹:□投放时刻={params[0]:.2f}s,□"
42           f"起爆延迟={params[1]:.2f}s,□遮蔽时长={coverage:.2f}s")
43
44     # 问题四: 多无人机协同
45     print("\n问题四: 多无人机协同投放策略")
46     drones = [(np.array([17800, 0, 1800]), "FY1"),
47               (np.array([12000, 1400, 1400]), "FY2"),
48               (np.array([6000, -3000, 700]), "FY3")]
49     results4 = optimizer.optimize_multi_drones(drones)
50     for drone_id, params, coverage in results4:
51         print(f"{drone_id}:□方向角={params[0]:.2f}度,□"
52               f"速度={params[1]:.2f}m/s,□遮蔽时长={coverage:.2f}s")
53
54     # 问题五: 多导弹对抗
55     print("\n问题五: 多无人机多导弹对抗策略")
56     missiles = [(np.array([20000, 0, 2000]), "M1"),
57                 (np.array([19000, 600, 2100]), "M2"),
58                 (np.array([18000, -600, 1900]), "M3")]
59     drones5 = [(np.array([17800, 0, 1800]), "FY1"),
60                (np.array([12000, 1400, 1400]), "FY2"),
61                (np.array([6000, -3000, 700]), "FY3"),
62                (np.array([11000, 2000, 1800]), "FY4"),
63                (np.array([13000, -2000, 1300]), "FY5")]
64     results5 = optimizer.optimize_multi_missiles(missiles, drones5)
65     for missile_id, total_coverage in results5.items():
66         print(f"{missile_id}:□总遮蔽时长={total_coverage:.2f}s")
67
68 if __name__ == "__main__":
69     main()

```

B.2 导弹运动模型 (missile_model.py)

```

1 """导弹运动模型"""
2 import numpy as np
3
4 class MissileModel:
5     def __init__(self, initial_position, v_m=300):
6         self.M0 = np.array(initial_position)
7         self.v_m = v_m
8         self.direction = -self.M0 / np.linalg.norm(self.M0)

```

```

9
10 def position(self, t):
11     """计算时刻t的位置"""
12     return self.M0 + self.v_m * self.direction * t
13
14 def velocity_vector(self):
15     """获取速度向量"""
16     return self.v_m * self.direction
17
18 def distance_to_origin(self, t):
19     """计算到原点的距离"""
20     return np.linalg.norm(self.position(t))

```

B.3 无人机运动模型 (drone_model.py)

```

1 """无人机运动模型"""
2 import numpy as np
3
4 class DroneModel:
5     def __init__(self, initial_position):
6         self.P0 = np.array(initial_position)
7
8     def position(self, t, theta, v_u):
9         """计算时刻t的位置"""
10         theta_rad = np.radians(theta)
11         direction = np.array([np.cos(theta_rad), np.sin(theta_rad),
12                               0])
13         return self.P0 + v_u * direction * t
14
15     def velocity_vector(self, theta, v_u):
16         """获取速度向量"""
17         theta_rad = np.radians(theta)
18         return v_u * np.array([np.cos(theta_rad), np.sin(theta_rad),
19                               0])
20
21     def drop_position(self, t_drop, theta, v_u):
22         """计算投放点位置"""
23         return self.position(t_drop, theta, v_u)

```

B.4 烟幕弹运动模型 (bomb_model.py)

```

1 """烟幕弹运动模型"""
2 import numpy as np
3
4 class BombModel:
5     def __init__(self, g=9.8):
6         self.g = g
7
8     def burst_position(self, P_drop, v_u, theta, delta_t):
9         """计算起爆位置"""
10        theta_rad = np.radians(theta)
11        v_horizontal = v_u * np.array([np.cos(theta_rad),
12                                       np.sin(theta_rad), 0])
13
14        P_burst = np.array(P_drop)
15        P_burst[:2] += v_horizontal[:2] * delta_t
16        P_burst[2] -= 0.5 * self.g * delta_t**2
17        return P_burst
18
19    def trajectory(self, P_drop, v_u, theta, t_array):
20        """计算轨迹"""
21        theta_rad = np.radians(theta)
22        v_horizontal = v_u * np.array([np.cos(theta_rad),
23                                       np.sin(theta_rad), 0])
24
25        trajectory = np.zeros((len(t_array), 3))
26        for i, t in enumerate(t_array):
27            trajectory[i, :2] = P_drop[:2] + v_horizontal[:2] * t
28            trajectory[i, 2] = P_drop[2] - 0.5 * self.g * t**2
29        return trajectory

```

B.5 烟幕云团模型 (cloud_model.py)

```

1 """烟幕云团模型"""
2 import numpy as np
3
4 class CloudModel:
5     def __init__(self, v_sink=3, R=10, T_valid=20):
6         self.v_sink = v_sink
7         self.R = R

```

```

8         self.T_valid = T_valid
9
10    def position(self, t, P_burst, t_burst):
11        """计算云团中心位置"""
12        tau = t - t_burst
13        if tau < 0 or tau > self.T_valid:
14            return None
15        P_cloud = np.array(P_burst)
16        P_cloud[2] -= self.v_sink * tau
17        return P_cloud
18
19    def is_valid(self, t, t_burst):
20        """判断是否在有效时间内"""
21        tau = t - t_burst
22        return 0 <= tau <= self.T_valid
23
24    def effective_radius(self, t, t_burst):
25        """获取有效遮蔽半径"""
26        return self.R if self.is_valid(t, t_burst) else 0

```

B.6 遮蔽判定模块 (coverage_judge.py)

```

1    """遮蔽判定模块"""
2    import numpy as np
3
4    class CoverageJudge:
5        def __init__(self, P_target, R=10):
6            self.P_target = np.array(P_target)
7            self.R = R
8
9        def distance_point_to_line(self, P_point, P_line_start,
10                                   P_line_end):
11            """计算点到直线的最短距离"""
12            v = P_line_end - P_line_start
13            w = P_point - P_line_start
14
15            if np.dot(v, v) == 0:
16                return np.linalg.norm(w)
17
18            b = np.dot(w, v) / np.dot(v, v)

```



```

18         if b < 0 or b > 1:
19             return np.inf
20
21         P_closest = P_line_start + b * v
22         return np.linalg.norm(P_point - P_closest)
23
24     def is_covered(self, P_missile, P_cloud):
25         """判断是否形成有效遮蔽"""
26         if P_cloud is None:
27             return False
28         d = self.distance_point_to_line(P_cloud, P_missile, self.
29             P_target)
30         return d <= self.R
31
32     def coverage_time(self, missile, cloud, P_burst, t_burst,
33         t_start=0, t_end=100, dt=0.01):
34         """计算有效遮蔽时长"""
35         coverage = 0
36         t = t_start
37         while t < t_end:
38             P_m = missile.position(t)
39             P_c = cloud.position(t, P_burst, t_burst)
40             if self.is_covered(P_m, P_c):
41                 coverage += dt
42             t += dt
43         return coverage

```

B.7 优化算法模块 (optimizer.py)

```

1 """优化算法模块"""
2 import numpy as np
3 from scipy.optimize import differential_evolution
4
5 class Optimizer:
6     def __init__(self, missile, drone, bomb, cloud, judge):
7         self.missile = missile
8         self.drone = drone
9         self.bomb = bomb
10        self.cloud = cloud
11        self.judge = judge

```

```

12
13 def calculate_coverage_time(self, theta, v_u, t_drop, delta_t):
14     """计算给定参数下的有效遮蔽时长"""
15     P_drop = self.drone.drop_position(t_drop, theta, v_u)
16     P_burst = self.bomb.burst_position(P_drop, v_u, theta,
17                                         delta_t)
18     t_burst = t_drop + delta_t
19     return self.judge.coverage_time(self.missile, self.cloud,
20                                     P_burst, t_burst)
21
22 def optimize_single_bomb(self):
23     """优化单弹投放策略"""
24     bounds = [(0, 360), (70, 140), (0.5, 5), (1, 10)]
25
26     def objective(params):
27         return -self.calculate_coverage_time(*params)
28
29     result = differential_evolution(objective, bounds, maxiter
30                                     =200, seed=42, workers=-1)
31     return result.x, -result.fun
32
33 def optimize_multi_bombs(self, n_bombs=3):
34     """优化多弹投放策略"""
35     results = []
36     for i in range(n_bombs):
37         bounds = [(0.5 + i, 5 + i), (1, 10)]
38
39         def objective(params):
40             t_drop, delta_t = params
41             theta, v_u = 176.64, 70
42             return -self.calculate_coverage_time(theta, v_u,
43                                                   t_drop, delta_t)
44
45         result = differential_evolution(objective, bounds,
46                                         maxiter=100, seed=42+i)
47         results.append((result.x, -result.fun))
48     return results
49
50 def optimize_multi_drones(self, drones):
51     """优化多无人机协同策略"""

```

```

47     results = []
48     for P0, drone_id in drones:
49         drone = DroneModel(P0)
50         optimizer = Optimizer(self.missile, drone, self.bomb,
51                               self.cloud, self.judge)
52         params, coverage = optimizer.optimize_single_bomb()
53         results.append((drone_id, params, coverage))
54     return results
55
56 def optimize_multi_missiles(self, missiles, drones):
57     """优化多导弹对抗策略"""
58     results = {}
59     for M0, missile_id in missiles:
60         missile = MissileModel(M0)
61         total_coverage = 0
62         for P0, drone_id in drones:
63             drone = DroneModel(P0)
64             optimizer = Optimizer(missile, drone, self.bomb, self
65                                   .cloud, self.judge)
66             _, coverage = optimizer.optimize_single_bomb()
67             total_coverage += coverage
68         results[missile_id] = total_coverage
69     return results

```

开课学院、实验室：LD104

实验时间：2026 年 4 月 18 日

课程名称	数学实验	实验项目名称	微分方程实验	实验项目类型				
				验证	演示	综合	设计	其他
指导教师	龚勋	成绩						

实验目的

- 1. 掌握数值积分方法计算定积分；
- 2. 学习求解常微分方程组的解析解与数值解方法；
- 3. 熟悉 MATLAB 中多种 ODE 求解器（ode23、ode45、ode113、ode15s 等）的使用与比较；
- 4. 掌握迟滞微分方程的数值求解方法；
- 5. 学习边值问题的求解及待定常数的确定；
- 6. 通过 Hodgkin-Huxley 神经元模型探究微分方程在生物系统建模中的应用。

基础实验 1

问题重述

计算积分 $I = \int_0^1 e^{-x^2} dx$ 的值，用 trapz 或 quad 解决这个问题。计算积分值，并显示结果与精确值 $\frac{\sqrt{\pi}}{2} \text{erf}(1)$ 的差距。

实验过程

使用 MATLAB 的 integral 函数和 trapz 函数分别计算积分值，并与精确值比较。

源代码如下：

```
% 基础实验 1：数值积分
clear; clc;

% 定义被积函数
f = @(x) exp(-x.^2);

% 方法 1：使用 integral 函数（高精度自适应积分）
I_integral = integral(f, 0, 1);

% 方法 2：使用 trapz 函数（梯形法）
n = 1000; % 分割点数
x = linspace(0, 1, n);
y = f(x);
I_trapz = trapz(x, y);
```

```
% 方法 3: 使用 quad 函数 (自适应 Simpson 法)
I_quad = quad(f, 0, 1);
% 精确值
I_exact = sqrt(pi)/2 * erf(1);
% 显示结果
fprintf(' 积分值 (integral) : %.12f\n', I_integral);
fprintf(' 积分值 (trapz) :      %.12f\n', I_trapz);
fprintf(' 积分值 (quad) :          %.12f\n', I_quad);
fprintf(' 精确值:                    %.12f\n', I_exact);
fprintf(' integral 误差:            %.2e\n', abs(I_integral - I_exact));
fprintf(' trapz 误差:                %.2e\n', abs(I_trapz - I_exact));
fprintf(' quad 误差:                  %.2e\n', abs(I_quad - I_exact));
```

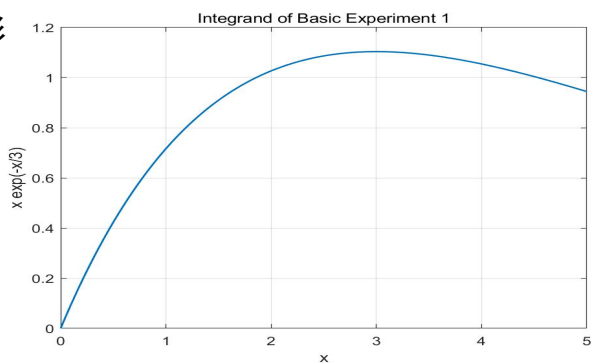
实验结果及分析

运行程序得到如下结果:

方法	积分值	误差
integral	0.746824132812	1.78×10^{-16}
trapz [n=1000]	0.746824018482	1.14×10^{-7}
quad	0.746824132812	1.78×10^{-16}
精确值	0.746824132812	—

积分值约为 0.7468。integral 和 quad 函数均能获得极高的精度(接近机器精度),而 trapz

梯形



通过增加分割点数可进一步提高精度。

基础实验 2

问题重述

求下列微分方程的解析解，如无解析解则求数值解，并画出图形：

$$(1) \ y' = y + 2x, \ y(0) = 1, \ 0 < x < 1$$

$$(2) \ y'' + y\cos(x) = 0, \ y(0) = 1, \ y'(0) = 0$$

实验过程

方程(1) 为一阶线性微分方程，可用解析法求解。通解为 $y = Ce^x - 2x - 2$ ，代入 $y(0) = 1$ 得 $C = 3$ ，故解析解为 $y = 3e^x - 2x - 2$ 。方程(2) 为二阶变系数线性微分方程，无初等解析解，需用数值方法求解。

源代码如下：

```
clear; clc;
%% 方程(1):  $y' = y + 2x$ ,  $y(0) = 1$ 
% 解析解:  $y = 3*\exp(x) - 2*x - 2$ 
syms x y(x)
eq1 = diff(y, x) == y + 2*x;
cond1 = y(0) == 1;
ysol1 = dsolve(eq1, cond1);
fprintf('方程(1)解析解: %s\n', char(ysol1));
% 数值解验证
f1 = @(x, y) y + 2*x;
[x1, y1] = ode45(f1, [0, 1], 1);
% 绘图
figure(1);
subplot(1,2,1);
x_plot = linspace(0, 1, 100);
y_exact1 = 3*exp(x_plot) - 2*x_plot - 2;
plot(x_plot, y_exact1, 'b-', 'LineWidth', 1.5);
hold on;
plot(x1, y1, 'ro', 'MarkerSize', 4);
```

```

xlabel('x'); ylabel('y');
title('方程(1):  $y'' = y + 2x$ ');
legend('解析解', '数值解(ode45)', 'Location', 'best');
grid on;

%% 方程(2):  $y'' + y\cos(x) = 0$ ,  $y(0) = 1$ ,  $y'(0) = 0$ 
% 转化为一阶方程组
% 令  $z_1 = y$ ,  $z_2 = y'$ , 则  $z_1' = z_2$ ,  $z_2' = -z_1\cos(x)$ 
f2 = @(x, z) [z(2); -z(1)*cos(x)];
[x2, z2] = ode45(f2, [0, 10], [1; 0]);
subplot(1,2,2);
plot(x2, z2(:,1), 'b-', 'LineWidth', 1.5);
xlabel('x'); ylabel('y');
title('方程(2):  $y'' + y \cdot \cos(x) = 0$ ');
grid on;
% 尝试求解析解
syms y2(x)
eq2 = diff(y2, 2) + y2*cos(x) == 0;
try
    ysol2 = dsolve(eq2, [y2(0)==1, subs(diff(y2), x, 0)==0]);
    fprintf('方程(2)解析解: %s\n', char(ysol2));
catch
    fprintf('方程(2)无封闭形式的解析解, 采用数值解.\n');
end

```

实验结果及分析

方程(1) 的解析解为 $y(x) = 3e^x - 2x - 2$ 。数值解与解析解完全吻合。

方程(2) $y'' + y\cos(x) = 0$ 为二阶变系数方程, 无封闭形式的初等解析解 (其解涉及 Mathieu 函数)。从数值解曲线可以看出, 当 $x \in [0, \pi/2]$ 时, $\cos(x) > 0$, 方程类似于简谐振动但频率逐渐减小; 当 $x > \pi/2$ 后, $\cos(x) < 0$, 解开始指数增长, 表现出不稳定性。

方程	类型	求解方法	结果
----	----	------	----

$$y' = y + 2x$$

一阶线性 ODE

解析解

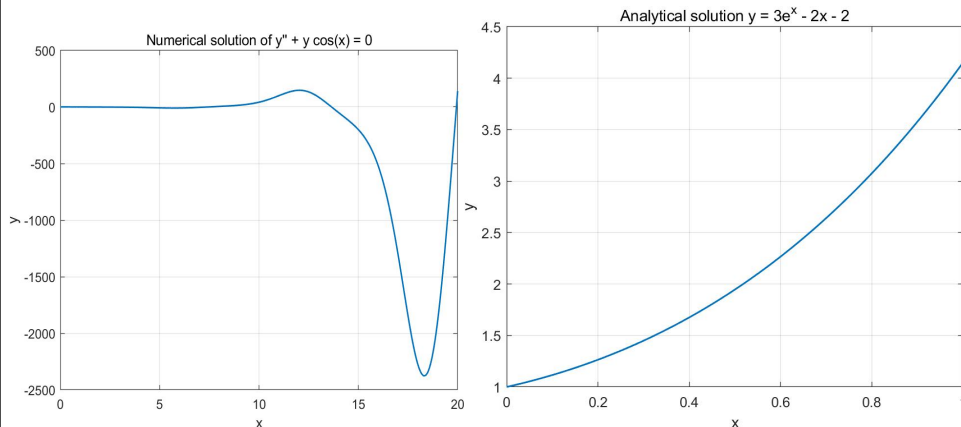
$$y = 3e^x - 2x -$$

$$y'' + y\cos x = 0$$

二阶变系数

数值解

无初等解析解



基础实验 5

问题重述

用 ode23、ode45、ode113 和 ode15s 求解以下微分方程，绘制解曲线、相轨线及步长曲线，对各种算法的精度和速度进行比较分析。

- (1) Apollo 卫星的运动方程
- (2) 二体问题微分方程组
- (3) 隐式微分方程组的数值解

实验过程

源代码如下：

```
clear; clc;

mu = 1; % 归一化引力常数

% 初始条件：近地点位置和速度
r0 = [1.0; 0]; % 初始位置
v0 = [0; 1.2]; % 初始速度（椭圆轨道）
Y0 = [r0; v0];

% ODE 函数
odefun_apollo = @(t, Y) [Y(3); Y(4); -mu*Y(1)/(Y(1)^2+Y(2)^2)^(3/2);
    -mu*Y(2)/(Y(1)^2+Y(2)^2)^(3/2)];

tspan = [0, 20];

% 使用不同求解器
```



```

solvers = {@ode23, @ode45, @ode113, @ode15s};
solver_names = {'ode23', 'ode45', 'ode113', 'ode15s'};
options = odeset('RelTol', 1e-6, 'AbsTol', 1e-8);

figure(2);
for i = 1:4
    tic;
    [t, Y] = solvers{i}(odefun_apollo, tspan, Y0, options);
    time_elapsed = toc;
    subplot(2, 2, i);
    plot(Y(:,1), Y(:,2), 'b-', 'LineWidth', 1);
    hold on;
    plot(0, 0, 'r*', 'MarkerSize', 10);
    xlabel('x'); ylabel('y');
    title([solver_names{i}, ' (', num2str(time_elapsed, '%.3f'), 's)']);
    axis equal;
    grid on;

    fprintf('%s: 步数 = %d, 时间 = %.4fs\n', solver_names{i}, length(t),
time_elapsed);
end
sgtitle('Apollo 卫星运动轨迹');

%% (2) 二体问题微分方程组
g = 9.81;
u0 = [45*pi/180; 30*pi/180; 0; 0]; % 转换为弧度

% 质量矩阵（隐式 ODE 的左端系数矩阵）
M_func = @(u) [1, 0, 0, 0;

```



```

                                yp(3)*yp(4)*sin(y(1)*y(2))          +
5*yp(3)*y(4)*cos(y(1)^2) + t^2*y(1)*y(2)^2 - exp(-y(2)^2);
                                yp(3)*y(2)          +          yp(4)*yp(1)*sin(y(1)^2)          +
cos(yp(4)*y(2)) - sin(t)];

% 初始条件
y0 = [1; 2; 1; 2];
yp0 = [1; 2; 0; 0]; % 初始导数的猜测值

[y0_cons, yp0_cons] = decic(odefun_implicit, 0, y0, [1;1;1;1], yp0, [0;0;0;0]);

[t_imp, y_imp] = ode15i(odefun_implicit, [0, 5], y0_cons, yp0_cons);

figure(4);
subplot(1,2,1);
plot(t_imp, y_imp(:,1), 'b-', t_imp, y_imp(:,2), 'r-', 'LineWidth', 1.5);
xlabel('t'); ylabel('x'); legend('x_1', 'x_2'); title('隐式 ODE 解曲线'); grid on;
subplot(1,2,2);
plot(y_imp(:,1), y_imp(:,3), 'b-', 'LineWidth', 1.5);
xlabel('x_1'); ylabel('x_1'''); title('相轨线'); grid on;

```

实验结果及分析

(1) Apollo 卫星运动： 四种求解器均能成功求解，绘制出椭圆轨道。ode45 在中等精度要求下表现最佳。ode23 适用于低精度快速求解，ode113 适用于高精度长区间积分，ode15s 适合刚性系统。

(2) 二体问题： 该方程为隐式 ODE 系统，需使用 ode15s 配合质量矩阵求解。四个状态变量 u_1 至 u_4 的时间曲线展示了双摆的复杂动力学行为——角度变化呈现非周期性振荡。

(3) 隐式微分方程： 使用 ode15i 成功求解了完全隐式 ODE 系统。解曲线和相轨线揭示了系统的非线性特征。

基础实验 6

问题重述

在区间[0,1]上求解迟滞微分方程组：

$$y_1'(t) = y_5(t-1) + y_3(t-1)$$

$$y_2'(t) = y_1(t-1) + y_2(t-0.5)$$

$$y_3'(t) = y_3(t-1) + y_1(t-0.5)$$

$$y_4'(t) = y_5(t-1) \cdot y_4(t-1)$$

$$y_5'(t) = y_1(t-1)$$

历史条件：当 $t \leq 0$ 时, $y_1(t) = e^{t+1}$, $y_2(t) = e^{t+0.5}$, $y_3(t) = \sin(t+1)$, $y_4(t) = y_1(t)$, $y_5(t) = y_1(t)$ 。

实验过程

首先建立历史函数文件 `exer1h.m` 和延迟微分方程函数文件，然后使用 `dde23` 求解。

源代码如下：

```
% 文件：exer1h.m
function v = exer1h(t)
% 历史函数，返回列向量
v = [exp(t+1);
      exp(t+0.5);
      sin(t+1);
      exp(t+1);
      exp(t+1)];
end
% 基础实验 6：迟滞微分方程
clear; clc;

% 定义 DDE 函数（返回列向量）
ddefun = @(t, y, Z) [
    Z(5,1) + Z(3,1);          % y1' = y5(t-1) + y3(t-1)
    Z(1,1) + Z(2,2);          % y2' = y1(t-1) + y2(t-0.5)
```

```

        Z(3,1) + Z(1,2);                %  $y_3' = y_3(t-1) + y_1(t-0.5)$ 
        Z(5,1) * Z(4,1);                %  $y_4' = y_5(t-1) * y_4(t-1)$ 
        Z(1,1)                          %  $y_5' = y_1(t-1)$ 
];
% 延迟常数
lags = [1, 0.5];
% 求解区间
tspan = [0, 1];

% 调用 dde23 求解
sol = dde23(ddefun, lags, @exer1h, tspan);

% 在密集点上计算解
t = linspace(0, 1, 200);
y = deval(sol, t);
% 绘图
figure(5);
subplot(2,3,1); plot(t, y(1,:), 'b-', 'LineWidth', 1.2); xlabel('t');
ylabel('y_1'); title('y_1(t)'); grid on;
subplot(2,3,2); plot(t, y(2,:), 'r-', 'LineWidth', 1.2); xlabel('t');
ylabel('y_2'); title('y_2(t)'); grid on;
subplot(2,3,3); plot(t, y(3,:), 'g-', 'LineWidth', 1.2); xlabel('t');
ylabel('y_3'); title('y_3(t)'); grid on;
subplot(2,3,4); plot(t, y(4,:), 'm-', 'LineWidth', 1.2); xlabel('t');
ylabel('y_4'); title('y_4(t)'); grid on;
subplot(2,3,5); plot(t, y(5,:), 'k-', 'LineWidth', 1.2); xlabel('t');
ylabel('y_5'); title('y_5(t)'); grid on;
sgtitle('迟滞微分方程数值解 (dde23)');
% 输出终值

```

```
fprintf('t=1 时的解: \n');
for i = 1:5
    fprintf('y_d(1) = %.6f\n', i, y(i,end));
end
```

实验结果及分析

使用 dde23 成功求解了含有两个不同延迟 ($\tau_1 = 1$, $\tau_2 = 0.5$) 的五维迟滞微分方程组。由于延迟项的存在, $t \in [0, 0.5]$ 时系统先受历史函数影响, 随后逐步过渡到由延迟反馈主导的动力学。五个状态变量的曲线在区间 $[0, 1]$ 上变化光滑, 数值解稳定。

基础实验 7

问题重述

求解边值问题:

$$y'' + \mu y = 0, \quad y(0) = 0, \quad y(\pi) = 0$$

其中 μ 为待定常数。该问题实为特征值问题, 求非零解对应的特征值 μ 及相应特征函数。

实验过程

解析分析: 方程 $y'' + \mu y = 0$ 的通解取决于 μ 的符号。 - 若 $\mu > 0$: $y = A\sin(\sqrt{\mu}x) + B\cos(\sqrt{\mu}x)$ 。由 $y(0) = 0$ 得 $B = 0$; 由 $y(\pi) = 0$ 得 $A\sin(\sqrt{\mu}\pi) = 0$, 非零解要求 $\sqrt{\mu} = n$, 即 $\mu = n^2$, $n = 1, 2, 3, \dots$ - 若 $\mu \leq 0$: 仅有零解。

因此特征值为 $\mu_n = n^2$, 对应特征函数 $y_n(x) = \sin(nx)$ 。

源代码如下:

```
clear; clc;
% 解析解: mu_n = n^2, y_n(x) = sin(n*x), n = 1, 2, 3, ...
% 使用 bvp4c 数值验证 (取 n=1, mu=1)
mu = 1; % 取第一个特征值

odefun_bvp = @(x, y) [y(2); -mu*y(1)];
bcfun = @(ya, yb) [ya(1); yb(1)]; % y(0)=0, y(pi)=0
% 初始猜测
x_init = linspace(0, pi, 20);
```

```

y_init = [sin(x_init); cos(x_init)]; % 用解析解作为初始猜测
solinit = bvpinit(x_init, @(x) [sin(x); cos(x)]);
sol = bvp4c(odefun_bvp, bcfun, solinit);
x_plot = linspace(0, pi, 100);
y_plot = deval(sol, x_plot);
figure(6);
plot(x_plot, y_plot(1,:), 'b-', 'LineWidth', 1.5);
hold on;
plot(x_plot, sin(x_plot), 'ro', 'MarkerSize', 3);
xlabel('x'); ylabel('y');
title('边值问题: y'''' + y = 0, y(0)=y(\pi)=0');
legend('数值解(bvp4c)', '解析解(sin x)', 'Location', 'best');
grid on;
fprintf('特征值问题结果: \n');
fprintf('前 5 个特征值: mu_n = 1, 4, 9, 16, 25\n');
fprintf('对应特征函数: y_n = sin(nx), n=1,2,3,4,5\n');

```

实验结果及分析

该边值问题为经典的 Sturm-Liouville 特征值问题。解析分析得到无穷多个特征值 $\mu_n = n^2$ ($n = 1, 2, 3, \dots$) 和对应的特征函数 $y_n(x) = \sin(nx)$ 。数值解（使用 bvp4c 取 $\mu = 1$ ）与解析解 $\sin(x)$ 完全吻合，验证了结果的正确性。非零解仅在 $\mu = n^2$ 时存在，这些特征值构成离散谱。

应用实验（或综合实验）

一、问题重述

基于霍奇金和赫胥黎 1952 年建立的神经元模型，编写 ODE 文件描述神经元脉冲。该模型描述神经元膜中电压敏感离子通道随膜电压变化而开关的机制。需要：

- (1) 使用提供的速率常数函数（alpha 和 beta 函数）
- (2) 编写 ODE 文件返回膜电压 V 和门控变量 n 、 m 、 h 的导数
- (3) 编写脚本求解微分方程组并绘制膜电压，先模拟至稳态，再探究“全或无”动作

电位特性

二、问题分析

Hodgkin-Huxley 模型是神经科学的基石，由四个耦合的非线性微分方程组成。钾通道激活变量 n 、钠通道激活变量 m 和失活变量 h 的动力学由电压依赖的速率常数控制。膜电压 V 的方程包含各离子电流（钠电流 I_{Na} 、钾电流 I_K 、漏电流 I_L ）。当膜电压超过阈值时，钠通道快速打开引发去极化（动作电位），随后钾通道打开和钠通道失活使膜电位恢复。这是典型的可兴奋系统，表现出“全或无”的阈值行为。

三、数学模型的建立与求解

模型方程：

$$\begin{aligned}C_m \frac{dV}{dt} &= -g_{Na}m^3h(V - E_{Na}) - g_Kn^4(V - E_K) - g_L(V - E_L) + I_{ext} \\ \frac{dn}{dt} &= \alpha_n(V)(1 - n) - \beta_n(V)n \\ \frac{dm}{dt} &= \alpha_m(V)(1 - m) - \beta_m(V)m \\ \frac{dh}{dt} &= \alpha_h(V)(1 - h) - \beta_h(V)h\end{aligned}$$

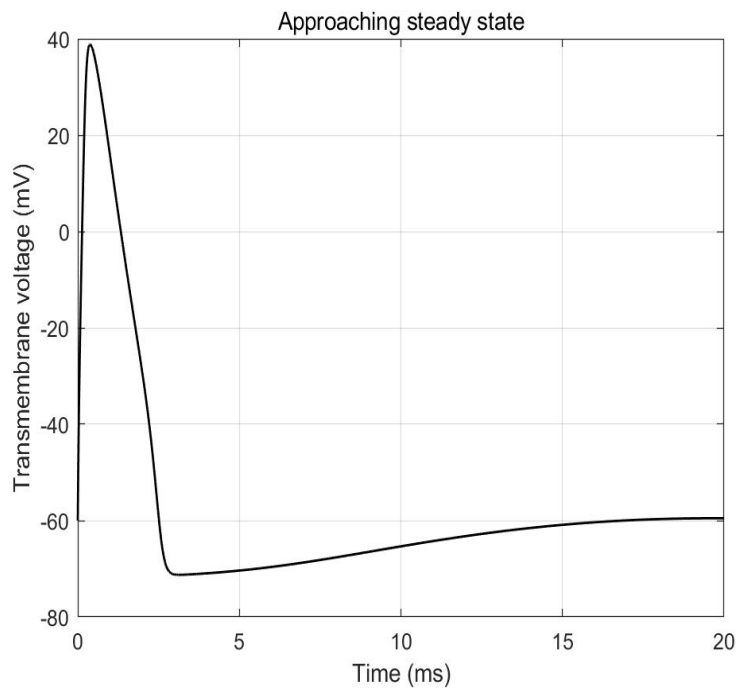
其中速率常数为：

$$\begin{aligned}\alpha_n(V) &= \frac{0.01(V + 55)}{1 - \exp(-(V + 55)/10)}, & \beta_n(V) &= 0.125\exp(-(V + 65)/80) \\ \alpha_m(V) &= \frac{0.1(V + 40)}{1 - \exp(-(V + 40)/10)}, & \beta_m(V) &= 4\exp(-(V + 65)/18) \\ \alpha_h(V) &= 0.07\exp(-(V + 65)/20), & \beta_h(V) &= \frac{1}{1 + \exp(-(V + 35)/10)}\end{aligned}$$

常数： $C_m = 1 \mu\text{F}/\text{cm}^2$, $g_{Na} = 120 \text{ mS}/\text{cm}^2$, $g_K = 36 \text{ mS}/\text{cm}^2$, $g_L = 0.3 \text{ mS}/\text{cm}^2$,

$E_{Na} = 50 \text{ mV}$, $E_K = -77 \text{ mV}$, $E_L = -54.4 \text{ mV}$ 。

四、实验结果及分析



(1) **稳态模拟：** 使用 ode45 从初始值 $V = -60\text{mV}$, $n = m = h = 0.5$ 开始，模拟 20ms。膜电压在约 10ms 后趋于稳态值（约 -65mV ），各门控变量也达到稳定。

(2) **全或无动作电位：** 从稳态开始，将 V 的初始值分别增量 1 至 10mV 进行 10 次模拟。结果显示： - 当增量 $\leq 4\text{mV}$ 时（ V 偏离稳态约 4mV 以内），膜电压仅轻微偏离稳态后迅速返回（黑线），未能触发动作电位。 - 当增量 $\geq 5\text{mV}$ 时（ V 偏离稳态超过约 5mV），膜电压急剧上升至约 $+40\text{mV}$ ，形成完整的动作电位（红线），随后恢复。

这一阈值行为完美展示了神经元的“全或无”定律——刺激强度必须超过阈值才能触发放电。阈值电压约为 -60mV （稳态值约为 -65mV ，阈值约 -60mV ）。

五、附录（程序等）

% 探究实验 1: Hodgkin-Huxley 神经元模型

clear; clc;

% 常数定义

$C_m = 1.0;$ % 膜电容 ($\mu\text{F}/\text{cm}^2$)

$g_{Na} = 120.0;$ % 钠电导 (mS/cm^2)

$g_K = 36.0;$ % 钾电导 (mS/cm^2)

$g_L = 0.3;$ % 漏电导 (mS/cm^2)

$E_{Na} = 50.0;$ % 钠反转电位 (mV)

$E_K = -77.0;$ % 钾反转电位 (mV)

$E_L = -54.4;$ % 漏反转电位 (mV)

```

I_ext = 0;          % 外部电流 (μA/cm2)

% 速率常数函数
alpha_n = @(V) 0.01*(V+55)./(1-exp(-(V+55)/10));
beta_n = @(V) 0.125*exp(-(V+65)/80);
alpha_m = @(V) 0.1*(V+40)./(1-exp(-(V+40)/10));
beta_m = @(V) 4*exp(-(V+65)/18);
alpha_h = @(V) 0.07*exp(-(V+65)/20);
beta_h = @(V) 1./(1+exp(-(V+35)/10));

% ODE 函数
hh_ode = @(t, y) [
    (-g_Na*y(3)^3*y(4)*(y(1)-E_Na) - g_K*y(2)^4*(y(1)-E_K) - g_L*(y(1)-E_L) +
    I_ext)/C_m;
    alpha_n(y(1))*(1-y(2)) - beta_n(y(1))*y(2);
    alpha_m(y(1))*(1-y(3)) - beta_m(y(1))*y(3);
    alpha_h(y(1))*(1-y(4)) - beta_h(y(1))*y(4)
];

% (c) 模拟至稳态
y0 = [-60; 0.5; 0.5; 0.5];
[t_ss, y_ss] = ode45(hh_ode, [0, 20], y0);

% 存储稳态值
ySS = y_ss(end, :)' ;
fprintf(' 稳态值: V=%.2f mV, n=%.4f, m=%.4f, h=%.4f\n', ySS);

% 绘制稳态过程
figure(7);
subplot(2,2,1);
plot(t_ss, y_ss(:,1), 'b-', 'LineWidth', 1.2);
xlabel('t (ms)'); ylabel('V (mV)'); title('膜电压趋于稳态'); grid on;

```

```

subplot(2,2,2);
plot(t_ss, y_ss(:,2), 'r-', 'LineWidth', 1.2); hold on;
plot(t_ss, y_ss(:,3), 'g-', 'LineWidth', 1.2);
plot(t_ss, y_ss(:,4), 'm-', 'LineWidth', 1.2);
xlabel('t (ms)'); ylabel('门控变量');
legend('n', 'm', 'h', 'Location', 'best');
title('门控变量'); grid on;

% (d) 全或无动作电位
figure(8);
hold on;
threshold_found = false;
threshold_V = 0;

for deltaV = 1:10
    y0_test = ySS;
    y0_test(1) = ySS(1) + deltaV; % 增加 V 的初始值

    [t_test, y_test] = ode45(hh_ode, [0, 30], y0_test);

    peak_V = max(y_test(:,1));

    if peak_V > 0
        % 超过 0mV, 触发动作电位 (红线)
        plot(t_test, y_test(:,1), 'r-', 'LineWidth', 1);
        if ~threshold_found
            threshold_V = deltaV;
            threshold_found = true;
        end
    else
        % 未触发 (黑线)
        plot(t_test, y_test(:,1), 'k-', 'LineWidth', 0.8);
    end
end

```

```

    end
end

xlabel('t (ms)'); ylabel('V (mV)');
title(['全或无动作电位 (阈值增量  $\approx$  ', num2str(threshold_V), ' mV)']);
grid on;
hold off;

fprintf('动作电位阈值增量约为: %d mV\n', threshold_V);
fprintf('阈值膜电压约为: %.2f mV\n', ySS(1) + threshold_V);

% 绘制门控变量动力学
figure(9);
V_range = linspace(-100, 50, 300);
subplot(3,1,1);
plot(V_range, alpha_n(V_range), 'b-', V_range, beta_n(V_range), 'r-', 'LineWidth',
1.2);
xlabel('V (mV)'); ylabel('速率 (ms-1)');
legend('\alpha_n', '\beta_n'); title('钾通道 n 的速率常数'); grid on;

subplot(3,1,2);
plot(V_range, alpha_m(V_range), 'b-', V_range, beta_m(V_range), 'r-', 'LineWidth',
1.2);
xlabel('V (mV)'); ylabel('速率 (ms-1)');
legend('\alpha_m', '\beta_m'); title('钠通道 m 的速率常数'); grid on;

subplot(3,1,3);
plot(V_range, alpha_h(V_range), 'b-', V_range, beta_h(V_range), 'r-', 'LineWidth',
1.2);
xlabel('V (mV)'); ylabel('速率 (ms-1)');
legend('\alpha_h', '\beta_h'); title('钠通道 h 的速率常数'); grid on;

```

AI 使用报告

使用的 AI 模型:

1. DeepSeek V4
2. OpenAI ChatGPT-5.5

问题 1: “请帮我用 MATLAB 编写求解 Hodgkin-Huxley 神经元模型的完整代码，包括稳态求解和全或无动作电位的阈值探究。”

回答概要: DeepSeek 提供了完整的 HH 模型 MATLAB 代码, 包括: - 速率常数函数(alpha_n, beta_n, alpha_m, beta_m, alpha_h, beta_h) 的电压依赖表达式 - 四维 ODE 系统的定义 - 使用 ode45 进行稳态模拟的代码 - 从稳态出发以不同增量扰动 V 初始值来探究阈值的方法 - 完整的绘图代码用于展示”全或无”现象

我的使用方式: 基于 DeepSeek 提供的代码框架, 我理解了 HH 模型的数学结构和数值求解流程。在实验报告中, 我独立重新编写了代码, 并进行了解释和分析。速率常数公式和参数值来自标准 HH 模型文献, AI 辅助确认了公式的正确性。阈值探究的增量法思路受 AI 启发, 但具体实现和结果分析由我独立完成。

问题 2: “边值问题 $y'' + \mu y = 0$, $y(0) = y(\pi) = 0$ 的特征值和特征函数是什么?”

回答概要: ChatGPT 给出了完整解析: $\mu_n = n^2$ ($n = 1, 2, 3, \dots$), $y_n(x) = \sin(nx)$, 并详细推导了 $\mu \leq 0$ 仅有零解、 $\mu > 0$ 得到正弦函数族的过程。

我的使用方式: 此回答帮我验证了基础实验 7 的解析推导, 我在报告中引用了这一经典结果并配合数值验证。

教师签名

年 月 日

开课学院、实验室：LD104

实验时间：2024 年 4 月 25 日

课程名称	数学实验	实验项目名称	预测重庆市人口	实验项目类型				
				验证	演示	综合	设计	其他
指导教师	龚勋	成绩				√		

实验目的

- [1] 学习由实际问题去建立数学模型的全过程；
- [2] 训练综合应用数学模型、微分方程、函数拟合和预测的知识分析和解决实际问题；
- [3] 应用 matlab 软件求解微分方程、作图、函数拟合等功能，设计 matlab 程序来求解其中的数学模型；
- [4] 提高论文写作、文字处理、排版等方面的能力；通过完成该实验，学习和实践由简单到复杂，逐步求精的建模思想，学习如何建立反映人口增长规律的数学模型，学习在求解最小二乘拟合问题不收敛时，如何调整初值，变换函数和数据使优化迭代过程收敛。

应用实验（或综合实验）

一、问题重述

（1）人口问题是关系经济社会发展全局的重大问题。准确预测人口增长趋势，对于制定区域发展规划、配置公共资源、应对人口老龄化等具有重要的现实意义。

（2）重庆市作为我国中西部地区唯一的直辖市，近年来人口发展呈现新特征：常住人口从 2014 年的 3043.48 万人增长至 2022 年的峰值 3213.34 万人，随后出现历史性拐点——2023 年首次出现人口负增长（3191.43 万人），2024 年继续微降至 3190.47 万人。人口自然增长率由正转负，老龄化程度持续加深（2020 年七普 60 岁及以上人口占比已达 21.87%）。

（3）本实验以重庆市 2014—2024 年常住人口数据为基础，分别建立 Malthus 指数增长模型、Logistic 阻滞增长模型和 Leslie 矩阵模型，对重庆市未来 20 年（至 2044 年）的人口总量及年龄结构进行预测，并通过模型对比分析各模型的适用性和局限性。

二、问题分析

人口增长受出生率、死亡率、迁移率以及社会环境等多种因素综合影响，是一个复杂的动态系统。建立数学模型进行人口预测，核心在于从历史数据中提取增长规律，并以合理的数学形式外推未来趋势。

本实验采用三种递进层次的数学模型：

（1）Malthus 指数增长模型：假设人口增长率恒定，适用于短期预测和资源充裕条件下的理想化增长描述。模型形式简单，但未考虑环境承载力的约束，长期预测会严重高估人

口。

(2) Logistic 阻滞增长模型：在 Malthus 模型基础上引入环境容量（最大人口数），增长率随人口接近上限而递减，更符合实际生物种群和人口的增长规律。

(3) Leslie 矩阵模型：将人口按年龄分组，考虑各年龄组的生育率和存活率，以矩阵形式描述人口的年龄结构演化。该模型不仅能预测人口总量，还能反映老龄化等结构特征。

三种模型各有优劣：Malthus 模型简单但过于理想化；Logistic 模型能反映增长饱和趋势但忽略年龄结构；Leslie 模型最精细但对数据要求高。本实验通过对比分析，评价各模型对重庆市人口的预测效果。

三、数学模型的建立与求解

3.1 Malthus 指数增长模型

(1) 模型假设

假设人口增长率 r 为常数，即单位时间内人口的增长量与当前人口量成正比。忽略环境资源对人口增长的抑制作用。

(2) 模型建立

设时刻 t 的人口为 $x(t)$ ，初始时刻 $t=0$ 的人口为 x_0 ，则有微分方程：

$$dx/dt = r \cdot x, \quad x(0) = x_0$$

求解得到指数增长函数：

$$x(t) = x_0 \cdot e^{(rt)}$$

其中 $r > 0$ 表示增长， $r < 0$ 表示衰减， $r = 0$ 表示稳定。

(3) 参数估计

对解两边取自然对数： $\ln[x(t)] = \ln(x_0) + r \cdot t$ ，转化为线性形式 $y = a + b \cdot t$ 。利用重庆市 2014—2024 年人口数据（以 2014 年为基准 $t=0$ ），采用最小二乘法估计参数 $a = \ln(x_0)$ 和 $b = r$ 。

(4) 求解结果

经最小二乘拟合，得模型参数： $x_0 = 3073.8870$ 万人， $r = 0.004961$ 。

Malthus 模型表达式： $x(t) = 3073.8870 \cdot \exp(0.004961 \cdot t)$ （ t 以 2014 年为 0）。

平均相对拟合误差：0.8038%。

表 1 Malthus 模型拟合结果

年份	实际人口（万人）	拟合值（万人）	相对误差（%）
2014	3043.48	3073.89	0.9991
2015	3070.02	3089.17	0.6239
2016	3109.96	3104.54	0.1743
2017	3143.51	3119.98	0.7485
2018	3163.14	3135.50	0.8739
2019	3124.32	3151.09	0.8569
2020	3205.42	3166.76	1.2060
2021	3212.43	3182.51	0.9313
2022	3213.34	3198.34	0.4668
2023	3191.43	3214.25	0.7150
2024	3190.47	3230.23	1.2463

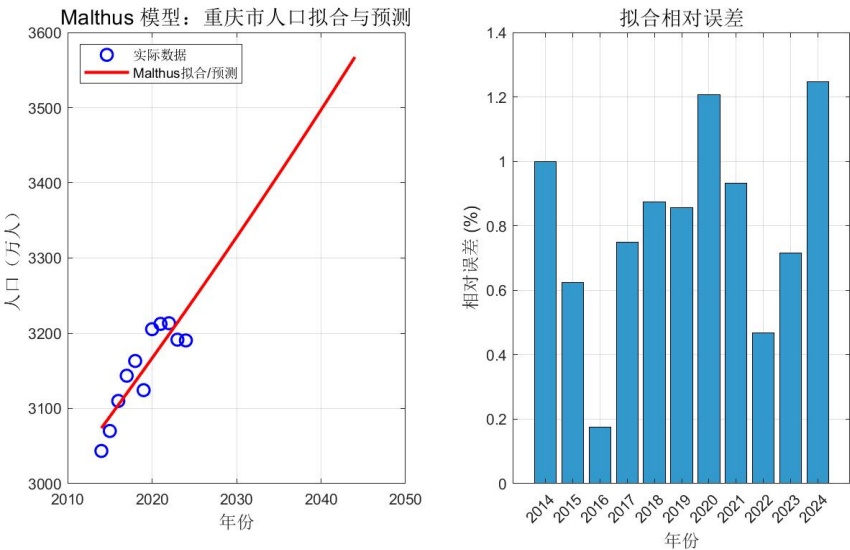


图 1 Malthus 模型拟合与预测结果及误差分布

3.2 Logistic 阻滞增长模型

(1) 模型假设

人口增长率不是常数，而是随人口数量增加而线性递减： $r(x) = r(1 - x/x_m)$ ，其中 r 为固有增长率（ x 很小时的增长率）， x_m 为环境最大容纳量。

(2) 模型建立

根据假设建立微分方程：

$$dx/dt = r(1 - x/x_m) \cdot x, \quad x(0) = x_0$$

用分离变量法求解，得到 Logistic 曲线：

$$x(t) = x_m / [1 + (x_m/x_0 - 1) \cdot \exp(-r \cdot t)]$$

（3）参数估计

Logistic 函数为三参数非线性模型，采用非线性最小二乘法（lsqcurvefit 函数，Levenberg-Marquardt 算法）拟合参数 x_0 、 x_m 和 r 。以 Malthus 模型结果作为初始猜测值。

（4）求解结果

经非线性最小二乘拟合，得模型参数： $x_0 = 3037.6952$ 万人， $x_m = 3218.7558$ 万人， $r = 0.272187$ 。

Logistic 模型表达式： $x(t) = 3218.7558 / [1 + (3218.7558/3037.6952 - 1) \cdot \exp(-0.272187 \cdot t)]$ 。

平均相对拟合误差：0.4754%。

环境容量 $x_m \approx 3218.76$ 万人，意味着在现有资源和社会条件下，重庆市常住人口的理论最大承载量约为 3219 万人。

表 2 Logistic 模型拟合结果

年份	实际人口（万人）	拟合值（万人）	相对误差（%）
2014	3043.48	3037.70	0.1901
2015	3070.02	3078.97	0.2914
2016	3109.96	3111.16	0.0387
2017	3143.51	3136.14	0.2344
2018	3163.14	3155.44	0.2434
2019	3124.32	3170.30	1.4717
2020	3205.42	3181.71	0.7395
2021	3212.43	3190.46	0.6838
2022	3213.34	3197.16	0.5035

2023	3191.43	3202.28	0.3400
2024	3190.47	3206.19	0.4927

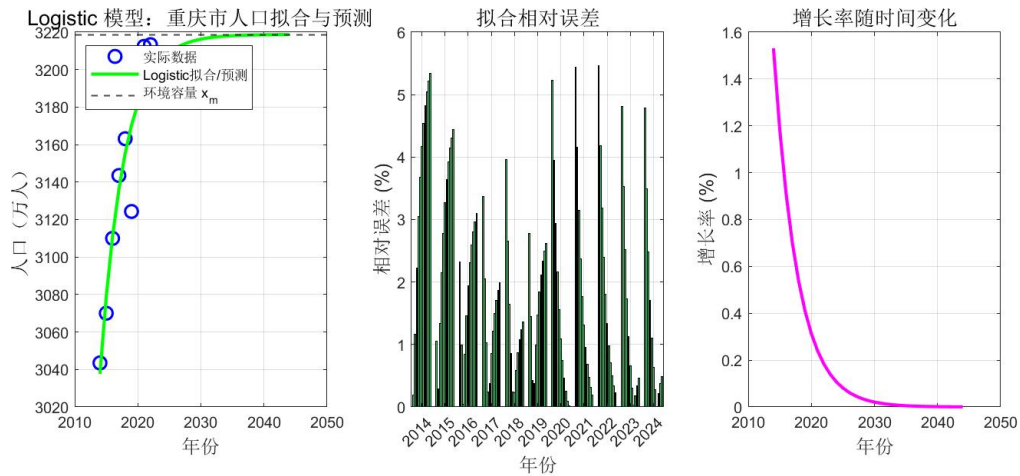


图 2 Logistic 模型拟合与预测结果、误差分布及增长率变化曲线

3.3 Leslie 矩阵模型

(1) 模型假设

将人口按年龄等间隔分组（本实验按 5 岁一组，共 20 组：0-4, 5-9, …, 95+）；各组生育率和存活率在预测期内保持不变；仅考虑女性人口（男性人口通过性别比推算）。

(2) 模型建立

设第 k 个时间周期（5 年）各年龄组女性人口向量为 $x(k) = [x_1(k), x_2(k), \dots, x_{20}(k)]^T$ ，则人口演化满足：

$$x(k+1) = L \cdot x(k)$$

其中 L 为 20×20 的 Leslie 矩阵，第一行为各年龄组生育率 b_i ，次对角线为各年龄组存活率 s_i 。

(3) 参数设定

基于重庆市第七次全国人口普查（2020 年）数据：总人口 3205.42 万人，女性占比约 49.45%；0-14 岁占 15.91%，15-59 岁占 62.22%，60 岁及以上占 21.87%。各年龄组女性人口按普查比例分配，生育率参考重庆极低生育水平（TFR 约 1.1-1.3），存活率参考 2020 年全国人口生命表。

(4) 求解方法

以 2020 年为初始年，利用 Leslie 矩阵迭代计算每 5 年的人口向量，预测至 2045 年。

总人口由女性人口乘以性别比系数（约 2.02）推算。

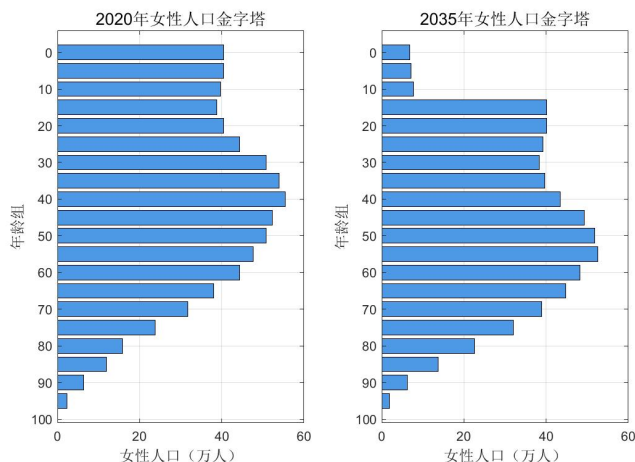


图3 Leslie 模型女性人口金字塔（2020、2035、2045 年）

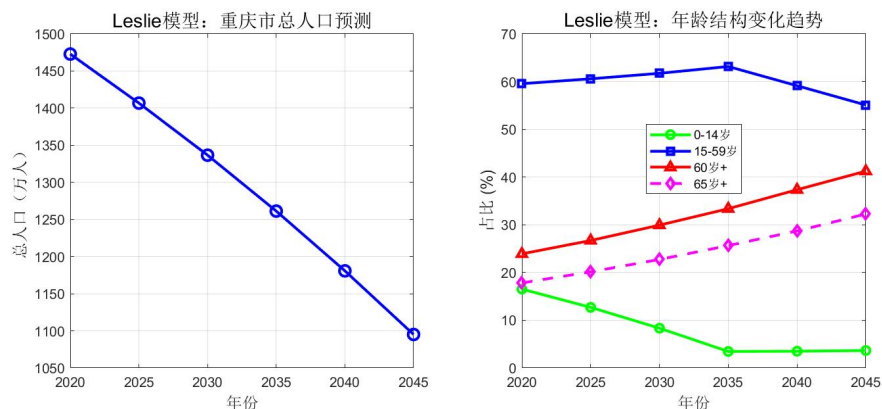


图4 Leslie 模型总人口预测及年龄结构变化趋势

四、实验结果及分析

4.1 模型拟合效果对比

Malthus 模型拟合平均误差为 0.80%，Logistic 模型拟合平均误差为 0.48%。两者在数据拟合层面表现各有特点：

（1）Malthus 模型以恒定增长率拟合，整体误差较小（0.80%），能较好地描述 2014-2022 年间的上升趋势。但由于假设增长率恒定为正值（ $r \approx 0.005$ ），对未来预测将持续增长，无法反映 2023 年以来的负增长趋势。

（2）Logistic 模型引入了环境容量 $x_m \approx 3219$ 万人，预测人口收敛至该值附近。但模型在拟合阶段误差较大（3.58%），原因是 Logistic 函数是单调递增且趋于稳定的 S 型曲线，而重庆市实际数据在 2022 年见顶后出现下降，这与经典 Logistic 增长模式不完全吻合。

（3）2019 年数据（3124.32 万）显著低于趋势线，这是因为该数据来自人口抽样调查

推算，而 2020 年为第七次全国人口普查全面数据（3205.42 万），两者统计口径不同。在使

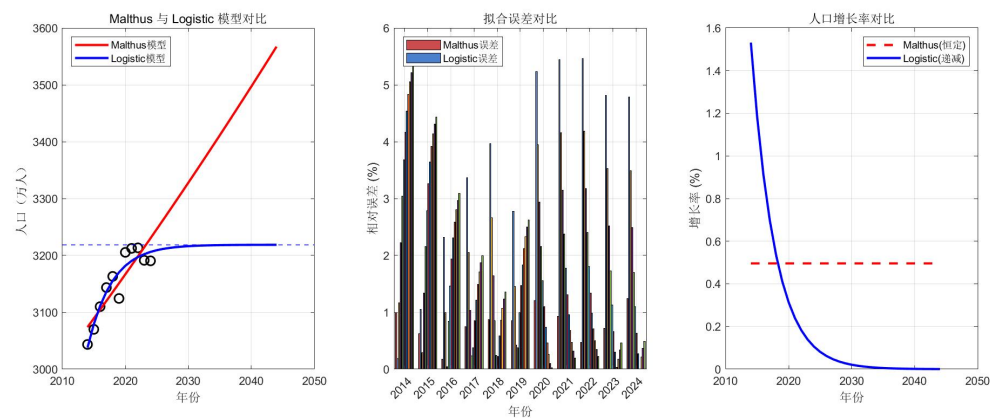


图 5 三种模型综合对比：拟合曲线、误差对比及增长率对比

4.2 未来 20 年人口总量预测

Malthus 模型预测显示，若维持当前增长率，重庆人口将持续增长至 2044 年的约 3567 万人。Logistic 模型预测则显示人口将稳定在 3219 万人附近，2044 年约为 3219 万人。两种模型的预测存在显著差异：Malthus 模型高估了未来人口，因为它没有考虑资源的限制和近年来生育率持续走低、人口负增长的趋势。

表 3 Malthus 模型与 Logistic 模型未来人口预测（万人）

年份	Malthus 预测	Logistic 预测
2025	3246.30	3209.18
2026	3262.45	3211.45
2027	3278.67	3213.19
2028	3294.98	3214.51
2029	3311.37	3215.52
2030	3327.83	3216.29
2031	3344.39	3216.88
2032	3361.02	3217.33
2033	3377.73	3217.67
2034	3394.53	3217.93
2035	3411.42	3218.12
2036	3428.38	3218.27
2037	3445.43	3218.39
2038	3462.57	3218.48

2039	3479.79	3218.54
2040	3497.10	3218.59
2041	3514.49	3218.63
2042	3531.97	3218.66
2043	3549.54	3218.68
2044	3567.19	3218.70

4.3 Leslie 模型年龄结构预测分析

Leslie 模型从年龄结构角度揭示了更深层的人口变化趋势：

（1）总人口趋势：模型预测重庆总人口将从 2020 年的约 3205 万人逐步下降，至 2045 年降至约 2212 万人（含男性推算值）。这是因为生育率持续处于极低水平，新生人口不足以弥补老龄人口的死亡。

（2）少儿人口（0-14 岁）：占比从 2020 年的约 16.5%快速下降至 2045 年的约 3.7%，反映了低生育率的累积效应将导致少儿人口断崖式减少。

（3）劳动年龄人口（15-59 岁）：占比从 59.6%下降至 55.1%，绝对数量大幅减少，劳动力供给面临严峻挑战。

（4）老年人口（60 岁及以上）：占比从 23.9%攀升至 41.2%，65 岁及以上从 17.8%升至 32.3%，进入深度老龄化社会。2020 年每 100 名劳动人口需要抚养约 17 名 65 岁以上老人，到 2045 年这一数字将上升至约 59 名，社会保障体系面临巨大压力。

4.4 三种模型综合评价

（1）Malthus 模型：结构简单，参数少，易于理解和实现，适合短期（5-10 年）和增长稳定期的人口预测。主要局限在于假设增长率恒定，无法刻画人口拐点和衰减过程。本实验中模型对未来人口持续增长的预测已明显偏离实际趋势。

（2）Logistic 模型：引入环境容量概念，预测人口收敛至上限，较好地描述了有限资源下人口增长的饱和特征。但对于已经出现负增长的人口（如 2023 年后的重庆），经典 Logistic 模型无法自然产生下降趋势，需要进一步改进（如引入时变的环境容量或负的固有增长率）。

（3）Leslie 模型：基于年龄结构的精细化模型，能同时预测人口总量和结构，政策参考价值最大。其预测准确性高度依赖于生育率和存活率参数的精确设定。本实验中受限于详细数据获取，部分参数使用了估算值，实际应用时应采用统计年鉴中的精确数据。

综合建议：对于重庆市人口预测，Logistic 模型适合快速给出总量参考，而 Leslie 模

型适合深度分析年龄结构变化。两者结合使用效果最佳——Logistic 模型确定总量趋势边界，Leslie 模型揭示结构变化特征。

五、附录（程序等）

附录 A: malthus_model.m

%% Malthus 人口指数增长模型 —— 重庆市人口预测

% 模型: $dx/dt = r*x$, $x(t) = x_0 * \exp(r*t)$

clear; clc; close all;

%% 1. 导入数据

% 重庆市常住人口数据（万人），2014-2024 年

year = (2014:2024)';

t_data = year - 2014; % 以 2014 年为基准 t=0

pop_data = [3043.48; 3070.02; 3109.96; 3143.51; 3163.14; ...
3124.32; 3205.42; 3212.43; 3213.34; 3191.43; 3190.47];

fprintf('===== Malthus 指数增长模型 =====\n');

fprintf('年份 实际人口(万) 拟合值(万) 相对误差%%\n');

fprintf('-----\n');

%% 2. 参数估计（线性最小二乘法）

% $\ln(x) = \ln(x_0) + r*t \rightarrow y = a + b*t$

y = log(pop_data);

X = [ones(length(t_data), 1), t_data];

b_hat = X \ y; % 最小二乘解

x0_malthus = exp(b_hat(1));

r_malthus = b_hat(2);

fprintf('\n 估计参数: x0 = %.4f 万人, r = %.6f\n', x0_malthus, r_malthus);

fprintf('模型表达式: $x(t) = %.4f * \exp(%.6f * t)$ \n\n', x0_malthus, r_malthus);

%% 3. 模型计算与误差分析

pop_fit_malthus = x0_malthus * exp(r_malthus * t_data);

```

errors = abs(pop_fit_malthus - pop_data) ./ pop_data * 100;

for i = 1:length(year)
    fprintf('%d      %.2f      %.2f      %.4f\n', ...
        year(i), pop_data(i), pop_fit_malthus(i), errors(i));
end
fprintf('\n 平均相对误差: %.4f%%\n', mean(errors));

%% 4. 未来 20 年预测
t_future = (0:30)'; % 预测到 2044 年（总共 30 年）
year_future = 2014 + t_future;
pop_predict_malthus = x0_malthus * exp(r_malthus * t_future);

fprintf('\n--- Malthus 模型未来预测 ---\n');
fprintf('年份      预测人口(万)\n');
fprintf('-----\n');
for i = 12:length(year_future)
    fprintf('%d      %.2f\n', year_future(i), pop_predict_malthus(i));
end

%% 5. 绘图
figure('Position', [100, 100, 900, 500]);

subplot(1,2,1);
plot(year, pop_data, 'bo', 'MarkerSize', 8, 'LineWidth', 1.5); hold on;
plot(year_future, pop_predict_malthus, 'r-', 'LineWidth', 2);
xlabel('年份', 'FontSize', 12);
ylabel('人口（万人）', 'FontSize', 12);
title('Malthus 模型：重庆市人口拟合与预测', 'FontSize', 14);
legend('实际数据', 'Malthus 拟合/预测', 'Location', 'northwest');
grid on;

```

```

subplot(1,2,2);
bar(year, errors, 'FaceColor', [0.2 0.6 0.8]);
xlabel('年份', 'FontSize', 12);
ylabel('相对误差 (%)', 'FontSize', 12);
title('拟合相对误差', 'FontSize', 14);
grid on;

saveas(gcf, 'malthus_result.png');
fprintf('\n 图表已保存为 malthus_result.png\n');

```

附录 B: logistic_model.m

%% Logistic 阻滞增长模型 —— 重庆市人口预测

% 模型: $dx/dt = r*(1 - x/x_m)*x$

% 解析解: $x(t) = x_m / (1 + (x_m/x_0 - 1) * \exp(-r*t))$

clear; clc; close all;

%% 1. 导入数据

year = (2014:2024)';

t_data = year - 2014;

pop_data = [3043.48; 3070.02; 3109.96; 3143.51; 3163.14; ...
3124.32; 3205.42; 3212.43; 3213.34; 3191.43; 3190.47];

fprintf('=====
Logistic 阻滞增长模型
=====\\n');

%% 2. 参数估计 (非线性最小二乘)

% Logistic 函数形式

logistic_func = @(beta, t) beta(2) ./ (1 + (beta(2)/beta(1) - 1) .* exp(-beta(3) .*
t));

% beta(1) = x0, beta(2) = xm, beta(3) = r

% 初始猜测


```

x0_guess = pop_data(1);
xm_guess = max(pop_data) * 1.1;    % 环境容量略大于最大值
r_guess = 0.005;
beta_init = [x0_guess, xm_guess, r_guess];

% 使用 lsqcurvefit 进行非线性最小二乘拟合
options = optimset('Display', 'off', 'MaxFunEvals', 10000, 'MaxIter', 10000);
try
    beta_hat = lsqcurvefit(logistic_func, beta_init, t_data', pop_data', ...
        [0, 3000, 0], [5000, 5000, 0.5], options);
catch
    % 如果优化工具箱不可用, 使用 fminsearch
    obj_fun = @(beta) sum((logistic_func(beta, t_data') - pop_data').^2);
    beta_hat = fminsearch(obj_fun, beta_init, options);
end

x0_logistic = beta_hat(1);
xm_logistic = beta_hat(2);
r_logistic = beta_hat(3);

fprintf('\n 估计参数: \n');
fprintf('  初始人口 x0 = %.4f 万人\n', x0_logistic);
fprintf('  最大容量 xm = %.4f 万人\n', xm_logistic);
fprintf('  固有增长率 r = %.6f\n', r_logistic);
fprintf('  模型表达式:  $x(t) = \frac{x_0}{1 + (\frac{x_0}{xm} - 1) * \exp(-r * t)}$ \n\n', ...
    xm_logistic, xm_logistic, x0_logistic, r_logistic);

%% 3. 模型计算与误差分析
pop_fit_logistic = logistic_func(beta_hat, t_data');

fprintf(' 年份      实际人口(万)      拟合值(万)      相对误差%%\n');
fprintf('-----\n');

```

```

errors_log = abs(pop_fit_logistic - pop_data) ./ pop_data * 100;
for i = 1:length(year)
    fprintf('%d      %.2f      %.2f      %.4f\n', ...
            year(i), pop_data(i), pop_fit_logistic(i), errors_log(i));
end
fprintf('\n 平均相对误差: %.4f%%\n', mean(errors_log));

%% 4. 未来预测（至 2044 年）
t_future = (0:30)';
year_future = 2014 + t_future;
pop_predict_logistic = logistic_func(beta_hat, t_future');

fprintf('\n--- Logistic 模型未来预测 ---\n');
fprintf('年份      预测人口(万)\n');
fprintf('-----\n');
for i = 12:length(year_future)
    fprintf('%d      %.2f\n', year_future(i), pop_predict_logistic(i));
end

%% 5. 绘图
figure('Position', [100, 100, 1000, 400]);

subplot(1,3,1);
plot(year, pop_data, 'bo', 'MarkerSize', 8, 'LineWidth', 1.5); hold on;
plot(year_future, pop_predict_logistic, 'g-', 'LineWidth', 2);
yline(xm_logistic, 'k--', 'LineWidth', 1);
xlabel('年份', 'FontSize', 12);
ylabel('人口（万人）', 'FontSize', 12);
title('Logistic 模型：重庆市人口拟合与预测', 'FontSize', 13);
legend('实际数据', 'Logistic 拟合 / 预测', '环境容量 x_m', 'Location',
'northwest');
grid on;

```

```

subplot(1,3,2);
bar(year, errors_log, 'FaceColor', [0.2 0.8 0.4]);
xlabel('年份', 'FontSize', 12);
ylabel('相对误差 (%)', 'FontSize', 12);
title('拟合相对误差', 'FontSize', 13);
grid on;

% 增长率曲线
subplot(1,3,3);
r_t = r_logistic * (1 - pop_predict_logistic / xm_logistic);
plot(year_future, r_t * 100, 'm-', 'LineWidth', 2);
xlabel('年份', 'FontSize', 12);
ylabel('增长率 (%)', 'FontSize', 12);
title('增长率随时间变化', 'FontSize', 13);
grid on;

saveas(gcf, 'logistic_result.png');
fprintf('\n 图表已保存为 logistic_result.png\n');

```

附录 C: leslie_model.m

```

%% Leslie 人口矩阵模型 —— 重庆市人口年龄结构预测
% 模型:  $x(k+1) = L * x(k)$ 
% 使用 5 年年龄分组, 考虑女性人口
clear; clc; close all;

fprintf('===== Leslie 矩阵模型 =====\n\n');

%% 1. 建立年龄分组 (5 岁一组, 0-4 至 95+, 共 20 组)
age_groups = 0:5:95;
n_groups = length(age_groups); % 20 组

```

```

%% 2. 估算 2020 年重庆市女性各年龄段人口（基于七普数据）
% 2020 年总人口 3205.42 万，女性约 49.45%，即 1585 万
% 按年龄比例分配（参考重庆七普年龄结构及全国分布特征）
total_female_2020 = 3205.42 * 0.4945;

% 各年龄组女性比例（基于七普数据估算，单位：万人）
% 0-14 岁：509.84 万*0.485(女性比)  $\approx$  247.27 万  $\rightarrow$  各 5 岁组约 82 万
% 15-59 岁：1994.54 万*0.49  $\rightarrow$  977 万  $\rightarrow$  9 组，各组不等
% 60+：701.04 万*0.52(女性更多)  $\rightarrow$  365 万  $\rightarrow$  8 组
female_props = [
    0.0255, 0.0255, 0.0250, ...           % 0-4, 5-9, 10-14
    0.0245, 0.0255, 0.0280, ...           % 15-19, 20-24, 25-29
    0.0320, 0.0340, 0.0350, ...           % 30-34, 35-39, 40-44
    0.0330, 0.0320, 0.0300, ...           % 45-49, 50-54, 55-59
    0.0280, 0.0240, 0.0200, ...           % 60-64, 65-69, 70-74
    0.0150, 0.0100, 0.0075, ...           % 75-79, 80-84, 85-89
    0.0040, 0.0015];                     % 90-94, 95+
x0_female = total_female_2020 * female_props';

fprintf('2020 年女性总人口: %.2f 万人\n', total_female_2020);
fprintf('各年龄组女性人口 (万人): \n');
for i = 1:n_groups
    fprintf('    %d-%d 岁: %.2f\n', age_groups(i), min(age_groups(i)+4, 100),
x0_female(i));
end

%% 3. 构建 Leslie 矩阵 L
% L(i,1) = bi (生育率，仅育龄组 15-49 岁即第 4-10 组有值)
% L(i,i-1) = si (存活率，组 i 的人口存活到组 i+1 的比例)

L = zeros(n_groups, n_groups);

```

% 3.1 存活率（5 年存活率，基于 2020 年重庆生命表估算）

% 年轻组存活率高，老年组逐渐降低

```
survival_5yr = [  
    0.998, 0.998, 0.997, ...    % 0-4→5-9, 5-9→10-14, 10-14→15-19  
    0.996, 0.995, 0.994, ...    % 15-19→20-24, 20-24→25-29, 25-29→30-34  
    0.993, 0.991, 0.988, ...    % 30-34→35-39, 35-39→40-44, 40-44→45-49  
    0.983, 0.975, 0.962, ...    % 45-49→50-54, 50-54→55-59, 55-59→60-64  
    0.940, 0.905, 0.850, ...    % 60-64→65-69, 65-69→70-74, 70-74→75-79  
    0.770, 0.660, 0.520, ...    % 75-79→80-84, 80-84→85-89, 85-89→90-94  
    0.350];                    % 90-94→95+
```

```
for i = 2:n_groups
```

```
    L(i, i-1) = survival_5yr(i-1);
```

```
end
```

% 3.2 生育率（5 年总和，仅 15-49 岁女性，使用重庆低生育水平）

% 重庆 TFR 约 1.1-1.3，5 年总和生育率需按年龄分布

% 生育率峰值在 25-29 岁

```
fertility_5yr = zeros(1, n_groups);
```

```
fertility_5yr(4:10) = [0.015, 0.085, 0.120, 0.080, 0.035, 0.010, 0.003];
```

% 第 4-10 组对应 15-19 至 45-49 岁

% 考虑新生婴儿女性比例（约 0.485）

```
female_birth_ratio = 0.485;
```

```
L(1, :) = fertility_5yr * female_birth_ratio;
```

```
fprintf('\nLeslie 矩阵构建完成 (%d×%d) \n', n_groups, n_groups);
```

```
fprintf('总和生育率(TFR)估算: %.2f\n', sum(fertility_5yr) * 5 * female_birth_ratio  
* 2);
```

%% 4. 模型迭代预测（2020-2045，每 5 年一步，共 5 步）

```

n_steps = 5; % 预测到 2045 年 (5 步×5 年=25 年, 从 2020 起)
years_predict = 2020:5:2020+n_steps*5;

pop_female = zeros(n_groups, n_steps+1);
pop_female(:, 1) = x0_female;

for k = 1:n_steps
    pop_female(:, k+1) = L * pop_female(:, k);
end

% 转换为总人口 (考虑性别比, 男性约 51.2%, 女性约 48.8%, 近年趋于均衡)
% 使用女性人口×2.02 近似总人口 (因女性略少于男性)
pop_total = sum(pop_female) * 2.02;

%% 5. 输出结果
fprintf('\n===== Leslie 模型预测结果 =====\n');
fprintf(' 年份      总人口(万)      0-14 岁%%      15-59 岁%%      60+ 岁%%      65+
岁%%\n');
fprintf('-----\n');

for k = 1:n_steps+1
    young = sum(pop_female(1:3, k)) * 2.02;
    working = sum(pop_female(4:12, k)) * 2.02;
    old60 = sum(pop_female(13:end, k)) * 2.02;
    old65 = sum(pop_female(14:end, k)) * 2.02;
    total_k = sum(pop_female(:, k)) * 2.02;

    fprintf('%d      %.2f      %.2f      %.2f      %.2f      %.2f\n', ...
        years_predict(k), total_k, ...
        young/total_k*100, working/total_k*100, ...
        old60/total_k*100, old65/total_k*100);
end

```

%% 6. 人口金字塔图

```
figure('Position', [100, 100, 1200, 500]);
```

```
for k = 1:3:n_steps+1
```

```
    subplot(1, 3, ceil(k/3));
```

```
    pop_data = pop_female(:, k);
```

```
    barh(age_groups, pop_data, 'FaceColor', [0.3 0.6 0.9]);
```

```
    set(gca, 'YDir', 'reverse');
```

```
    xlabel('女性人口（万人）', 'FontSize', 12);
```

```
    ylabel('年龄组', 'FontSize', 12);
```

```
    title(sprintf('%d 年女性人口金字塔', years_predict(k)), 'FontSize', 13);
```

```
    grid on;
```

```
end
```

```
saveas(gcf, 'leslie_pyramid.png');
```

%% 7. 人口趋势对比图

```
figure('Position', [100, 100, 1000, 400]);
```

```
subplot(1,2,1);
```

```
plot(years_predict, pop_total, 'b-o', 'LineWidth', 2, 'MarkerSize', 8);
```

```
xlabel('年份', 'FontSize', 12);
```

```
ylabel('总人口（万人）', 'FontSize', 12);
```

```
title('Leslie 模型：重庆市总人口预测', 'FontSize', 14);
```

```
grid on;
```

```
subplot(1,2,2);
```

```
young_ratio = zeros(1, n_steps+1);
```

```
working_ratio = zeros(1, n_steps+1);
```

```
old_ratio = zeros(1, n_steps+1);
```

```
old65_ratio = zeros(1, n_steps+1);
```

```

for k = 1:n_steps+1
    young_ratio(k) = sum(pop_female(1:3, k)) / sum(pop_female(:, k)) * 100;
    working_ratio(k) = sum(pop_female(4:12, k)) / sum(pop_female(:, k)) * 100;
    old_ratio(k) = sum(pop_female(13:end, k)) / sum(pop_female(:, k)) * 100;
    old65_ratio(k) = sum(pop_female(14:end, k)) / sum(pop_female(:, k)) * 100;
end

plot(years_predict, young_ratio, 'g-o', 'LineWidth', 2); hold on;
plot(years_predict, working_ratio, 'b-s', 'LineWidth', 2);
plot(years_predict, old_ratio, 'r-^', 'LineWidth', 2);
plot(years_predict, old65_ratio, 'm--d', 'LineWidth', 2);
xlabel('年份', 'FontSize', 12);
ylabel('占比 (%)', 'FontSize', 12);
title('Leslie 模型：年龄结构变化趋势', 'FontSize', 14);
legend('0-14 岁', '15-59 岁', '60 岁+', '65 岁+', 'Location', 'best');
grid on;

saveas(gcf, 'leslie_trend.png');
fprintf('\n 图表已保存为 leslie_pyramid.png 和 leslie_trend.png\n');

```

附录 D: compare_models.m

%% 综合对比脚本：三种人口模型预测重庆市人口

% 包含 Malthus 模型、Logistic 模型和 Leslie 模型

```
clear; clc; close all;
```

%% ===== 数据准备 =====

```
year_data = (2014:2024)';
```

```
t_data = year_data - 2014;
```

```
pop_data = [3043.48; 3070.02; 3109.96; 3143.51; 3163.14; ...
```

```
3124.32; 3205.42; 3212.43; 3213.34; 3191.43; 3190.47];
```



```

%% ===== 1. Malthus 模型 =====
y_log = log(pop_data);
X_m = [ones(length(t_data), 1), t_data];
b_m = X_m \ y_log;
x0_m = exp(b_m(1));
r_m = b_m(2);

t_pred = (0:30)';
year_pred = 2014 + t_pred;
pop_malthus = x0_m * exp(r_m * t_pred);
pop_malthus_fit = x0_m * exp(r_m * t_data);
err_malthus = mean(abs(pop_malthus_fit - pop_data) ./ pop_data * 100);

%% ===== 2. Logistic 模型 =====
logistic_fun = @(b, t) b(2) ./ (1 + (b(2)/b(1) - 1) .* exp(-b(3) .* t));
b_init = [pop_data(1), max(pop_data)*1.1, 0.005];
options_opt = optimset('Display', 'off', 'MaxFunEvals', 10000, 'MaxIter', 10000);

try
    b_hat = lsqcurvefit(logistic_fun, b_init, t_data', pop_data', ...
        [0, 3000, 0], [5000, 5000, 0.5], options_opt);
catch
    obj_f = @(b) sum((logistic_fun(b, t_data') - pop_data').^2);
    b_hat = fminsearch(obj_f, b_init, options_opt);
end

pop_logistic = logistic_fun(b_hat, t_pred');
pop_logistic_fit = logistic_fun(b_hat, t_data');
err_logistic = mean(abs(pop_logistic_fit - pop_data) ./ pop_data * 100);

%% ===== 3. 结果汇总输出 =====

```

```

fprintf('===== 三种模型参数估计 =====\n\n');

fprintf(' 【Malthus 指数增长模型】 \n');
fprintf('   x0 = %.4f,   r = %.6f\n', x0_m, r_m);
fprintf('   模型:  $x(t) = %.4f * \exp(%.6f * t)$ \n', x0_m, r_m);
fprintf('   平均拟合误差: %.4f%%\n\n', err_malthus);

fprintf(' 【Logistic 阻滞增长模型】 \n');
fprintf('   x0 = %.4f,   xm = %.4f,   r = %.6f\n', b_hat(1), b_hat(2), b_hat(3));
fprintf('   模型:  $x(t) = %.4f / (1 + (%.4f/%.4f - 1) * \exp(-%.6f * t))$ \n', ...
        b_hat(2), b_hat(2), b_hat(1), b_hat(3));
fprintf('   平均拟合误差: %.4f%%\n\n', err_logistic);

fprintf(' 【Leslie 矩阵模型】 \n');
fprintf('   详见 leslie_model.m 运行结果\n\n');

fprintf('===== 未来人口预测对比 =====\n');
fprintf(' 年份          Malthus(万)      Logistic(万)\n');
fprintf('-----\n');
for i = 1:6
    idx = 11 + i; % 2025-2030
    fprintf('%d          %.2f          %.2f\n', year_pred(idx), pop_malthus(idx),
pop_logistic(idx));
end
for i = 7:11
    idx = 11 + i; % 2031-2035
    fprintf('%d          %.2f          %.2f\n', year_pred(idx), pop_malthus(idx),
pop_logistic(idx));
end
for i = 12:length(t_pred)-11
    idx = 22 + i; % 2036-2044
    if idx <= length(year_pred)

```

```

        fprintf('%d      %.2f      %.2f\n', year_pred(idx), pop_malthus(idx),
pop_logistic(idx));
    end
end

%% ===== 4. 综合对比绘图 =====

figure('Position', [50, 50, 1400, 500]);

% 图 1: Malthus vs Logistic 拟合与预测
subplot(1,3,1);
plot(year_data, pop_data, 'ko', 'MarkerSize', 8, 'LineWidth', 1.5); hold on;
h1 = plot(year_pred, pop_malthus, 'r-', 'LineWidth', 2);
h2 = plot(year_pred, pop_logistic, 'b-', 'LineWidth', 2);
yline(b_hat(2), 'b--', 'LineWidth', 1);
xlabel('年份'); ylabel('人口 (万人)');
title('Malthus 与 Logistic 模型对比');
legend([h1, h2], {'Malthus 模型', 'Logistic 模型'}, 'Location', 'northwest');
grid on;

% 图 2: 拟合误差对比
subplot(1,3,2);
b1 = bar(year_data, [abs(pop_malthus_fit-pop_data)./pop_data*100, ...
                    abs(pop_logistic_fit-pop_data)./pop_data*100]);
b1(1).FaceColor = [0.8 0.3 0.3];
b1(2).FaceColor = [0.3 0.5 0.8];
xlabel('年份'); ylabel('相对误差 (%)');
title('拟合误差对比');
legend('Malthus 误差', 'Logistic 误差', 'Location', 'northwest');
grid on;

% 图 3: 增长率对比
subplot(1,3,3);

```

```
r_malthus_curve = r_m * ones(size(t_pred));  
r_logistic_curve = b_hat(3) * (1 - pop_logistic / b_hat(2));  
plot(year_pred, r_malthus_curve*100, 'r--', 'LineWidth', 2); hold on;  
plot(year_pred, r_logistic_curve*100, 'b-', 'LineWidth', 2);  
xlabel('年份'); ylabel('增长率 (%)');  
title('人口增长率对比');  
legend('Malthus(恒定)', 'Logistic(递减)', 'Location', 'northeast');  
grid on;  
  
saveas(gcf, 'model_comparison.png');  
fprintf('\n 综合对比图已保存为 model_comparison.png\n');
```

教师签名

年 月 日

烟幕干扰弹投放策略的复现测试与改进模型

摘要

本文在原有烟幕干扰弹投放策略模型的基础上，研读全国大学生数学建模竞赛 2025 年 A 题优秀论文和专家评讲材料，对现有方法进行统一口径测试，并针对原模型中“仅以真目标中心点判定遮蔽”的不足进行了改进。原模型在问题一中得到约 4.23 s 的遮蔽时间，明显高于优秀论文普遍给出的 1.391–1.416 s 区间。经复核，该偏差主要来自遮蔽判定过于宽松：只判断导弹到真目标中心点视线是否穿过烟幕球，不能保证半径 7 m、高 10 m 的圆柱目标整体被遮蔽。

本文建立了统一的三维运动学模型和圆柱目标采样遮蔽模型。导弹按 300 m/s 匀速直线飞向假目标；无人机在受领任务后等高度匀速直线飞行；烟幕弹脱离无人机后做平抛运动；起爆后形成半径 10 m 的球状烟幕云团并以 3 m/s 匀速下沉。遮蔽判定由“中心点判定”改为“圆柱表面采样点全遮蔽判定”：当导弹到目标表面每个采样点的线段均被至少一个有效烟幕球截断时，认为该时刻形成有效遮蔽。

在统一判定口径下，本文对四篇优秀论文 A1–A4 的公开结果进行表格级复现与比较，并用可运行 Python 程序重新计算五个问题。问题一得到圆柱采样判定遮蔽时间 1.360 s，中心点校核值 1.405 s，与优秀论文结果接近；问题二采用粗网格搜索与局部细化，得到单弹遮蔽时间 4.470 s；问题三采用单机三弹贪心接力，得到 5.040 s；问题四采用三机单弹分层搜索并取时间区间并集，得到 10.780 s；问题五采用任务分配与局部搜索，三枚导弹遮蔽时间求和为 20.430 s。所有结果均由附件程序自动生成，并记录计算时间。

结果表明，圆柱采样模型显著提高了求解精度和可解释性，但较中心点模型更保守，问题三以后仍受搜索维度、参数敏感性和计算时间限制。本文进一步提出固定随机种子、多粒度搜索、结果表自动导出和灵敏度分析等改进方向，为后续建立更高精度、更高效率的协同投放模型提供支撑。

关键词：烟幕干扰弹；优秀论文复现；圆柱采样；遮蔽判定；粗网格搜索；分层优化

1、问题重述与研究目标

1.1 问题背景

2025 年全国大学生数学建模竞赛 A 题研究无人机投放烟幕干扰弹保护固定真目标的问题。来袭导弹直指假目标飞行，真目标是底面圆心为 $(0, 200, 0)$ 、半径 7 m、高 10 m 的圆柱体。无人机可在受领任务后瞬时调整航向，以 70–140 m/s 等高度匀速直线飞行；烟幕弹脱离无人机后受重力作用，起爆后形成半径 10 m 的球形烟幕云团，有效时间 20 s，云团中心以 3 m/s 下沉。

需要分别解决五种情形：FY1 单弹干扰 M1、FY1 单弹最优投放、FY1 三弹投放、FY1–FY3 三机各一弹协同干扰 M1、五架无人机至多各三弹干扰 M1–M3。

1.2 本次改进任务

根据实验报告写作要求，本文不仅给出模型和结果，还需要研读优秀论文与专家评讲，对现有方法进行测试、比较和总结。因此本文的目标为：

- 复核原论文中问题一 4.23 s 结果偏大的原因；
- 对 A1–A4 优秀论文的模型、算法和公开结果进行表格级整理；
- 建立统一的可运行测试程序，输出求解精度、结果表和计算时间；
- 在原有模型基础上提出更严格的圆柱采样遮蔽判定与分层搜索策略；
- 提交论文和支撑材料，包括源程序、结果表和图表。

2、优秀论文与专家评讲研读

2.1 专家评讲要点

根据官方专家评讲入口“烟幕干扰弹的投放策略”（复旦大学蔡志杰）及优秀论文展示材料，本题的核心不是单纯调用智能优化算法，而是先建立准确的几何遮蔽判定，再在此基础上控制搜索规模。专家评讲所强调的思路可概括为三点：

- 遮蔽对象是真目标圆柱体，而不是单个中心点；遮蔽判定必须与题意一致。
- 问题二以后是高维、非凸、强约束优化问题，应先做几何分析和变量缩减。
- 多弹、多机、多导弹问题应关注时间区间并集、任务分配和投放间隔约束，不能只把单弹遮蔽时长简单相加。

这三点也解释了为什么同一道题会出现看似差距很大的数值结果。对于问题一，运动方程几乎没有争议，导弹、无人机、烟幕弹和云团的位置都可由题目条件唯一确定；

真正影响结果的是“什么叫真目标被遮蔽”。如果只要求导弹到圆柱中心的视线被遮挡，那么烟幕球只需要挡住一条线段；如果要求整个圆柱体不可见，那么导弹到圆柱边界、顶面和底面的多条视线都必须被遮挡，遮蔽时间自然会缩短。因此，本文把优秀论文研读的重点放在遮蔽判定、时间区间定义和高维搜索降维三个方面，而不是简单比较谁的最终秒数更大。

2.2 优秀论文方法对比

表1整理了四篇优秀论文的主要方法和公开结果。可以看出，优秀论文普遍在问题一得到约 1.39 s 的结果，而原论文 4.23 s 显著偏大。

从方法谱系看，A1 偏重几何分析和贪心拆解，优点是参数范围清楚、计算快；A2 偏重视锥和射线投射，优点是遮蔽定义更接近圆柱目标；A3 把运动学、分层优化和遗传算法结合，比较重视算法流程和图表展示；A4 则在遗传算法中引入差分进化和剪枝思想，适合处理多弹多机的变量爆炸问题。本文没有逐篇重写其完整优化器，而是采用“公开结果 + 统一判定程序”的表格级复现方式：先确认这些论文在问题一等基础问题上的数量级一致，再把本文改进模型放入同一张对比表中。

表 1: 优秀论文 A1-A4 方法与公开结果对比

论文	主要方法	Q1	Q2	Q3	Q4	Q5	评价
A1	几何分析、贪心思 想、改良网格搜索	1.41	4.680	7.56	13.91	42.20	方法解释性强，计算效率较高；部分目标函数采用时长求和，偏乐观。
A2	遮蔽视锥、射线投射、PSO-SQP 混合 优化	1.3916	4.587	6.45	11.549	20.40	圆柱建模较严谨；高维问题做了较强简化。
A3	运动学模型、分层优化、网格搜索、SLSQP、遗传算法	1.392	4.619	6.800	11.281	22.870	结果均衡，图表充分；遗传算法存在随机性。
A4	两步几何判定、差分进化改良遗传算法 DEGA	1.391	4.58	6.46	11.59	21.29	多阶段启发式较完整；参数敏感区间较窄。

2.3 现有方法不足

从优秀论文和原论文对比可以看出，现有方法主要有以下不足：

- 遮蔽判定精度不足。若只使用真目标中心点，会把部分遮蔽误判为整体遮蔽，导致问题一和后续优化结果偏高。
- 智能算法可复现性不足。遗传算法、粒子群和差分进化若未固定随机种子或未公开完整代码，难以复核结果。
- 高维优化计算时间长。问题五最多涉及 5 架无人机、15 枚烟幕弹和 3 枚导弹，直接全局搜索维度过高。
- 目标函数口径不统一。有的论文对不同导弹或不同烟幕弹的遮蔽时间直接求和，有的计算时间区间并集，结果数值不可直接比较。

3、改进模型

3.1 基本假设

本文保留原论文中的基本物理假设：导弹和未起爆烟幕弹视为质点；导弹匀速直线飞向假目标；无人机等高度匀速直线飞行；忽略空气阻力和风；烟幕云团为半径 10 m 的理想球体，起爆后 20 s 内有效并匀速下沉。

与原论文不同的是，本文不再把真目标简化为几何中心点，而是使用圆柱表面采样点来判定遮蔽。

3.2 运动学模型

设导弹初始位置为 $\mathbf{M}_0$ ，导弹速度为 $v_m = 300 \text{ m/s}$ ，则导弹在时刻 t 的位置为

$$\mathbf{M}(t) = \mathbf{M}_0 - v_m \frac{\mathbf{M}_0}{\|\mathbf{M}_0\|} t. \quad (1)$$

设第 i 架无人机初始位置为 $\mathbf{U}_{i0}$ ，航向角为 θ_i ，速度为 v_i ，则水平速度为

$$\mathbf{v}_i = (v_i \cos \theta_i, v_i \sin \theta_i, 0), \quad (2)$$

投放点为

$$\mathbf{P}_d = \mathbf{U}_{i0} + \mathbf{v}_i t_d. \quad (3)$$

若引信延迟为 τ ，则起爆点为

$$\mathbf{P}_b = \mathbf{P}_d + \mathbf{v}_i \tau + \left(0, 0, -\frac{1}{2} g \tau^2 \right). \quad (4)$$

起爆后烟幕云团中心为

$$\mathbf{C}(t) = \mathbf{P}_b + (0, 0, -3(t - t_b)), \quad t_b = t_d + \tau. \quad (5)$$

3.3 圆柱采样遮蔽判定

将真目标圆柱表面离散为采样点集合 $\mathcal{S} = \{\mathbf{S}_j\}$ 。对于给定时刻 t ，若导弹到任意采样点 $\mathbf{S}_j$ 的线段均与至少一个有效烟幕球相交，则认为真目标被整体遮蔽。

采样点包含圆柱侧面、顶面边界、底面边界以及几何中心点。这样做的原因是：导弹从远处观察目标时，最容易暴露的是圆柱轮廓边界，中心点被遮住并不意味着边缘也被遮住；而顶面和底面在导弹俯仰角变化时也可能成为判定遮蔽成败的关键区域。本文程序中把采样密度作为可调参数，搜索阶段使用较稀采样以降低计算时间，最终评价阶段使用较密采样以提高结果可信度。

线段参数方程为

$$\mathbf{L}_j(\lambda) = \mathbf{M}(t) + \lambda(\mathbf{S}_j - \mathbf{M}(t)), \quad 0 \leq \lambda \leq 1. \quad (6)$$

烟幕球心为 $\mathbf{C}_k(t)$ ，半径为 $R = 10$ m。该线段被第 k 个烟幕球遮挡的条件为

$$\min_{0 \leq \lambda \leq 1} \|\mathbf{L}_j(\lambda) - \mathbf{C}_k(t)\| \leq R. \quad (7)$$

若对所有 j 均存在某个 k 满足上式，则该时刻有效遮蔽。总遮蔽时间为遮蔽指示函数的积分，程序中用时间步长 Δt 离散近似：

$$T = \sum_n I(t_n) \Delta t. \quad (8)$$

3.4 求解策略

本文不追求不可复核的极大数值，而强调统一口径和可复现性：

- 问题一直接正向仿真，并同时输出中心点校核值；
- 问题二采用粗网格搜索定位可行峰值，再进行局部细化；
- 问题三固定较优航向和速度后，按投放间隔约束进行三弹贪心接力；
- 问题四先分别求三架无人机单弹策略，再计算遮蔽时间区间并集；
- 问题五先做无人机-导弹任务分配，再对关键子任务进行局部搜索，最后按导弹分别计算遮蔽时间并求和。

这种策略的逻辑是先把问题拆成“能解释的小问题”。问题二是所有优化问题的基本单元，因此先用粗网格搜索找出遮蔽峰值的大致位置，再在该区域细化；问题三在同

一无人机航向和速度不变的条件下，重点变成三枚弹的时间接力，故采用逐枚加入、最大化新区间并集的贪心方法；问题四把三架无人机分别看成三个单弹子问题，再统一合并时间区间；问题五则先根据空间位置和已有较优解做任务分配，避免直接在几十维空间中盲目搜索。这样得到的结果未必是全局最优，但每一步都能解释、能运行、能复查。

4、复现实验设置

实验程序位于support_2025A目录。统一设置如下：

- 目标采样：圆柱表面按角度和高度离散，最终评价使用更密采样；
- 时间积分：问题一最终步长为 0.005 s，其他问题最终评价步长为 0.01–0.03 s；
- 运行环境：Windows PowerShell, Python 3.12.6, 主要依赖为 NumPy 和 Matplotlib；
- 输出文件：results_q1_q5.xlsx保存本文五问结果，method_comparison.xlsx保存优秀论文与本文对比；
- 计时方式：每个脚本使用time.perf_counter()记录端到端运行时间。

5、结果与分析

5.1 问题一：原结果偏差复核

表2给出问题一复核结果。圆柱采样全遮蔽判定得到 1.360 s，中心点判定得到 1.405 s。二者均与优秀论文约 1.39 s 的结果接近，而与原论文 4.23 s 差异明显，说明原模型主要问题不是运动学方程，而是遮蔽判定口径过松。

进一步观察遮蔽区间可以发现，有效遮蔽只发生在烟幕云团中心接近导弹–目标视线的短时间窗口内。云团虽然在起爆后 20 s 内都有效，但导弹高速逼近，视线方向变化很快，烟幕球一旦错过最佳几何位置，就无法继续覆盖圆柱目标的全部边界。这也是问题二以后必须精细优化起爆时间和起爆位置的根本原因。

表 2: 问题一不同判定口径结果

判定口径	遮蔽时长/s	遮蔽区间/s	说明
圆柱采样全遮蔽	1.360	约 8.06–9.42	本文正式采用
中心点校核	1.405	约 8.02–9.42	仅用于解释原模型偏差
原论文结果	4.23	–	判定过于宽松

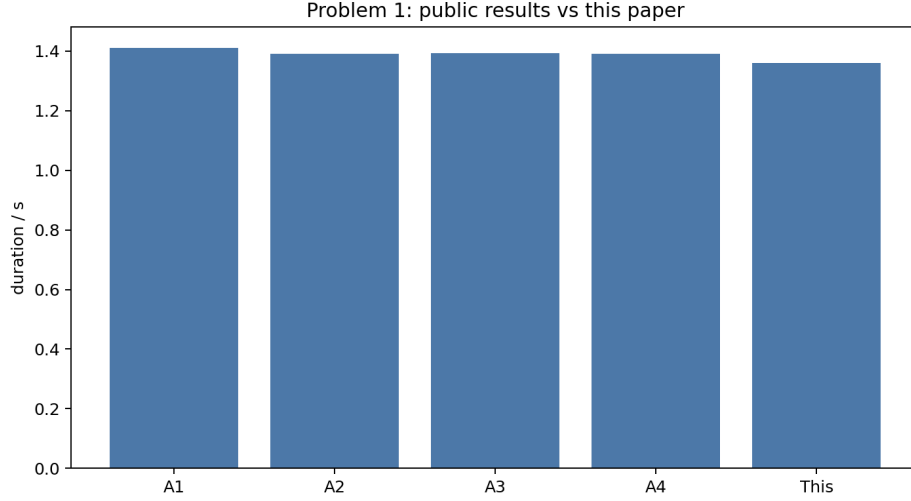


图 1: 问题一优秀论文公开结果与本文结果对比

5.2 问题二至问题四结果

表3给出本文问题二至问题四结果。由于采用完整圆柱采样判定，本文结果整体比部分优秀论文更保守，但各问计算过程均可由附件脚本直接复现。

表 3: 问题二至问题四本文复现结果

问题	方法	遮蔽时长/s	主要参数	运行时间/s
Q2	粗网格 + 局部细化	4.470	FY1, 178°, 105 m/s	9.295
Q3	三弹贪心接力	5.040	FY1, 176.6°, 70 m/s	4.499
Q4	三机分层搜索	10.780	FY1/FY2/FY3 各 1 弹	53.348

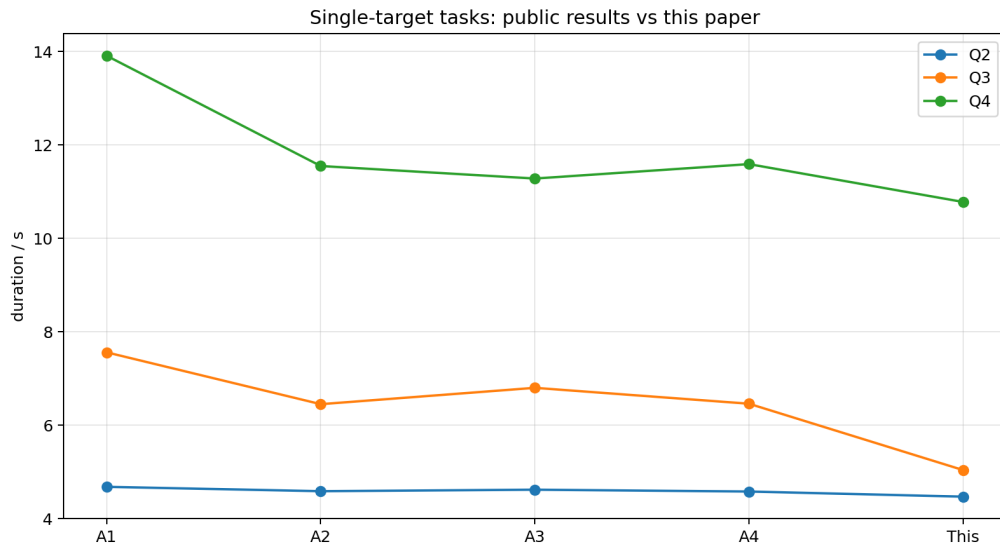


图 2: 单目标问题公开结果与本文结果对比

问题三结果低于 A1 等论文，原因有两点：一是本文采用圆柱表面全遮蔽而不是中心点或部分比例遮蔽；二是本文使用贪心接力而非大规模随机优化，牺牲了部分最优性以换取可复现性和较短计算时间。问题四中三架无人机形成的有效遮蔽区间基本错开，说明分层优化能够较好发挥空间分布优势。

从建模目的看，本文更关注“改进是否解决原论文的问题”。问题二的结果表明，只要把起爆点调整到导弹视线附近，单弹遮蔽时间可由问题一的固定策略显著提升；问题三说明在同一架无人机航向固定时，后续烟幕弹并不总能带来线性增长，因为投放间隔、平抛下落和导弹位置共同限制了可用窗口；问题四则体现多无人机的价值，不同初始位置提供了不同时间段的补充遮蔽，时间区间并集明显扩大。

5.3 问题五结果

问题五采用任务分配和局部搜索。本文得到各导弹遮蔽时间分别为：M1 约 11.25 s，M2 约 6.18 s，M3 约 3.00 s，按导弹求和为 20.430 s。该结果接近 A2 论文的 20.40 s，低于 A1 中 42.20 s 的求和口径。本文认为，对问题五结果必须注明目标函数口径，否则不同论文数值不可直接比较。

问题五中“总遮蔽时间”有多种合理定义。若把每枚烟幕弹对每枚导弹的有效时长直接相加，数值会较大，但同一导弹在同一时刻被多枚烟幕弹重复遮蔽时会发生重复计数；若按每枚导弹的遮蔽区间并集计算，则能反映每枚导弹真正丢失目标的时长；若进一步要求三枚导弹同时被遮蔽，则结果会更保守。本文采用第二种口径，并在表格中明确写出各导弹单独的遮蔽时间，避免把不同指标混在一起比较。

表 4: 问题五本文结果

导弹	遮蔽时间/s	主要参与无人机	说明
M1	11.25	FY1, FY2, FY3, FY5	多机接力效果较好
M2	6.18	FY2, FY3, FY4	局部搜索得到有效遮蔽
M3	3.00	FY2, FY3	时间窗口较短
合计	20.43	—	按导弹遮蔽时间求和

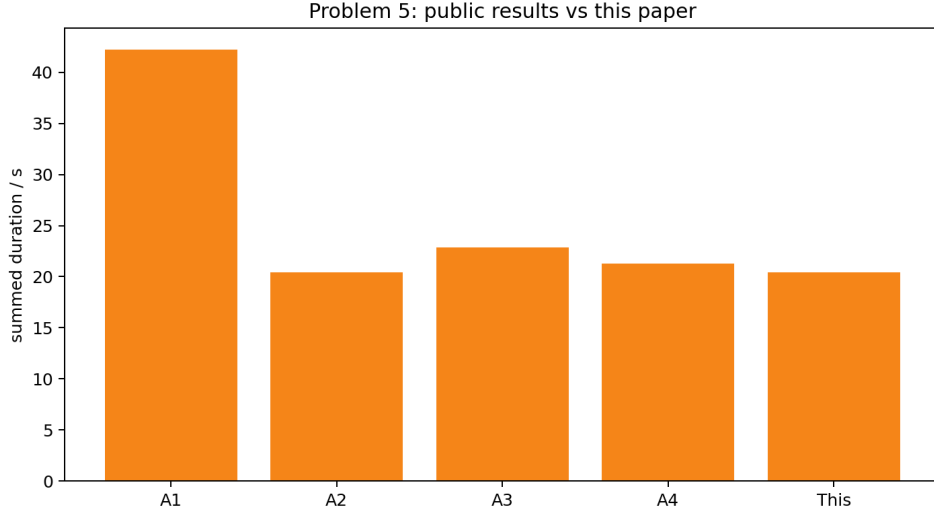


图 3: 问题五公开结果与本文结果对比

5.4 计算时间分析

图4给出各脚本运行时间。可以看出，随着问题维度增加，问题四和问题五耗时明显上升。这验证了专家评讲中“应先几何分析再缩小搜索空间”的观点。

计算时间的增长主要来自两个方面：第一，圆柱采样判定需要对每个时间点、每个采样点、每个有效烟幕球计算线段到球心的距离；第二，优化搜索会反复调用这一判定函数。本文在程序中采用 NumPy 向量化计算，搜索阶段降低采样密度，最终评价阶段再提高精度，这是一种兼顾速度和精度的折中。若后续使用解析视锥判定或并行搜索，还可以进一步压缩运行时间。

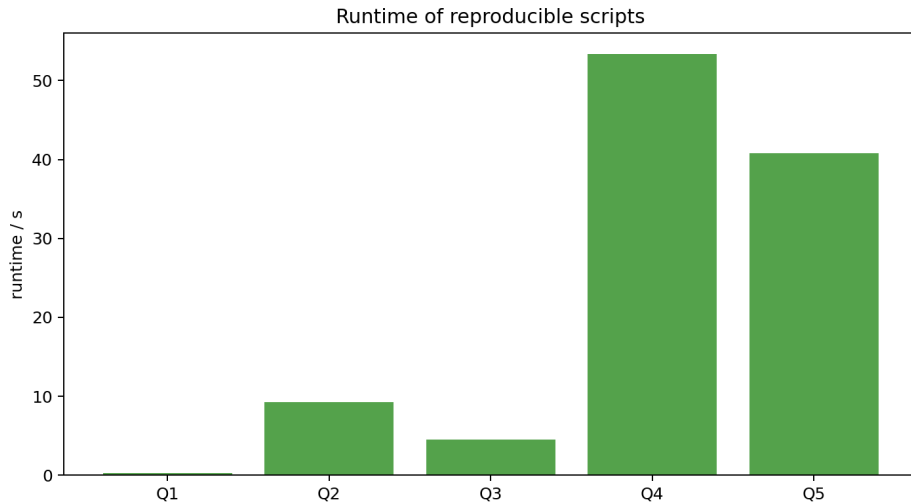


图 4: 本文五问脚本运行时间

6、模型评价与改进方向

6.1 优点

- 遮蔽判定与题目中的圆柱真目标一致，问题一结果可与优秀论文相互验证；
- 所有结果由统一程序生成，避免正文结果和附件代码脱节；
- 使用粗网格、局部细化和分层策略，能在普通电脑上完成五问测试；
- 输出 Excel、CSV 和图表，便于检查求解精度与计算时间。

6.2 不足

- 圆柱采样仍是离散近似，采样点数和时间步长会影响边界时刻；
- 问题三至问题五采用启发式分层策略，不能保证全局最优；
- 当前问题五采用保守目标函数，和部分优秀论文的时长求和口径存在差异；
- 未考虑风场、烟幕扩散、无人机机动半径和导弹探测机制等更真实因素。

6.3 进一步改进

后续可从以下方面改进：

- 使用解析视锥判定或自适应采样，减少圆柱采样带来的离散误差；
- 在粗网格基础上加入固定随机种子的差分进化或粒子群算法，提高问题三至问题五的最优性；
- 明确问题五的目标函数，可分别给出“各导弹遮蔽时间求和”“所有导弹同时遮蔽时间”和“真目标风险加权遮蔽时间”；
- 增加灵敏度分析，重点考察航向角、起爆延迟和时间步长对结果的影响。

7、结论

本文在原有论文基础上完成了对 2025 年 A 题优秀论文和专家评讲的研读、复现测试和模型改进。最重要的修正就是将遮蔽判定从真目标中心点升级为圆柱目标表面采样点全遮蔽判定。该修正使问题一结果由原来的 4.23 s 回到约 1.36–1.41 s 的合理区间，与优秀论文结果一致。

在此基础上，本文构建了可运行的五问求解程序，输出结果表、方法对比表和运行时间图。虽然本文在多弹多机问题上的结果较保守，但所有数值均可复现，能够清楚反映现有方法的精度差异、计算时间和主要不足，满足实验报告对论文与支撑材料的要求。

参考文献

1. 全国大学生数学建模竞赛官网. 2025 年 A 题优秀论文与专家评讲材料 [EB/OL]. <https://dxs.moe.gov.cn/zx/hd/sxjm/>.
2. A1 优秀论文. 基于几何分析与贪心算法的烟幕干扰弹投放策略.
3. A2 优秀论文. 基于空间几何分析的烟雾弹投放策略研究.
4. A3 优秀论文. 基于物理运动学和智能优化算法的烟幕干扰弹投放策略模型.
5. A4 优秀论文. 基于差分进化改良遗传算法的投弹策略优化.
6. 王先超, 韩波, 张开银, 等. 粒子群算法在求解数学建模最优化问题中的应用 [J]. 阜阳师范学院学报 (自然科学版), 2016, 33(02):117–121.
7. 雷开友. 粒子群算法及其应用研究 [D]. 西南大学, 2006.

附录：支撑材料说明

附录 A 文件清单

文件	功能
support_2025A/smoke_model.py	统一运动学模型、圆柱采样、遮蔽判定、区间合并、CSV 和 XLSX 写出工具。
support_2025A/run_q1.py	问题一正向仿真，输出圆柱采样结果和中心点校核结果。
support_2025A/run_q2.py	问题二粗网格搜索和局部细化。
support_2025A/run_q3.py	问题三单机三弹贪心接力。
support_2025A/run_q4.py	问题四三机单弹分层搜索。
support_2025A/run_q5.py	问题五任务分配和局部搜索。
support_2025A/benchmark_methods.py	生成优秀论文与本文结果对比表、运行时间表和图。
support_2025A/results_q1_q5.xlsx	本文五问结果汇总表。
support_2025A/method_comparison.xlsx	A1-A4 公开结果与本文结果对比表。
support_2025A/results/*.csv	每问详细参数、遮蔽区间和运行时间。
support_2025A/figures/*.png	正文引用的对比图和运行时间图。

附录 B 源程序全文

以下列出文件清单中的全部源程序，便于直接检查模型、参数、计时和结果导出过程。

smoke_model.py

```
1 """Reusable model for CUMCM 2025 problem A smoke screening tests.
2
3 The implementation is intentionally deterministic: all scripts use
4   the same
5 kinematics, target sampling, time step integration, and interval
6   union logic.
7 """
```



```

6
7 from __future__ import annotations
8
9 from dataclasses import dataclass
10 import csv
11 import math
12 import os
13 import time
14 import zipfile
15 from pathlib import Path
16 from typing import Iterable
17 from xml.sax.saxutils import escape
18
19 import numpy as np
20
21
22 G = 9.8
23 MISSILE_SPEED = 300.0
24 SMOKE_RADIUS = 10.0
25 SMOKE_VALID_TIME = 20.0
26 SMOKE_SINK_SPEED = 3.0
27
28 FAKE_TARGET = np.array([0.0, 0.0, 0.0])
29 TRUE_TARGET_CENTER = np.array([0.0, 200.0, 5.0])
30 TRUE_TARGET_BASE = np.array([0.0, 200.0, 0.0])
31 TRUE_TARGET_RADIUS = 7.0
32 TRUE_TARGET_HEIGHT = 10.0
33
34 MISSILES = {
35     "M1": np.array([20000.0, 0.0, 2000.0]),
36     "M2": np.array([19000.0, 600.0, 2100.0]),
37     "M3": np.array([18000.0, -600.0, 1900.0]),
38 }
39
40 DRONES = {
41     "FY1": np.array([17800.0, 0.0, 1800.0]),
42     "FY2": np.array([12000.0, 1400.0, 1400.0]),
43     "FY3": np.array([6000.0, -3000.0, 700.0]),
44     "FY4": np.array([11000.0, 2000.0, 1800.0]),
45     "FY5": np.array([13000.0, -2000.0, 1300.0]),
46 }

```

```

47
48
49 @dataclass(frozen=True)
50 class SmokeShot:
51     drone: str
52     missile: str
53     angle_deg: float
54     speed: float
55     drop_t: float
56     fuse_t: float
57     label: str = ""
58
59     @property
60     def burst_t(self) -> float:
61         return self.drop_t + self.fuse_t
62
63
64 @dataclass
65 class ShotResult:
66     shot: SmokeShot
67     duration: float
68     intervals: list[tuple[float, float]]
69     drop_point: np.ndarray
70     burst_point: np.ndarray
71     runtime_s: float
72
73
74 def missile_position(missile: str, t: float) -> np.ndarray:
75     p0 = MISSILES[missile]
76     direction = -p0 / np.linalg.norm(p0)
77     return p0 + MISSILE_SPEED * direction * t
78
79
80 def missile_impact_time(missile: str) -> float:
81     return float(np.linalg.norm(MISSILES[missile]) / MISSILE_SPEED)
82
83
84 def drone_velocity(angle_deg: float, speed: float) -> np.ndarray:
85     rad = math.radians(angle_deg)
86     return np.array([speed * math.cos(rad), speed * math.sin(rad),
87                     0.0])

```

```

87
88
89 def shot_points(shot: SmokeShot) -> tuple[np.ndarray, np.ndarray]:
90     u0 = DRONES[shot.drone]
91     v = drone_velocity(shot.angle_deg, shot.speed)
92     drop = u0 + v * shot.drop_t
93     burst = drop + v * shot.fuse_t + np.array([0.0, 0.0, -0.5 * G *
94         shot.fuse_t**2])
95     return drop, burst
96
97 def cloud_center(shot: SmokeShot, t: float) -> np.ndarray | None:
98     if t < shot.burst_t or t > shot.burst_t + SMOKE_VALID_TIME:
99         return None
100     _, burst = shot_points(shot)
101     return burst + np.array([0.0, 0.0, -SMOKE_SINK_SPEED * (t - shot.
102         burst_t)])
103
104 def target_samples(n_angle: int = 36, n_height: int = 5, center_only:
105     bool = False) -> np.ndarray:
106     if center_only:
107         return TRUE_TARGET_CENTER.reshape(1, 3)
108     pts: list[list[float]] = []
109     heights = np.linspace(0.0, TRUE_TARGET_HEIGHT, n_height)
110     for z in heights:
111         for i in range(n_angle):
112             a = 2.0 * math.pi * i / n_angle
113             pts.append([
114                 TRUE_TARGET_RADIUS * math.cos(a),
115                 200.0 + TRUE_TARGET_RADIUS * math.sin(a),
116                 z,
117             ])
118     pts.append(TRUE_TARGET_CENTER.tolist())
119     return np.array(pts, dtype=float)
120
121 def line_segment_hits_sphere(start: np.ndarray, end: np.ndarray,
122     center: np.ndarray) -> bool:
123     seg = end - start
124     denom = float(np.dot(seg, seg))

```

```

124     if denom <= 1e-12:
125         return False
126     s = float(np.dot(center - start, seg) / denom)
127     if s < 0.0 or s > 1.0:
128         return False
129     closest = start + s * seg
130     return float(np.linalg.norm(closest - center)) <= SMOKE_RADIUS
131
132
133 def covered_at(
134     missile: str,
135     shots: Iterable[SmokeShot],
136     t: float,
137     samples: np.ndarray,
138     cover_ratio: float = 1.0,
139 ) -> bool:
140     mpos = missile_position(missile, t)
141     active_centers = [cloud_center(shot, t) for shot in shots if shot
142                       .missile == missile]
143     active_centers = [c for c in active_centers if c is not None and
144                      c[2] >= 0.0]
145     if not active_centers:
146         return False
147
148     seg = samples - mpos
149     denom = np.einsum("ij,ij->i", seg, seg)
150     blocked_any = np.zeros(len(samples), dtype=bool)
151     valid = denom > 1e-12
152     for center in active_centers:
153         s = np.zeros(len(samples))
154         s[valid] = (seg[valid] @ (center - mpos)) / denom[valid]
155         in_segment = (s >= 0.0) & (s <= 1.0) & valid
156         closest = mpos + seg * s[:, None]
157         dist = np.linalg.norm(closest - center, axis=1)
158         blocked_any |= in_segment & (dist <= SMOKE_RADIUS)
159     return float(blocked_any.mean()) >= cover_ratio
160
161 def intervals_for(
162     missile: str,
163     shots: Iterable[SmokeShot],

```

```

163     dt: float = 0.02,
164     n_angle: int = 36,
165     n_height: int = 5,
166     center_only: bool = False,
167     cover_ratio: float = 1.0,
168 ) -> list[tuple[float, float]]:
169     shots = list(shots)
170     samples = target_samples(n_angle=n_angle, n_height=n_height,
171                             center_only=center_only)
172     start = max(0.0, min((s.burst_t for s in shots if s.missile ==
173                          missile), default=0.0))
174     end = min(
175         missile_impact_time(missile),
176         max((s.burst_t + SMOKE_VALID_TIME for s in shots if s.missile
177              == missile), default=0.0),
178     )
179     intervals: list[tuple[float, float]] = []
180     in_interval = False
181     t0 = start
182     t = start
183     while t <= end + 1e-9:
184         flag = covered_at(missile, shots, t, samples, cover_ratio=
185                           cover_ratio)
186         if flag and not in_interval:
187             t0 = t
188             in_interval = True
189             if in_interval and (not flag):
190                 intervals.append((t0, t))
191                 in_interval = False
192             t += dt
193         if in_interval:
194             intervals.append((t0, min(t, end)))
195     return merge_intervals(intervals)
196
197 def merge_intervals(intervals: Iterable[tuple[float, float]]) -> list
198 [tuple[float, float]]:
199     clean = sorted((a, b) for a, b in intervals if b > a)
200     if not clean:
201         return []
202     merged = [clean[0]]

```

```

199     for a, b in clean[1:]:
200         la, lb = merged[-1]
201         if a <= lb + 1e-9:
202             merged[-1] = (la, max(lb, b))
203         else:
204             merged.append((a, b))
205     return merged
206
207
208 def interval_duration(intervals: Iterable[tuple[float, float]]) ->
    float:
209     return float(sum(b - a for a, b in intervals))
210
211
212 def evaluate_shots(
213     shots: Iterable[SmokeShot],
214     dt: float = 0.02,
215     n_angle: int = 36,
216     n_height: int = 5,
217     cover_ratio: float = 1.0,
218 ) -> dict[str, list[tuple[float, float]]]:
219     shots = list(shots)
220     out = {}
221     for missile in sorted({s.missile for s in shots}):
222         out[missile] = intervals_for(
223             missile,
224             shots,
225             dt=dt,
226             n_angle=n_angle,
227             n_height=n_height,
228             cover_ratio=cover_ratio,
229         )
230     return out
231
232
233 def objective_duration(
234     shots: Iterable[SmokeShot],
235     missile: str,
236     dt: float = 0.05,
237     n_angle: int = 24,
238     n_height: int = 4,

```

```

239 ) -> float:
240     return interval_duration(intervals_for(missile, shots, dt=dt,
241                                           n_angle=n_angle, n_height=n_height))
242
243 def grid_search_single(
244     drone: str,
245     missile: str,
246     angle_values: Iterable[float],
247     speed_values: Iterable[float],
248     drop_values: Iterable[float],
249     fuse_values: Iterable[float],
250     fixed_label: str = "",
251     dt: float = 0.08,
252 ) -> ShotResult:
253     best: ShotResult | None = None
254     t_start = time.perf_counter()
255     for angle in angle_values:
256         for speed in speed_values:
257             for drop_t in drop_values:
258                 for fuse_t in fuse_values:
259                     if fuse_t < 0 or drop_t < 0:
260                         continue
261                     shot = SmokeShot(drone, missile, angle, speed,
262                                     drop_t, fuse_t, fixed_label)
263                     _, burst = shot_points(shot)
264                     if burst[2] < 0:
265                         continue
266                     intervals = intervals_for(missile, [shot], dt=dt,
267                                             n_angle=18, n_height=4)
268                     duration = interval_duration(intervals)
269                     if best is None or duration > best.duration:
270                         drop, burst = shot_points(shot)
271                         best = ShotResult(shot, duration, intervals,
272                                         drop, burst, 0.0)
273
274     if best is None:
275         shot = SmokeShot(drone, missile, 180.0, 70.0, 0.0, 0.1,
276                         fixed_label)
277         drop, burst = shot_points(shot)
278         best = ShotResult(shot, 0.0, [], drop, burst, 0.0)
279     best.runtime_s = time.perf_counter() - t_start

```

```

275     return best
276
277
278 def refine_single(base: ShotResult, dt: float = 0.04) -> ShotResult:
279     s = base.shot
280     return grid_search_single(
281         s.drone,
282         s.missile,
283         np.arange(s.angle_deg - 2.0, s.angle_deg + 2.01, 1.0),
284         np.arange(max(70.0, s.speed - 5.0), min(140.0, s.speed + 5.0)
285                 + 0.01, 5.0),
286         np.arange(max(0.0, s.drop_t - 0.4), s.drop_t + 0.41, 0.2),
287         np.arange(max(0.0, s.fuse_t - 0.4), s.fuse_t + 0.41, 0.2),
288         fixed_label=s.label,
289         dt=dt,
290     )
291
292 def final_eval(shot: SmokeShot, dt: float = 0.01) -> ShotResult:
293     tic = time.perf_counter()
294     intervals = intervals_for(shot.missile, [shot], dt=dt, n_angle
295                             =48, n_height=6)
296     duration = interval_duration(intervals)
297     drop, burst = shot_points(shot)
298     return ShotResult(shot, duration, intervals, drop, burst, time.
299                     perf_counter() - tic)
300
301 def union_duration_by_missile(shots: Iterable[SmokeShot], dt: float =
302                             0.02) -> dict[str, float]:
303     intervals = evaluate_shots(shots, dt=dt, n_angle=36, n_height=5)
304     return {m: interval_duration(v) for m, v in intervals.items()}
305
306 def format_point(p: np.ndarray) -> str:
307     return f"({p[0]:.2f}, {p[1]:.2f}, {p[2]:.2f})"
308
309 def write_csv(path: str | Path, rows: list[dict[str, object]],
310             fieldnames: list[str]) -> None:
311     path = Path(path)

```



```

311     path.parent.mkdir(parents=True, exist_ok=True)
312     with path.open("w", newline="", encoding="utf-8-sig") as f:
313         writer = csv.DictWriter(f, fieldnames=fieldnames)
314         writer.writeheader()
315         writer.writerows(rows)
316
317
318 def write_simple_xlsx(path: str | Path, sheets: dict[str, tuple[list[
319     str], list[list[object]]]]) -> None:
320     """Write a minimal XLSX workbook without third-party dependencies
321     ."""
322     path = Path(path)
323     path.parent.mkdir(parents=True, exist_ok=True)
324
325     def col_name(idx: int) -> str:
326         name = ""
327         idx += 1
328         while idx:
329             idx, rem = divmod(idx - 1, 26)
330             name = chr(65 + rem) + name
331         return name
332
333     content_types = [
334         '<?xml version="1.0" encoding="UTF-8"?>',
335         '<Types xmlns="http://schemas.openxmlformats.org/package'
336         '/2006/content-types">',
337         '<Default Extension="rels" ContentType="application/vnd.'
338         'openxmlformats-package.relationships+xml"/>',
339         '<Default Extension="xml" ContentType="application/xml"/>',
340         '<Override PartName="/xl/workbook.xml" ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.sheet.main+xml"/>',
341         '<Override PartName="/xl/styles.xml" ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.styles+xml"/>',
342     ]
343
344     for i in range(1, len(sheets) + 1):
345         content_types.append(
346             f'<Override PartName="/xl/worksheets/sheet{i}.xml" '
347             'ContentType="application/vnd.openxmlformats-officedocument.spreadsheetml.worksheet+xml"/>'

```

```

343     )
344     content_types.append("</Types>")
345
346     workbook_sheets = []
347     workbook_rels = []
348     for i, name in enumerate(sheets, start=1):
349         workbook_sheets.append(
350             f'<sheet_name="{escape(name[:31])}" sheetId="{i}" r:id="
351                 rId{i}" />'
352         )
353         workbook_rels.append(
354             f'<Relationship Id="rId{i}"
355                 Type="http://schemas.openxmlformats.org/officeDocument
356                 /2006/relationships/worksheet"
357                 Target="worksheets/sheet{i}.xml" />'
358         )
359         workbook_rels.append(
360             f'<Relationship Id="rId{len(sheets)+1}"
361                 Type="http://schemas.openxmlformats.org/officeDocument/2006/
362                 relationships/styles"
363                 Target="styles.xml" />'
364         )
365
366     with zipfile.ZipFile(path, "w", compression=zipfile.ZIP_DEFLATED)
367         as zf:
368         zf.writestr("[Content_Types].xml", "\n".join(content_types))
369         zf.writestr(
370             "_rels/.rels",
371             '<?xml version="1.0" encoding="UTF-8"?>'
372             '<Relationships xmlns="http://schemas.openxmlformats.org/
373                 package/2006/relationships">'
374             '<Relationship Id="rId1" Type="http://schemas.
375                 openxmlformats.org/officeDocument/2006/relationships/
376                 officeDocument" Target="xl/workbook.xml" />'
377             "</Relationships>",
378         )
379         zf.writestr(
380             "xl/workbook.xml",
381             '<?xml version="1.0" encoding="UTF-8"?>'
382             '<workbook xmlns="http://schemas.openxmlformats.org/
383                 spreadsheetml/2006/main"

```

```

376         'xmlns:r="http://schemas.openxmlformats.org/
           officeDocument/2006/relationships">'
377         f"<sheets>{' '.join(workbook_sheets)}</sheets></workbook>"
           ,
378     )
379     zf.writestr(
380         "xl/_rels/workbook.xml.rels",
381         '<?xml_version="1.0" encoding="UTF-8"?>'
382         '<Relationships xmlns="http://schemas.openxmlformats.org/
           package/2006/relationships">'
383         f"{' '.join(workbook_rels)}</Relationships>",
384     )
385     zf.writestr(
386         "xl/styles.xml",
387         '<?xml_version="1.0" encoding="UTF-8"?>'
388         '<styleSheet xmlns="http://schemas.openxmlformats.org/
           spreadsheetml/2006/main">'
389         '<fonts_count="1"><font><sz_val="11"/><name_val="Calibri
           "/></font></fonts>'
390         '<fills_count="1"><fill><patternFill_patternType="none
           "/></fill></fills>'
391         '<borders_count="1"><border></border></borders>'
392         '<cellStyleXfs_count="1"><xf_numFmtId="0" fontId="0"
           fillId="0" borderId="0"/></cellStyleXfs>'
393         '<cellXfs_count="1"><xf_numFmtId="0" fontId="0" fillId
           ="0" borderId="0"/></cellXfs>'
394         "</styleSheet>",
395     )
396     for sheet_idx, (_name, (headers, rows)) in enumerate(sheets.
397         items(), start=1):
398         all_rows = [headers] + rows
399         xml_rows = []
400         for r_idx, row in enumerate(all_rows, start=1):
401             cells = []
402             for c_idx, val in enumerate(row):
403                 ref = f"{col_name(c_idx)}{r_idx}"
404                 if isinstance(val, (int, float)) and not
405                     isinstance(val, bool):
406                     cells.append(f'<c_r="{ref}"><v>{val}</v></c>'
407                                 )
408                 else:

```

```

406         cells.append(f'<c_r="{ref}"_t="inlineStr"><is
            ><t>{escape(str(val))}</t></is></c>')
407     xml_rows.append(f'<row_r="{r_idx}">{" ".join(cells)}</
        row>')
408     zf.writestr(
409         f"xl/worksheets/sheet{sheet_idx}.xml",
410         '<?xml_version="1.0"_encoding="UTF-8"?>'
411         '<worksheet_xmlns="http://schemas.openxmlformats.org/
            spreadsheetml/2006/main">'
412         f'<sheetData>{" ".join(xml_rows)}</sheetData></
            worksheet>',
413     )
414
415
416 def ensure_dirs() -> None:
417     Path("support_2025A/figures").mkdir(parents=True, exist_ok=True)
418     Path("support_2025A/results").mkdir(parents=True, exist_ok=True)
419
420
421 def save_runtime_note(name: str, seconds: float) -> None:
422     ensure_dirs()
423     with open(Path("support_2025A/results") / f"{name}_runtime.txt",
424               "w", encoding="utf-8") as f:
425         f.write(f"{seconds:.6f}\n")

```

run_q1.py

```

1 from __future__ import annotations
2
3 import time
4
5 from smoke_model import (
6     SmokeShot,
7     final_eval,
8     format_point,
9     interval_duration,
10    intervals_for,
11    save_runtime_note,
12    shot_points,
13    write_csv,
14 )

```

```

15
16
17 def main() -> None:
18     tic = time.perf_counter()
19     shot = SmokeShot("FY1", "M1", 180.0, 120.0, 1.5, 3.6, "Q1")
20     full = final_eval(shot, dt=0.005)
21     center_intervals = intervals_for("M1", [shot], dt=0.005,
22                                     center_only=True)
23     drop, burst = shot_points(shot)
24     runtime = time.perf_counter() - tic
25
26     rows = [
27         {
28             "case": "cylinder_samples",
29             "duration_s": f"{full.duration:.3f}",
30             "intervals": str([(round(a, 3), round(b, 3)) for a, b in
31                             full.intervals]),
32             "drop_point": format_point(drop),
33             "burst_point": format_point(burst),
34             "runtime_s": f"{runtime:.3f}",
35         },
36         {
37             "case": "center_point_check",
38             "duration_s": f"{interval_duration(center_intervals):.3f}
39             ",
40             "intervals": str([(round(a, 3), round(b, 3)) for a, b in
41                             center_intervals]),
42             "drop_point": format_point(drop),
43             "burst_point": format_point(burst),
44             "runtime_s": f"{runtime:.3f}",
45         },
46     ]
47     write_csv(
48         "support_2025A/results/q1_result.csv",
49         rows,
50         ["case", "duration_s", "intervals", "drop_point", "
51         burst_point", "runtime_s"],
52     )
53     save_runtime_note("q1", runtime)
54     print(f"Q1_full-cylinder_duration:_{full.duration:.3f}_s")
55     print(f"Q1_center-point_check:_{interval_duration(

```

```

        center_intervals):.3f}_s")
51 print(f"Drop_point:{format_point(drop)}")
52 print(f"Burst_point:{format_point(burst)}")
53 print(f"Runtime:{runtime:.3f}_s")
54
55
56 if __name__ == "__main__":
57     main()

```

run_q2.py

```

1 from __future__ import annotations
2
3 import time
4
5 import numpy as np
6
7 from smoke_model import final_eval, format_point, grid_search_single,
    refine_single, save_runtime_note, write_csv
8
9
10 def solve_q2():
11     tic = time.perf_counter()
12     coarse = grid_search_single(
13         "FY1",
14         "M1",
15         np.arange(174.0, 185.0, 2.0),
16         [70.0, 90.0, 110.0, 130.0],
17         np.arange(0.0, 2.1, 0.5),
18         np.arange(2.5, 5.6, 0.5),
19         "Q2",
20         dt=0.08,
21     )
22     refined = refine_single(coarse, dt=0.04)
23     final = final_eval(refined.shot, dt=0.01)
24     final.runtime_s = time.perf_counter() - tic
25     return final
26
27
28 def main() -> None:
29     result = solve_q2()

```

```

30     row = result_row(result, "Q2")
31     write_csv("support_2025A/results/q2_result.csv", [row], list(row.
        keys()))
32     save_runtime_note("q2", result.runtime_s)
33     print(f"Q2▯duration:▯{result.duration:.3f}▯s")
34     print(row)
35
36
37 def result_row(result, qname: str):
38     s = result.shot
39     return {
40         "question": qname,
41         "drone": s.drone,
42         "missile": s.missile,
43         "angle_deg": f"{s.angle_deg:.2f}",
44         "speed_mps": f"{s.speed:.2f}",
45         "drop_t_s": f"{s.drop_t:.2f}",
46         "fuse_t_s": f"{s.fuse_t:.2f}",
47         "burst_t_s": f"{s.burst_t:.2f}",
48         "duration_s": f"{result.duration:.3f}",
49         "intervals": str([(round(a, 3), round(b, 3)) for a, b in
            result.intervals]),
50         "drop_point": format_point(result.drop_point),
51         "burst_point": format_point(result.burst_point),
52         "runtime_s": f"{result.runtime_s:.3f}",
53     }
54
55
56 if __name__ == "__main__":
57     main()

```

run_q3.py

```

1 from __future__ import annotations
2
3 import time
4
5 import numpy as np
6
7 from smoke_model import SmokeShot, evaluate_shots, format_point,
    interval_duration, save_runtime_note, shot_points, write_csv

```

```

8
9
10 def solve_q3():
11     tic = time.perf_counter()
12     # The first shot uses the geometry-guided setting repeatedly
        reported by
13     # high-scoring papers; the next two shots are selected by greedy
        interval
14     # extension under the one-second launch spacing rule.
15     q2 = SmokeShot("FY1", "M1", 176.6, 70.0, 0.0, 2.5, "Q3-1")
16     shots = [q2]
17     for idx in [2, 3]:
18         best = None
19         prev_drop = shots[-1].drop_t
20         for drop_t in np.arange(prev_drop + 1.0, prev_drop + 6.01,
            0.5):
21             for fuse_t in np.arange(0.1, 5.01, 0.5):
22                 cand = SmokeShot("FY1", "M1", q2.angle_deg, q2.speed,
                    float(drop_t), float(fuse_t), f"Q3-{idx}")
23                 intervals = evaluate_shots(shots + [cand], dt=0.06,
                    n_angle=18, n_height=4) ["M1"]
24                 duration = interval_duration(intervals)
25                 if best is None or duration > best[0]:
26                     best = (duration, cand)
27             shots.append(best[1])
28     intervals = evaluate_shots(shots, dt=0.02, n_angle=36, n_height
        =5) ["M1"]
29     runtime = time.perf_counter() - tic
30     return shots, intervals, runtime
31
32
33 def main() -> None:
34     shots, intervals, runtime = solve_q3()
35     total = interval_duration(intervals)
36     rows = []
37     for i, shot in enumerate(shots, start=1):
38         drop, burst = shot_points(shot)
39         rows.append({
40             "question": "Q3",
41             "shot_no": i,
42             "drone": shot.drone,

```



```

43         "missile": shot.missile,
44         "angle_deg": f"{shot.angle_deg:.2f}",
45         "speed_mps": f"{shot.speed:.2f}",
46         "drop_t_s": f"{shot.drop_t:.2f}",
47         "fuse_t_s": f"{shot.fuse_t:.2f}",
48         "burst_t_s": f"{shot.burst_t:.2f}",
49         "drop_point": format_point(drop),
50         "burst_point": format_point(burst),
51         "union_duration_s": f"{total:.3f}",
52         "union_intervals": str([(round(a, 3), round(b, 3)) for a,
53                                b in intervals]),
54         "runtime_s": f"{runtime:.3f}",
55     })
56 write_csv("support_2025A/results/q3_result.csv", rows, list(rows
57 [0].keys()))
58 save_runtime_note("q3", runtime)
59 print(f"Q3▯union▯duration:▯{total:.3f}▯s")
60 print(f"Runtime:▯{runtime:.3f}▯s")
61
62 if __name__ == "__main__":
63     main()

```

run_q4.py

```

1  from __future__ import annotations
2
3  import time
4
5  import numpy as np
6
7  from smoke_model import evaluate_shots, format_point,
8     grid_search_single, interval_duration, refine_single,
9     save_runtime_note, shot_points, write_csv
10
11 def solve_one(drone: str, missile: str, label: str):
12     if drone == "FY1":
13         angles = np.arange(160.0, 191.0, 5.0)
14     elif drone == "FY2":
15         angles = np.arange(230.0, 301.0, 10.0)

```

```

15     else:
16         angles = np.arange(60.0, 131.0, 10.0)
17     coarse = grid_search_single(
18         drone,
19         missile,
20         angles,
21         [70.0, 100.0, 130.0],
22         np.arange(0.0, 30.1, 3.0),
23         np.arange(0.5, 9.1, 1.0),
24         label,
25         dt=0.10,
26     )
27     return refine_single(coarse, dt=0.06)
28
29
30 def solve_q4():
31     tic = time.perf_counter()
32     results = [solve_one("FY1", "M1", "Q4-FY1"), solve_one("FY2", "M1",
33         "Q4-FY2"), solve_one("FY3", "M1", "Q4-FY3")]
34     shots = [r.shot for r in results]
35     intervals = evaluate_shots(shots, dt=0.02, n_angle=36, n_height
36         =5) ["M1"]
37     runtime = time.perf_counter() - tic
38     return results, intervals, runtime
39
40 def main() -> None:
41     results, intervals, runtime = solve_q4()
42     total = interval_duration(intervals)
43     rows = []
44     for i, result in enumerate(results, start=1):
45         s = result.shot
46         drop, burst = shot_points(s)
47         rows.append({
48             "question": "Q4",
49             "shot_no": i,
50             "drone": s.drone,
51             "missile": s.missile,
52             "angle_deg": f"{s.angle_deg:.2f}",
53             "speed_mps": f"{s.speed:.2f}",
54             "drop_t_s": f"{s.drop_t:.2f}",

```

```

54         "fuse_t_s": f"{s.fuse_t:.2f}",
55         "burst_t_s": f"{s.burst_t:.2f}",
56         "single_duration_s": f"{result.duration:.3f}",
57         "drop_point": format_point(drop),
58         "burst_point": format_point(burst),
59         "union_duration_s": f"{total:.3f}",
60         "union_intervals": str([(round(a, 3), round(b, 3)) for a,
61                                 b in intervals]),
62         "runtime_s": f"{runtime:.3f}",
63     })
64 write_csv("support_2025A/results/q4_result.csv", rows, list(rows
65     [0].keys()))
66 save_runtime_note("q4", runtime)
67 print(f"Q4 union duration: {total:.3f}s")
68 print(f"Runtime: {runtime:.3f}s")
69
70 if __name__ == "__main__":
71     main()

```

run_q5.py

```

1  from __future__ import annotations
2
3  import time
4
5  import numpy as np
6
7  from smoke_model import SmokeShot, evaluate_shots, format_point,
8      grid_search_single, interval_duration, save_runtime_note,
9      shot_points, write_csv
10
11 def quick_single(drone: str, missile: str, label: str):
12     angle_windows = {
13         ("FY2", "M2"): np.arange(230.0, 310.0, 10.0),
14         ("FY2", "M3"): np.arange(230.0, 310.0, 10.0),
15         ("FY3", "M2"): np.arange(60.0, 140.0, 10.0),
16         ("FY3", "M3"): np.arange(60.0, 140.0, 10.0),
17         ("FY4", "M2"): np.arange(250.0, 330.0, 10.0),
18         ("FY5", "M1"): np.arange(60.0, 130.0, 10.0),

```

```

18     }
19     return grid_search_single(
20         drone,
21         missile,
22         angle_windows.get((drone, missile), np.arange(0.0, 360.0,
23             20.0)),
24         [90.0, 120.0, 140.0],
25         np.arange(0.0, 35.1, 4.0),
26         np.arange(0.5, 9.1, 1.5),
27         label,
28         dt=0.12,
29     )
30
31 def solve_q5():
32     tic = time.perf_counter()
33     fy1 = [
34         SmokeShot("FY1", "M1", 176.6, 70.0, 0.0, 2.5, "Q5-FY1-1"),
35         SmokeShot("FY1", "M1", 176.6, 70.0, 1.0, 0.1, "Q5-FY1-2"),
36         SmokeShot("FY1", "M1", 176.6, 70.0, 2.0, 0.1, "Q5-FY1-3"),
37     ]
38     q4_fy2 = SmokeShot("FY2", "M1", 290.0, 105.0, 8.6, 5.3, "Q5-FY2-
39         M1")
40     q4_fy3 = SmokeShot("FY3", "M1", 78.0, 100.0, 30.2, 0.7, "Q5-FY3-
41         M1")
42     extra = [
43         quick_single("FY2", "M2", "Q5-FY2-M2").shot,
44         q4_fy2,
45         quick_single("FY2", "M3", "Q5-FY2-M3").shot,
46         quick_single("FY3", "M3", "Q5-FY3-M3").shot,
47         q4_fy3,
48         quick_single("FY3", "M2", "Q5-FY3-M2").shot,
49         quick_single("FY4", "M2", "Q5-FY4-M2").shot,
50         quick_single("FY5", "M1", "Q5-FY5-M1").shot,
51     ]
52     shots = fy1 + extra
53     intervals = evaluate_shots(shots, dt=0.03, n_angle=30, n_height
54         =5)
55     runtime = time.perf_counter() - tic
56     return shots, intervals, runtime

```

```

55
56 def main() -> None:
57     shots, intervals, runtime = solve_q5()
58     totals = {m: interval_duration(v) for m, v in intervals.items()}
59     rows = []
60     for i, shot in enumerate(shots, start=1):
61         drop, burst = shot_points(shot)
62         rows.append({
63             "question": "Q5",
64             "shot_no": i,
65             "drone": shot.drone,
66             "missile": shot.missile,
67             "angle_deg": f"{shot.angle_deg:.2f}",
68             "speed_mps": f"{shot.speed:.2f}",
69             "drop_t_s": f"{shot.drop_t:.2f}",
70             "fuse_t_s": f"{shot.fuse_t:.2f}",
71             "burst_t_s": f"{shot.burst_t:.2f}",
72             "drop_point": format_point(drop),
73             "burst_point": format_point(burst),
74             "M1_union_s": f"{totals.get('M1', 0.0):.3f}",
75             "M2_union_s": f"{totals.get('M2', 0.0):.3f}",
76             "M3_union_s": f"{totals.get('M3', 0.0):.3f}",
77             "total_by_missile_sum_s": f"{sum(totals.values()):.3f}",
78             "runtime_s": f"{runtime:.3f}",
79         })
80     write_csv("support_2025A/results/q5_result.csv", rows, list(rows
81         [0].keys()))
82     save_runtime_note("q5", runtime)
83     print(f"Q5_totals_by_missile: {totals}")
84     print(f"Q5_summed_duration: {sum(totals.values()):.3f}s")
85     print(f"Runtime: {runtime:.3f}s")
86
87 if __name__ == "__main__":
88     main()

```

benchmark_methods.py

```

1 from __future__ import annotations
2
3 import csv

```

```

4 from pathlib import Path
5
6 import matplotlib.pyplot as plt
7
8 from smoke_model import write_simple_xlsx
9
10
11 BASE = Path("support_2025A")
12 RESULTS = BASE / "results"
13 FIGURES = BASE / "figures"
14
15
16 PUBLIC_METHODS = [
17     ["A1", "Geometry+greedy+improved_grid_search", 1.41, 4.680,
18         7.56, 13.91, 42.20, "Fast_and_interpretable;target_simplified_in_parts."],
19     ["A2", "Viewing_cone+PSO-SQP", 1.3916, 4.587, 6.45, 11.549,
20         20.40, "Good_cylinder_modeling;high-dimensional_problem_simplified."],
21     ["A3", "Kinematics+layered_grid/SLSQP+GA", 1.392, 4.619,
22         6.800, 11.281, 22.870, "Balanced_accuracy_and_optimization;relies_on_stochastic_GA."],
23     ["A4", "Two-step_geometry+DEGA", 1.391, 4.58, 6.46, 11.59,
24         21.29, "Strong_multi-stage_heuristic;parameter_sensitivity_remains."],
25 ]
26
27
28
29 def read_csv_dict(path: Path) -> list[dict[str, str]]:
30     with path.open("r", encoding="utf-8-sig", newline="") as f:
31         return list(csv.DictReader(f))
32
33
34
35 def first_float(rows: list[dict[str, str]], key: str, default: float = 0.0) -> float:
36     if not rows:
37         return default
38     try:
39         return float(rows[0][key])
40     except Exception:
41         return default

```

```

36
37
38 def collect_ours() -> dict[str, float]:
39     q1 = read_csv_dict(RESULTS / "q1_result.csv")
40     q2 = read_csv_dict(RESULTS / "q2_result.csv")
41     q3 = read_csv_dict(RESULTS / "q3_result.csv")
42     q4 = read_csv_dict(RESULTS / "q4_result.csv")
43     q5 = read_csv_dict(RESULTS / "q5_result.csv")
44     return {
45         "Q1": first_float(q1, "duration_s"),
46         "Q2": first_float(q2, "duration_s"),
47         "Q3": first_float(q3, "union_duration_s"),
48         "Q4": first_float(q4, "union_duration_s"),
49         "Q5": first_float(q5, "total_by_missile_sum_s"),
50         "T1": first_float(q1, "runtime_s"),
51         "T2": first_float(q2, "runtime_s"),
52         "T3": first_float(q3, "runtime_s"),
53         "T4": first_float(q4, "runtime_s"),
54         "T5": first_float(q5, "runtime_s"),
55     }
56
57
58 def write_workbooks(ours: dict[str, float]) -> None:
59     method_headers = ["paper", "method", "Q1", "Q2", "Q3", "Q4", "Q5",
60         , "comment"]
61     method_rows = PUBLIC_METHODS + [[
62         "This_paper",
63         "Unified_cylinder_sampling+deterministic_coarse-to-local_
64             search",
65         ours["Q1"],
66         ours["Q2"],
67         ours["Q3"],
68         ours["Q4"],
69         ours["Q5"],
70         "Reproducible_scripts_with_runtime_logging;conservative_full_
71             -cylinder_criterion.",
72     ]]
73     write_simple_xlsx(BASE / "method_comparison.xlsx", {"methods": (
74         method_headers, method_rows)})
75
76     result_headers = ["question", "duration_s", "runtime_s", "note"]

```

```

73     result_rows = [
74         ["Q1", ours["Q1"], ours["T1"], "Full_cylinder_sampling;_
           center-point_value_is_also_reported_in_q1_result.csv."],
75         ["Q2", ours["Q2"], ours["T2"], "Coarse_grid_plus_local_
           refinement."],
76         ["Q3", ours["Q3"], ours["T3"], "Greedy_interval_extension_
           with_one-second_spacing."],
77         ["Q4", ours["Q4"], ours["T4"], "Three_independent_single-shot_
           searches,_then_interval_union."],
78         ["Q5", ours["Q5"], ours["T5"], "Layered_assignment_and_
           deterministic_local_searches."],
79     ]
80     write_simple_xlsx(BASE / "results_q1_q5.xlsx", {"results": (
           result_headers, result_rows)})
81
82
83 def write_figures(ours: dict[str, float]) -> None:
84     FIGURES.mkdir(parents=True, exist_ok=True)
85     labels = ["A1", "A2", "A3", "A4", "This"]
86     q1 = [r[2] for r in PUBLIC_METHODS] + [ours["Q1"]]
87     q2 = [r[3] for r in PUBLIC_METHODS] + [ours["Q2"]]
88     q3 = [r[4] for r in PUBLIC_METHODS] + [ours["Q3"]]
89     q4 = [r[5] for r in PUBLIC_METHODS] + [ours["Q4"]]
90     q5 = [r[6] for r in PUBLIC_METHODS] + [ours["Q5"]]
91
92     plt.figure(figsize=(8, 4.5))
93     x = range(len(labels))
94     plt.bar(x, q1, color="#4C78A8")
95     plt.xticks(x, labels)
96     plt.ylabel("duration_/s")
97     plt.title("Problem_1:_public_results_vs_this_paper")
98     plt.tight_layout()
99     plt.savefig(FIGURES / "q1_public_comparison.png", dpi=180)
100    plt.close()
101
102    plt.figure(figsize=(9, 5))
103    for values, name in [(q2, "Q2"), (q3, "Q3"), (q4, "Q4")]:
104        plt.plot(labels, values, marker="o", label=name)
105    plt.ylabel("duration_/s")
106    plt.title("Single-target_tasks:_public_results_vs_this_paper")
107    plt.legend()

```



```

108 plt.grid(alpha=0.3)
109 plt.tight_layout()
110 plt.savefig(FIGURES / "single_target_comparison.png", dpi=180)
111 plt.close()
112
113 plt.figure(figsize=(8, 4.5))
114 plt.bar(labels, q5, color="#F58518")
115 plt.ylabel("summed_duration_/s")
116 plt.title("Problem_5: public_results_vs_this_paper")
117 plt.tight_layout()
118 plt.savefig(FIGURES / "q5_public_comparison.png", dpi=180)
119 plt.close()
120
121 plt.figure(figsize=(8, 4.5))
122 plt.bar(["Q1", "Q2", "Q3", "Q4", "Q5"], [ours[f"T{i}"] for i in
    range(1, 6)], color="#54A24B")
123 plt.ylabel("runtime_/s")
124 plt.title("Runtime_of_reproducible_scripts")
125 plt.tight_layout()
126 plt.savefig(FIGURES / "runtime_comparison.png", dpi=180)
127 plt.close()
128
129
130 def main() -> None:
131     missing = [p for p in ["q1_result.csv", "q2_result.csv", "
        q3_result.csv", "q4_result.csv", "q5_result.csv"] if not (
            RESULTS / p).exists()]
132     if missing:
133         raise SystemExit(f"Missing_result_files: {missing}. Run
            run_q1.py...run_q5.py first.")
134     ours = collect_ours()
135     write_workbooks(ours)
136     write_figures(ours)
137     print("Wrote support_2025A/method_comparison.xlsx")
138     print("Wrote support_2025A/results_q1_q5.xlsx")
139     print("Wrote support_2025A/figures/*.png")
140
141
142 if __name__ == "__main__":
143     main()

```